ADS T1 ASSIGNMENT REPORT

UCE2021507 - Rajasi Barapatre
UCE2021510 - Sanika Chavan
UCE2021547 - Ketaki Paranjape

GOOGLE PLAY STORE

IMPORT STATEMENTS

```
import zipfile
import os
from google.colab import files
from google.colab import drive
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

LOADING DATASET

```
! pip install -q kaggle
files.upload()
drive.mount('/content/drive',force_remount=True)
!cp '/content/kaggle.json' '/content'
os.environ['KAGGLE_CONFIG_DIR']="/content"
!chmod 600 /content/kaggle.json
!kaggle datasets download -d gauthamp10/google-playstore-apps
zip_ref1=zipfile.ZipFile('/content/google-playstore-apps.zip','r')
zip_ref1.extractall('/tmp')
zip_ref1.close()
os.listdir('/tmp')
data1=pd.read_csv('/tmp/Google-Playstore.csv',encoding='latin')
```

ANALYSING DATA

data1.shape
(2312944, 24)

data1.columns
Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count',
'Maximum Installs', 'Free', 'Price', 'Currency', 'Size',
'Minimum Android', 'Developer Id', 'Content Rating', 'Ad Supported',
'In App Purchases', 'Editors Choice', 'Released_Year', 'Updated_Year'],
dtype='object')

The dataset consists of 23,12,944 rows and 24 columns.

data1.head()



	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs	Maximum Installs	Free	Price	...	Developer Website	Developer Email	Released	Last Updated	Content Rating	Privacy Policy	Ad Supported	In App Purchases	Editors Choice	Scraped Time
0	Gakondo	com.ishakwe.gakondo	Adventure	0.0	0.0	10+	10.0	15	True	0.0	...	https://benilytzibyose.tk/#/	jean21101999@gmail.com	Feb 26, 2020	Feb 26, 2020	Everyone	https://benilytzibyose.tk/projects/	False	False	False	2021-06-15 20:19:35
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0	7662	True	0.0	...	https://webserveis.netlify.app/	webserveis@gmail.com	May 21, 2020	May 06, 2021	Everyone	https://dev4phones.wordpress.com/licencia-de-uso/	True	False	False	2021-06-15 20:19:35
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0	58	True	0.0	...	NaN	vnacrewit@gmail.com	Aug 9, 2019	Aug 19, 2019	Everyone	https://www.vietnamairlines.com/vn/en/terms-an...	False	False	False	2021-06-15 20:19:35
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0	19	True	0.0	...	http://www.climatesmarttech.com/	climatesmarttech2@gmail.com	Sep 10, 2018	Oct 13, 2018	Everyone	NaN	True	False	False	2021-06-15 20:19:35
4	GROW.me	com.horodyski.grower	Tools	0.0	0.0	100+	100.0	478	True	0.0	...	http://www.horodyski.com.pl	rmilekhorodyski@gmail.com	Feb 21, 2020	Nov 12, 2018	Everyone	http://www.horodyski.com.pl	False	False	False	2021-06-15 20:19:35

5 rows × 24 columns

	App Name	App Id	Category	Rating	Rating Count	Installs	Minimum Installs
0	Gakondo	com.ishakwe.gakondo	Adventure	0.0	0.0	10+	10.0
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	5,000+	5000.0
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	50+	50.0
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	10+	10.0
4	GROW.me	com.horodyski.grower	Tools	0.0	0.0	100+	100.0

1. App Name - Name of the App
2. App Id - An unique ID for each Application
3. Category - Each application belongs to certain category like Education, Productivity etc.
4. Rating - The average rating given to the application by the users.
5. Rating count - The number of users who have given the rating.
6. Installs - The number of installations made of the application by the users.
7. Minimum Installs - The minimum number of installations made.
8. Maximum Installs - The maximum number of installations made.
9. Free - Boolean column which tells if an application is paid or free.
10. Price - Tells the price of the apps which are not free.
11. Size - The size of the application in bytes, MB, kb, GB and "Varies with device".
12. Developer Website - The website of the developer who developed the app.
13. Developer Email - Email address of the developer.
14. Released - The date the app was released.
15. Last Updated - The date the application was last updated.
16. Content Rating - The stratas of users.
17. Privacy Policy - Link of privacy policy.
18. Ad Supported - The boolean column which tells if the application supports advertisements or not.
19. In App Purchases - The boolean column which tells if the application needs in app purchases or not.
20. Editors Choice - The boolean column which tells if the application is editors choice or not.
21. Scraped Time - The time at which the data was scraped.
22. Developer Id - The Id of the developer who developed the app.
23. Currency - Currency of the price of the app.
24. Minimum Android - The minimum version of android required to run the application.

CHECKING NULL VALUES IN A DATASET

```
data1.isnull().sum().sort_values(ascending = False)
```

```
Developer Website 760835
Privacy Policy 420953
Released 71053
Rating 22883
Rating Count 22883
Minimum Android 6530
Size 196
Currency 135
Installs 107
Minimum Installs 107
Developer Id 33
Developer Email 31
App Name 2
App Id  0
Price 0
Free 0
Maximum Installs 0
Last Updated 0
Content Rating 0
Category 0
Ad Supported 0
In App Purchases 0
Editors Choice 0
Scraped Time 0
dtype: int64
```

Developer Website has null values that contribute to 32.89% of the entire dataset, followed by Privacy Policy and Released Date.

CHECKING DATA TYPES

```
data1.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2312944 entries, 0 to 2312943
Data columns (total 24 columns):
# Column Dtype
--- ------ -----
--- ------ -----
0 App Name object
1 App Id object
2 Category object
3 Rating float64
4 Rating Count float64
5 Installs object
6 Minimum Installs float64
7 Maximum Installs int64
8 Free bool
9 Price float64
10 Currency object
11 Size object
12 Minimum Android object
13 Developer Id object
14 Developer Website object
15 Developer Email object
16 Released object
17 Last Updated object
18 Content Rating object
19 Privacy Policy object
20 Ad Supported bool
21 In App Purchases bool
22 Editors Choice bool
23 Scraped Time object
dtypes: bool(4), float64(4), int64(1), object(15)
memory usage: 361.8+ MB
```

The dataset consists of 4 numeric columns, 4 boolean columns and the rest are of object data type (class type is string).

```
missing_percentage = (data1.isnull().sum().sort_values(ascending = False)/len(data1))*100
missing_percentage
```

```
Developer Website 32.894657
Privacy Policy 18.199879
Released 3.071972
Rating 0.989345
Rating Count 0.989345
Minimum Android 0.282324
Size 0.008474
Currency 0.005837
Installs 0.004626
Minimum Installs 0.004626
Developer Id 0.001427
Developer Email 0.001340
App Name 0.000086
App Id 0.000000
Price 0.000000
Free 0.000000
Maximum Installs 0.000000
Last Updated 0.000000
Content Rating 0.000000
Category 0.000000
Ad Supported 0.000000
In App Purchases 0.000000
Editors Choice 0.000000
Scraped Time 0.000000
dtype: float64
```

Developer Website has null values that contribute to 32.89% of the entire dataset, followed by Privacy Policy and Released Date.

CHECKING UNIQUE VALUES AND THEIR COUNTS

data1['App Name'].unique()
array(['Gakondo', 'Ampere Battery Info', 'Vibook', ...,
'å¤§ä¿¡\xa0å®¢â\x80\x94ç\x86±è¡\x80æ\xad¸ä¾¿\x86', 'ORU Online',
'Biliyor Musun - Sonsuz YarÄ±Å\x9f'], dtype=object)

data1['App Id'].unique()
array(['com.ishakwe.gakondo', 'com.webserveis.batteryinfo',
'com.doantiepvien.crm', ...,
'datastructure.appoworld.datastucture', 'ishan.devi.suktam',
'com.yyazilim.biliyormusun'], dtype=object)

data1['Category'].unique()
array(['Adventure', 'Tools', 'Productivity', 'Communication', 'Social',
'Libraries & Demo', 'Lifestyle', 'Personalization', 'Racing',
'Maps & Navigation', 'Travel & Local', 'Food & Drink',
'Books & Reference', 'Medical', 'Puzzle', 'Entertainment',
'Arcade', 'Auto & Vehicles', 'Photography', 'Health & Fitness',
'Education', 'Shopping', 'Board', 'Music & Audio', 'Sports',
'Beauty', 'Business', 'Educational', 'Finance', 'News & Magazines',
'Casual', 'Art & Design', 'House & Home', 'Card', 'Events',
'Trivia', 'Weather', 'Strategy', 'Word', 'Video Players & Editors',
'Action', 'Simulation', 'Music', 'Dating', 'Role Playing',
'Casino', 'Comics', 'Parenting'], dtype=object)

data1['Rating'].unique()
array([0. , 4.4, 5. , 4.5, 2. , 4.7, 4.9, 3.9, 3.7, 4.2, 3.4, 3.8, 4.6,
4.1, 2.5, 2.3, 2.1, 2.7, 4.3, 3.1, 4.8, 3.3, 4. , 2.4, 3.2, 3.6,
2.6, nan, 1.5, 3.5, 2.9, 1.3, 2.8, 3. , 2.2, 1.8, 1.4, 1.2, 1.9,
1.7, 1.6, 1. , 1.1])

data1['Rating Count'].unique()
array([0.0000e+00, 6.4000e+01, 5.0000e+00, ..., 8.7553e+04, 7.5960e+04,7.8351e+04])

data1['Maximum Installs'].unique()
array([15, 7662, 58, ..., 590469, 7274660, 814548])

data1['Free'].unique()
array([True, False])

data1['Price'].unique()
array([0. , 1.99 , 4.99 , ..., 3.041816, 26.746362,

18.903596])

```
data1['Currency'].unique()
array(['USD', 'XXX', 'CAD', 'EUR', 'INR', nan, 'VND', 'GBP', 'BRL', 'KRW',
'TRY', 'RUB', 'SGD', 'AUD', 'PKR', 'ZAR'], dtype=object)

data1['Size'].unique()
array(['10M', '2.9M', '3.7M', ..., '405M', '3.2k', '512M'], dtype=object)

data1['Minimum Android'].unique()
array(['7.1', '5.0', '4.0.3', '4.1', '6.0', '4.4', '4.0', '4.2', '2.1',
'7.0', '2.2', '2.3', '4.3', '5.1', '1.0', '1.6', '3.0',
'Varies with device', '8.0', '2.3.3', '3.2', '1.5', '3.1', '2.0',
'1.1', '2.0.1'], dtype=object)
data1['Developer Id'].unique()
array(['Jean Confident IrÃ©nÃ©e NIYIZIBYOSE', 'Webserveis', 'Cabin Crew',
..., 'Domenico Sanfilippo Editore S.p.A.', 'GOKarli e.K.',
'Eventi'], dtype=object)

data1['Content Rating'].unique()
array(['Everyone', 'Teen', 'Mature 17+', 'Everyone 10+',
'Adults only 18+', 'Unrated'], dtype=object)

data1['Ad Supported'].unique()
array([False, True])

data1['In App Purchases'].unique()
array([False, True])

data1['Editors Choice'].unique()
array([False, True])

data1['Released_Year'].unique()
array(['2020', '2019', '2018', nan, '2016', '2021', '2017', '2015',
'2013', '2012', '2014', '2011', '2010'], dtype=object)

data1['Updated_Year'].unique()
array(['2020', '2021', '2019', '2018', '2017', '2015', '2013', '2016',
'2014', '2011', '2012', '2010', '2009'], dtype=object)

data1['Content Rating'].unique()
array(['Everyone', 'Teen', 'Mature 17+', 'Everyone 10+',
'Adults only 18+', 'Unrated'], dtype=object)
```

```
data1['Content Rating'].value_counts()
Everyone 2022089
Teen 196375
Mature 17+ 60289
Everyone 10+ 33901
Unrated 154
Adults only 18+ 136
Name: Content Rating, dtype: int64
```

```
data1['Currency'].value_counts()
USD 2311548
XXX 1236
EUR 6
INR 5
GBP 3
CAD 2
VND 1
BRL 1
KRW 1
TRY 1
RUB 1
SGD 1
AUD 1
PKR 1
ZAR 1
Name: Currency, dtype: int64
```

REMOVING IRRELEVANT DATA

data1=data1.drop(['Developer Website','Privacy Policy','Installs','Minimum Installs','Developer Email','Scraped Time'],axis=1)

1. Developer Website - Since this attribute has most missing values and is also irrelevant to further analysis of the dataset.
2. Privacy Policy - This column had the second most missing values and is irrelevant to the analysis of the application.
3. Installs and Minimum Installs - Due to the presence of 'Maximum Installs' column in the dataset which more significantly justifies the datset, these two columns contributed to the redundancy in the dataset.
4. Developer Email - This attribute is irrelvant to the analysis, since the developer name has been retained, this can be dropped.
5. Scraped Time - The scarpping time is irrelavnt to the application analysis as it is specific to the owner of the dataset.

data1.dropna(subset=['App Name'], inplace=True)
There were only 2 missing values in the App Name, which is a very small portion relative to the entire data.

data1.head()

	App Name	App Id	Category	Rating	Rating Count	Maximum Installs	Free	Price	Currency	Size	Minimum Android	Developer Id	Released	Last Updated	Content Rating	Ad Supported	P
0	Gakondo	com.ishakwe.gakondo	Adventure	0.0	0.0	15	True	0.0	USD	10M	7.1 and up	Jean Confident IrÃ©nÃ©e NIYIZIBYOSE	Feb 26, 2020	Feb 26, 2020	Everyone	False	
1	Ampere Battery Info	com.webserveis.batteryinfo	Tools	4.4	64.0	7662	True	0.0	USD	2.9M	5.0 and up	Webserveis	May 21, 2020	May 06, 2021	Everyone	True	
2	Vibook	com.doantiepvien.crm	Productivity	0.0	0.0	58	True	0.0	USD	3.7M	4.0.3 and up	Cabin Crew	Aug 9, 2019	Aug 19, 2019	Everyone	False	
3	Smart City Trichy Public Service Vehicles 17UC...	cst.stJoseph.ug17ucs548	Communication	5.0	5.0	19	True	0.0	USD	1.8M	4.0.3 and up	Climate Smart Tech2	Sep 10, 2018	Oct 13, 2018	Everyone	True	
4	GROW.me	com.horodyski.grower	Tools	0.0	0.0	478	True	0.0	USD	6.2M	4.1 and up	Rafal Milek-Horodyski	Feb 21, 2020	Nov 12, 2018	Everyone	False	

CHECKING FOR DUPLICATES

```
has_duplicate_rows = data1.duplicated().any()
print(has_duplicate_rows)
```
False

CREATION OF NEW VARIABLES

```
data1['Released_Year'] = data1['Released'].str.extract(r'(\d{4})')
data1['Updated_Year'] = data1['Last Updated'].str.extract(r'(\d{4})')
data1.drop(columns=['Released'], inplace=True)
data1.drop(columns=['Last Updated'], inplace=True)
data1.head()
```



App Id	Category	Rating	Rating Count	Maximum Installs	Free	Price	Currency	Size	Minimum Android	Developer Id	Content Rating	Ad Supported	In App Purchases	Editors Choice	Released_Year	Updated_Year
we.gakondo	Adventure	0.0	0.0	15	True	0.0	USD	10M	7.1 and up	Jean Confident IrÃ©nÃ©e NIYIZIBYOSE	Everyone	False	False	False	2020	2020
s.batteryinfo	Tools	4.4	64.0	7662	True	0.0	USD	2.9M	5.0 and up	Webserveis	Everyone	True	False	False	2020	2021
iepvien.crm	Productivity	0.0	0.0	58	True	0.0	USD	3.7M	4.0.3 and up	Cabin Crew	Everyone	False	False	False	2019	2019
ug17ucs548	Communication	5.0	5.0	19	True	0.0	USD	1.8M	4.0.3 and up	Climate Smart Tech2	Everyone	True	False	False	2018	2018
lyski.grower	Tools	0.0	0.0	478	True	0.0	USD	6.2M	4.1 and up	Rafal Milek-Horodyski	Everyone	False	False	False	2020	2018

The date as a whole is not required, as only the year can be extracted and suitable analysis can be performed on it.

ADJUSTING DATA

```
data1['Minimum Android'] = data1['Minimum Android'].str.replace(' and up', '', regex=True)
data1['Minimum Android'].unique()
array(['7.1', '5.0', '4.0.3', '4.1', '6.0', '4.4', '4.0', '4.2', '2.1',
'7.0', '2.2', '2.3', '4.3', '5.1', nan, '1.6', '3.0',
'Varies with device', '8.0', '2.3.3', '4.4W', '3.2', '1.5',
'4.0.3 - 7.1.1', '3.1', '2.0', '1.0', '1.1', '2.0.1', '4.4 - 6.0',
'2.1 - 4.4', '4.0 - 5.0', '2.0 - 2.3.4', '2.2 - 4.4', '4.1 - 7.0',
'4.1 - 6.0', '4.0 - 4.4W', '4.0 - 4.4', '4.0.3 - 7.0', '2.3 - 5.0',
'4.0 - 7.1.1', '4.1 - 7.1.1', '4.0 - 7.0', '4.1 - 8.0',
'4.0 - 6.0', '2.3 - 7.0', '4.0 - 8.0', '2.3 - 6.0', '2.1 - 2.3.4',
'3.0 - 4.1.1', '2.2 - 3.0', '2.0 - 8.0', '4.0.3 - 8.0',
'3.0 - 4.4W', '4.4 - 7.1.1', '4.4 - 8.0', '4.0.3 - 6.0',
'1.6 - 4.0.4', '2.3 - 5.1', '2.1 - 7.1.1', '2.3 - 4.4W',
'4.4 - 7.0', '6.0 - 7.1.1', '2.3 - 4.4', '2.2 - 4.0.4',
'2.1 - 4.1.1', '3.0 - 8.0', '3.0 - 5.1', '2.1 - 6.0', '6.0 - 8.0',
'5.0 - 8.0', '4.1 - 5.1', '3.2 - 4.4', '2.1 - 5.0', '4.1 - 4.3',
'5.0 - 6.0', '5.1 - 7.1.1', '2.2 - 5.1', '4.1 - 5.0',
'2.3.3 - 4.4', '2.2 - 5.0', '4.1 - 4.4', '1.6 - 5.1',
'4.2 - 7.1.1', '2.1 - 5.1', '1.6 - 4.4W', '4.3 - 4.4W',
'3.0 - 4.4', '2.1 - 3.1', '2.3 - 7.1.1', '1.6 - 4.0.2',
'4.0 - 4.0.4', '4.2 - 4.4W', '3.0 - 7.1.1', '3.2 - 7.1.1',
'3.0 - 6.0', '2.2 - 4.4W', '4.0.3 - 4.4', '4.0.3 - 5.1',
'1.1 - 4.4', '2.3.3 - 8.0', '2.3.3 - 5.0', '2.2 - 6.0',
'3.0 - 5.0', '2.3.3 - 2.3.4', '1.5 - 2.1', '2.2 - 3.2',
'5.0 - 7.1.1', '2.1 - 3.2', '4.2 - 4.2.2', '2.3.3 - 5.1',
'2.2 - 4.1.1', '1.6 - 4.4', '4.2 - 4.3', '4.0 - 5.1', '1.6 - 7.0',
'3.0 - 7.0', '1.0 - 6.0', '2.3.3 - 4.0.4', '2.3 - 3.2',
'4.3 - 8.0', '2.3 - 8.0', '3.2 - 6.0', '2.3.3 - 7.1.1',
'4.2 - 8.0', '2.2 - 4.3', '3.2 - 4.1.1', '4.4 - 5.1',
'2.3 - 4.1.1', '2.1 - 4.4W', '2.3.3 - 4.4W', '4.1 - 4.4W',
'1.6 - 7.1.1', '2.2 - 4.2.2', '4.3 - 5.1', '4.0.3 - 4.2.2',
'2.2 - 2.3.4', '4.3 - 4.4', '4.2 - 6.0', '4.4 - 4.4W',
'2.3 - 4.0.2', '3.2 - 5.1', '3.0 - 3.2', '1.6 - 4.2.2',
'1.5 - 3.2', '2.2 - 8.0', '1.6 - 2.1', '2.3.3 - 6.0'], dtype=object)
```

```
data1['Minimum Android'] = data1['Minimum Android'].str.split(' - ').str[0]
data1['Minimum Android'] = data1['Minimum Android'].str.replace('W', '')
data1['Minimum Android'].unique()
array(['7.1', '5.0', '4.0.3', '4.1', '6.0', '4.4', '4.0', '4.2', '2.1',
'7.0', '2.2', '2.3', '4.3', '5.1', nan, '1.6', '3.0',
'Varies with device', '8.0', '2.3.3', '3.2', '1.5', '3.1', '2.0',
'1.0', '1.1', '2.0.1'], dtype=object)
```

Since the column has values of Minimum Android version, only the version number is required.

```
free={
True:1,
False:0
}
data1['Free']=data1['Free'].map(free)
data1['Ad Supported']=data1['Ad Supported'].map(free)
data1['In App Purchases']=data1['In App Purchases'].map(free)
data1['Editors Choice']=data1['Editors Choice'].map(free)
```

Mapped the values of True and False in all boolean columns to 1 and 0 so as to make computation easier.

data1.head()

App Id	Category	Rating	Rating Count	Maximum Installs	Free	Price	Currency	Size	Minimum Android	Developer Id	Content Rating	Ad Supported	In App Purchases	Editors Choice	Released_Year	Updated_Year
we.gakondo	Adventure	0.0	0.0	15	1	0.0	USD	10M	NaN	Jean Confident IrÃ©nÃ©e NIYIZIBYOSE	Everyone	0	0	0	2020	2020
s.batteryinfo	Tools	4.4	64.0	7662	1	0.0	USD	2.9M	NaN	Webserveis	Everyone	1	0	0	2020	2021
iepvien.crm	Productivity	0.0	0.0	58	1	0.0	USD	3.7M	NaN	Cabin Crew	Everyone	0	0	0	2019	2019
ug17ucs548	Communication	5.0	5.0	19	1	0.0	USD	1.8M	NaN	Climate Smart Tech2	Everyone	1	0	0	2018	2018
lyski.grower	Tools	0.0	0.0	478	1	0.0	USD	6.2M	NaN	Rafal Milek-Horodyski	Everyone	0	0	0	2020	2018

FILLING MISSING VALUES

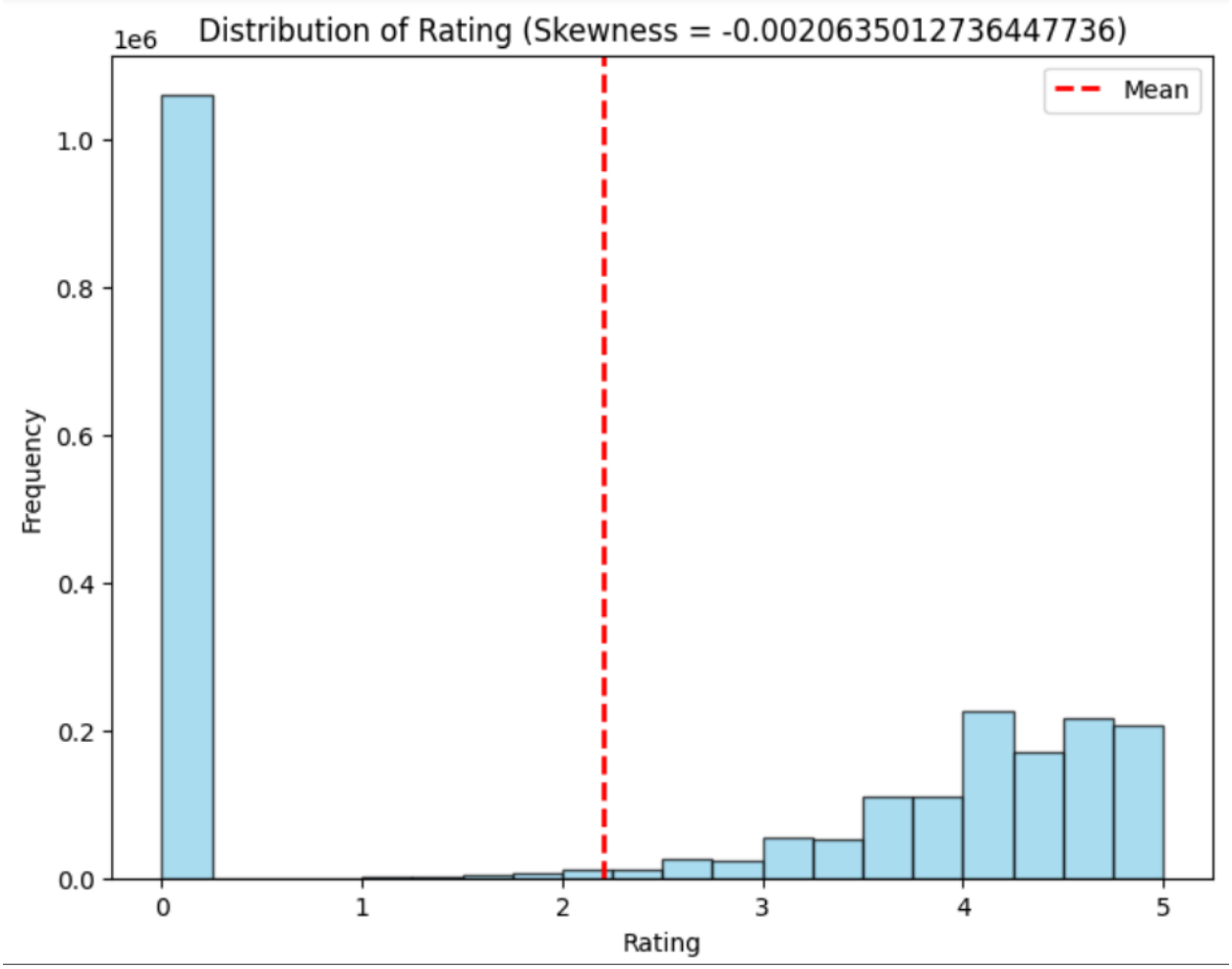
```
print(data1['Rating'].max())
print(data1['Rating'].min())
print(data1['Rating Count'].max())
print(data1['Rating Count'].min())
print(data1['Maximum Installs'].max())
print(data1['Maximum Installs'].min())
```

```
5.0
0.0
138557570.0
0.0
12057627016
0
```

1. Rating ranges from 0 to 5.
2. Rating count ranges from 0 to 1,38,557,570.
3. Maximum Installs ranges from 0 to 12,057,627,016.

```
rating_skewness = data1['Rating'].skew()
print(rating_skewness)
plt.figure(figsize=(8, 6))
plt.hist(data1['Rating'], bins=20, color='skyblue', edgecolor='black', alpha=0.7)
plt.xlabel('Rating')
plt.ylabel('Frequency')
plt.title(f'Distribution of Rating (Skewness = {rating_skewness})')
plt.axvline(data1['Rating'].mean(), color='red', linestyle='dashed', linewidth=2, label='Mean')
plt.legend()
plt.show()
```

```
-0.0020635012736447736
```



The data is negatively skewed with the skewness of -0.002 which tells that mean<median<mode. This also means that the maximum users have less rating.

```
mode_rating = data1['Rating'].mode()[0]
print(mode_rating)
```

0.0

```
data1['Rating'].fillna(0, inplace=True)
```

Hence, the null values in rating are replaced by the mode that is 0

```
df = pd.DataFrame(data1)
df = df[df['Size'] != 'varies with device']
modes = df.groupby('Category')['Size'].apply(lambda x: x.mode().iloc[0] if not x.mode().empty
else
for category, mode_size in modes.items():
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
```

<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/i
data1['Size'][(data1['Category'] == category) & data1['Size'].isna()] = mode_size
<ipython-input-36-5be4248f4e5d>:8: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

The applications are grouped by category. The mode size of each category was found overlooking the "varies with device" value which is further used to replace the null values in Size based on the Category the application belongs to.

data1.head()

data1['Size'].value_counts()
Varies with device 74919
11M 62184
12M 56090
13M 48034
14M 45211
...
8.7k 1
784M 1
385M 1
7.6k 1
512M 1
Name: Size, Length: 1657, dtype: int64
data1['Developer Id'].fillna('Unknown', inplace=True)

The missing values in Developer Id is filled with "Unknown" value.

data1['Currency'].value_counts()
USD 2311546
XXX 1236
EUR 6
INR 5
GBP 3
CAD 2
VND 1
BRL 1

KRW 1
TRY 1
RUB 1
SGD 1
AUD 1
PKR 1
ZAR 1
Name: Currency, dtype: int64

```
data1['Currency'].fillna('USD', inplace=True)
```

Since the maximum values of Currency are "USD", the missing values where replaced by it.

```
data1['Minimum Android'].value_counts()
```

4.1 604533
4.4 402860
5.0 397009
4.0.3 180524
4.0 153527
4.2 115984
6.0 89933
2.3 65630
5.1 59290
Varies with device 46214
4.3 41363
7.0 34446
2.2 23770
2.3.3 21567
3.0 17135
2.1 16719
8.0 13831
1.6 8550
7.1 3044
2.0 2778
3.2 2684
1.5 2100
3.1 2003
2.0.1 443
1.0 309
1.1 166
Name: Minimum Android, dtype: int64

```
df = pd.DataFrame(data1)
df['Digits Before Period'] = df['Minimum Android'].str.extract(r'(\d+)\.')
df['Digits Before Period'] = df['Digits Before Period'].astype(float)
```

```
minimum_digits_before_period = df['Digits Before Period'].min()
print(minimum_digits_before_period)
data1 = data1.drop('Digits Before Period',axis=1)
1.0
data1['Minimum Android'].fillna('1.0', inplace=True)
data1['Minimum Android'].unique()
array(['7.1', '5.0', '4.0.3', '4.1', '6.0', '4.4', '4.0', '4.2', '2.1',
'7.0', '2.2', '2.3', '4.3', '5.1', '1.0', '1.6', '3.0',
'Varies with device', '8.0', '2.3.3', '3.2', '1.5', '3.1', '2.0',
'1.1', '2.0.1'], dtype=object)
```

We wanted to find out the minimum android version of the applications in the dataset, so that the rows with missing android versions can be replaced with the minimum version supported by applications.

```
data1['Released_Year'].fillna(data1['Updated_Year'], inplace=True)
```

The missing values of released year are replaced by the corresponding updated year.

```
condition = data1['Released_Year'] > data1['Updated_Year']
rows_with_condition = condition.sum()
print(rows_with_condition)
2716
data1.loc[condition, ['Released_Year', 'Updated_Year']] = data1.loc[condition, ['Updated_Year',
condition = data1['Released_Year'] > data1['Updated_Year']
rows_with_condition = condition.sum()
print(rows_with_condition)
0
```

The released year must be lesser than the updated year, this condition must be true for all rows. Wherever the condition is not satisfied, the years have been interchanged.

```
data1['Rating Count'].fillna('1',inplace=True)
```
The missing values in Rating count have been replaced by 1 as the missing Rating was filled with one value.

```
data1.isnull().sum().sort_values(ascending = False)
Rating 22883
Size 196
Currency 135
Developer Id 33
Minimum Android 0
Released_Year 0
Editors Choice 0
```

In App Purchases 0
In App Purchases 0
Ad Supported 0
Content Rating 0
App Name 0
App Id 0
Price 0
Free 0
Maximum Installs 0
Rating Count 0
Category 0
Updated_Year 0
dtype: int64

Hence, all the null values have been cleaned.

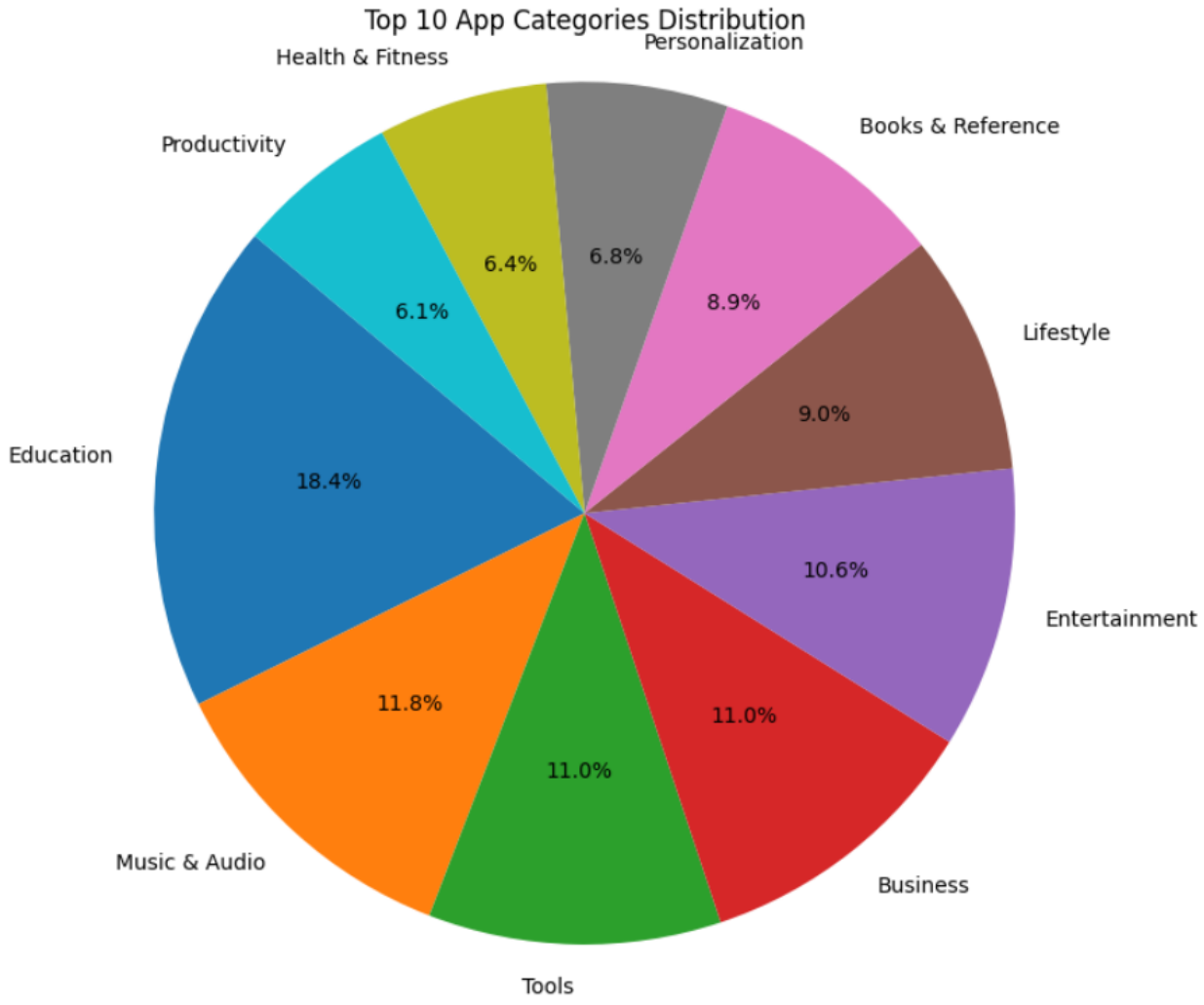
UNIVARIATE ANALYSIS

- Distribution of Top 10 Categories of applications

```
category_counts = data1['Category'].value_counts().head(10)

plt.figure(figsize=(8, 8))
plt.pie(category_counts, labels=category_counts.index, autopct='%1.1f%%',
startangle=140)
plt.title('Top 10 App Categories Distribution')

plt.axis('equal')
plt.show()
```



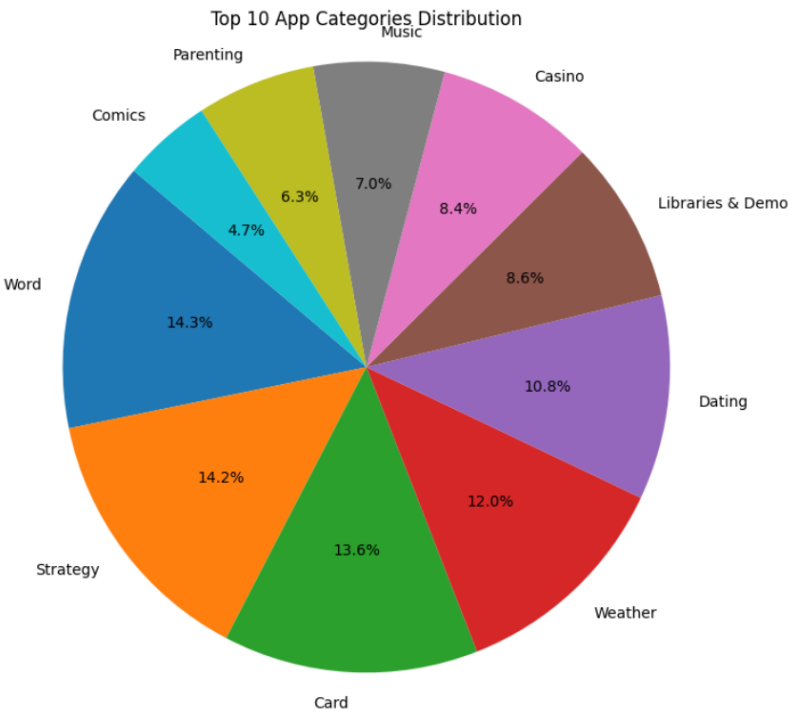
Maximum percentage of apps fall under the Education category followed by Music&Audio implying that the most app deployment has been focused towards the educational sector.

- Distribution of last 10 Categories

```
category_counts = data1['Category'].value_counts().tail(10)
print(category_counts)

plt.figure(figsize=(8, 8))
plt.pie(category_counts, labels=category_counts.index, autopct='%1.1f%%',
startangle=140)
plt.title('Top 10 App Categories Distribution')
plt.axis('equal')
plt.show()
```

```
Word              8630
Strategy          8526
Card              8179
Weather           7246
Dating            6524
Libraries & Demo  5198
Casino            5076
Music             4202
Parenting         3810
Comics            2862
Name: Category, dtype: int64
```



Least percentage of apps fall under the Comics category preceded by Parenting implying that the least app deployment has been made towards the Comics sector.

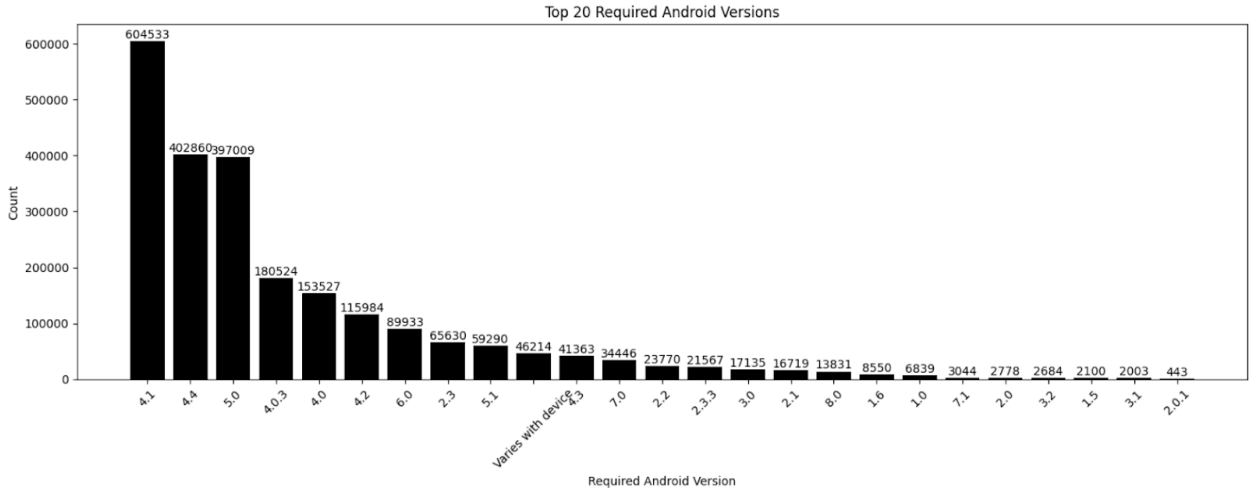
- Exploring the Minimum Android attribute

```
os_version_counts = data1['Minimum Android'].value_counts().head(25)


plt.figure(figsize=(15, 6))
bars = plt.bar(os_version_counts.index, os_version_counts.values, color='black')
plt.xlabel('Required Android Version')
plt.ylabel('Count')
plt.title('Top 20 Required Android Versions')
plt.xticks(rotation=45)


for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 5,
        f'{height}',
        ha='center',
        va='bottom'
    )

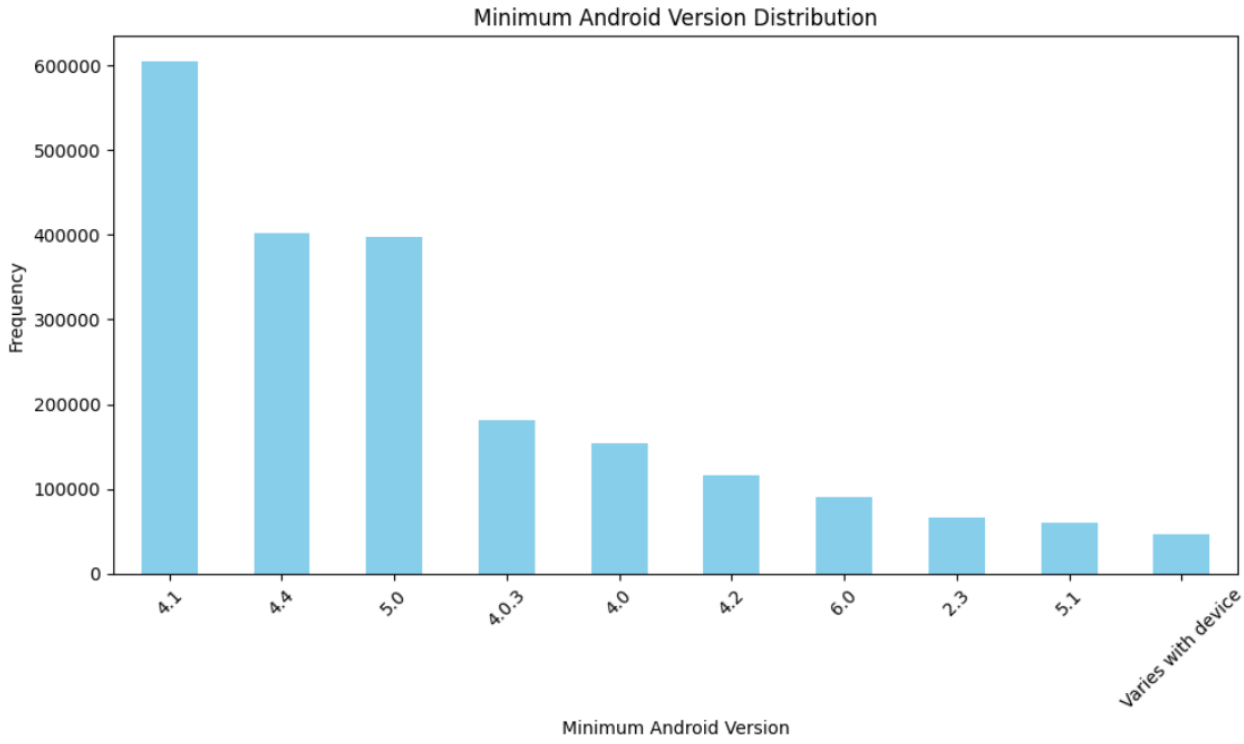
plt.tight_layout()
plt.show()
```



```
android_version_counts = data1['Minimum Android'].value_counts().head(10)

plt.figure(figsize=(10, 6))
android_version_counts.plot(kind='bar', color='skyblue')
plt.xlabel('Minimum Android Version')
plt.ylabel('Frequency')
plt.title('Minimum Android Version Distribution')


plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



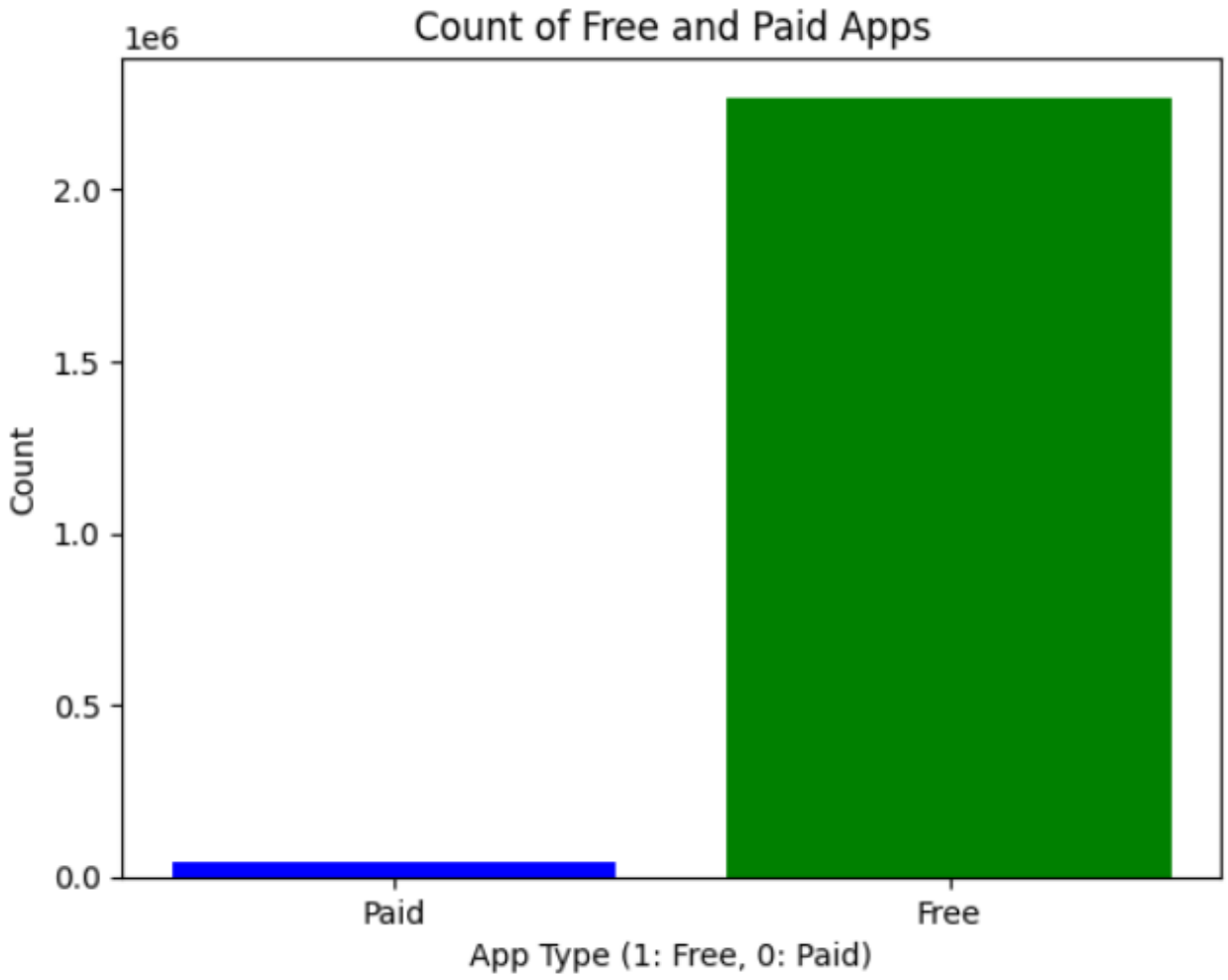
Maximum apps are supported by Android version 4.1 on Google PlayStore.

- Exploring Free attribute

```
app_counts = data1['Free'].value_counts()
print(app_counts)

plt.bar(app_counts.index, app_counts.values, color=['green', 'blue'])
plt.xlabel('App Type (1: Free, 0: Paid)')
plt.ylabel('Count')
plt.title('Count of Free and Paid Apps')
plt.xticks(app_counts.index, ['Free', 'Paid'])
plt.show()
```

```
1    2267874
0      45068
Name: Free, dtype: int64
```



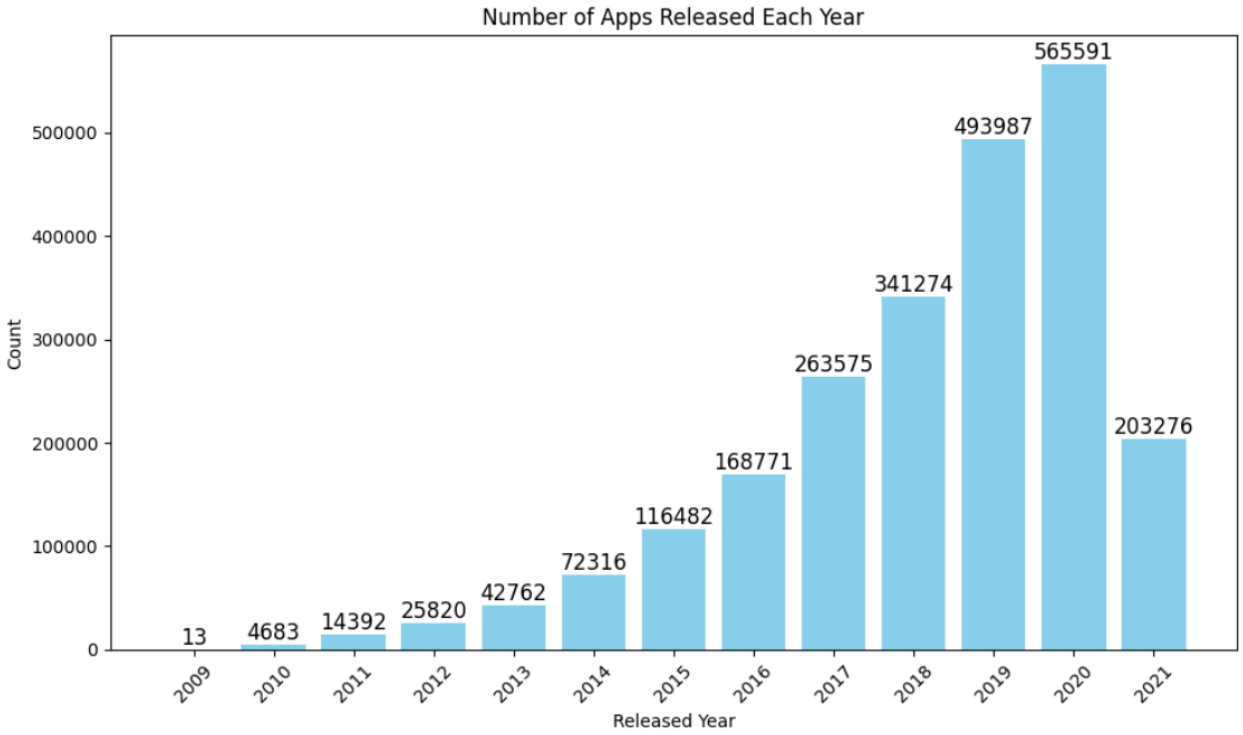
According to the above plot, free apps are higher in number as compared to paid apps.

- Exploring Released year attribute

```
year_counts = data1['Released_Year'].value_counts().sort_index()
plt.figure(figsize=(10, 6))
plt.bar(year_counts.index, year_counts.values, color='skyblue')
plt.xlabel('Released Year')
plt.ylabel('Count')
plt.title('Number of Apps Released Each Year')

for i, count in enumerate(year_counts.values):
    plt.text(year_counts.index[i], count, str(count), ha='center', va='bottom', fontsize=12)

plt.xticks(year_counts.index, rotation=45)
plt.tight_layout()
plt.show()
```



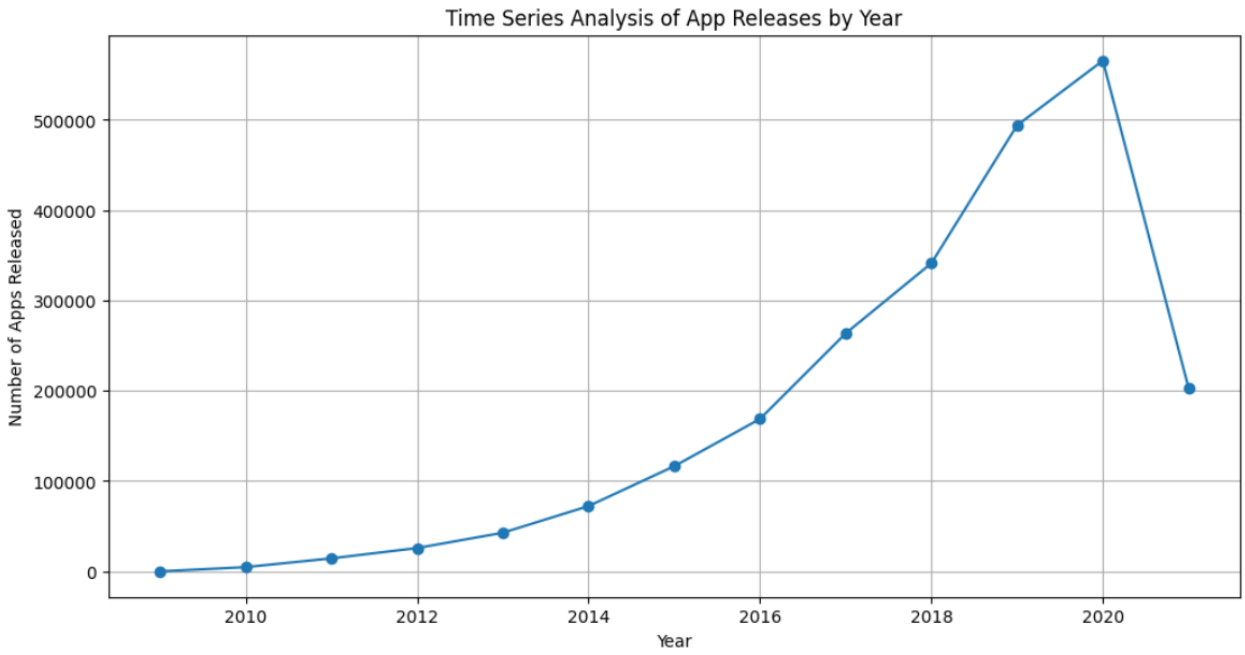
Maximum apps(565591) have been released in the year 2020. Since, it was the year of lockdown, it was the peak year to release apps which is rightly justified by the data.

```
data1['Released_Year'] = pd.to_datetime(data1['Released_Year'], format='%Y')

yearly_counts = data1['Released_Year'].dt.year.value_counts().sort_index()

plt.figure(figsize=(12, 6))
```

```
plt.plot(yearly_counts.index, yearly_counts.values, marker='o', linestyle='-')
plt.title('Time Series Analysis of App Releases by Year')
plt.xlabel('Year')
plt.ylabel('Number of Apps Released')
plt.grid(True)
plt.show()
```



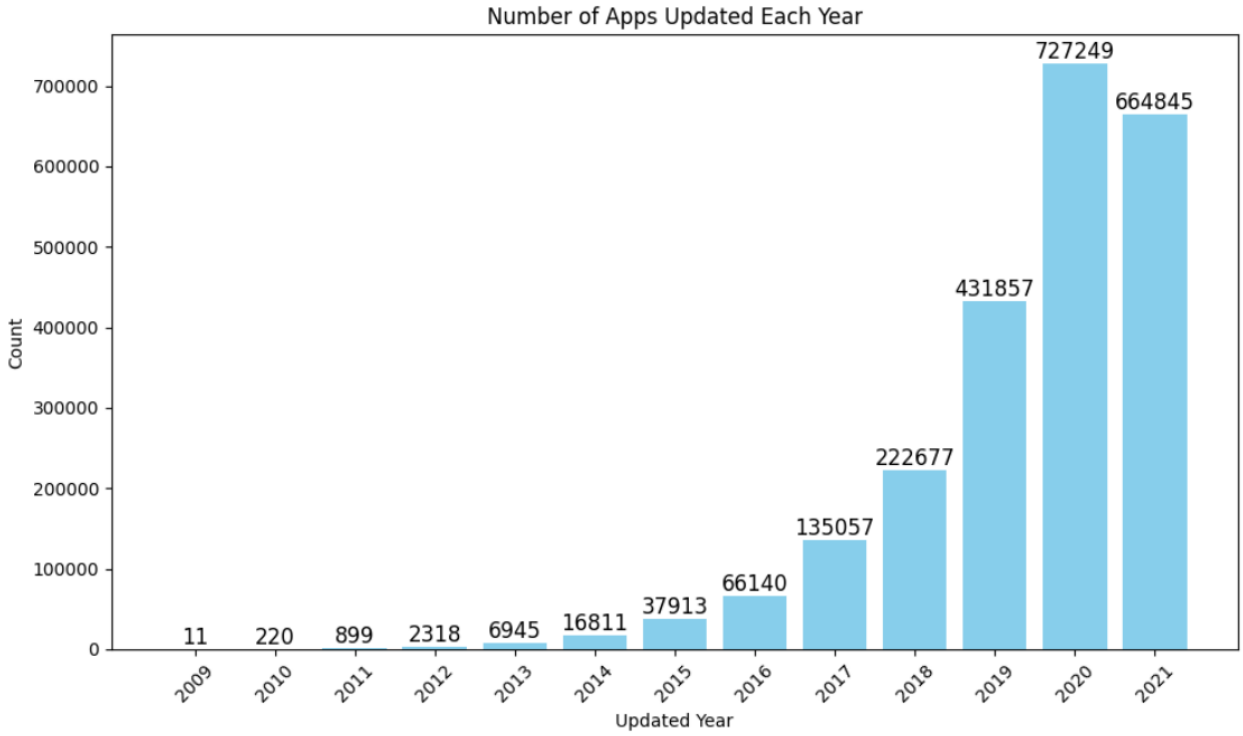
- Exploring Updated Year Attribute

```
year_counts = data1['Updated_Year'].value_counts().sort_index()

plt.figure(figsize=(10, 6))
plt.bar(year_counts.index, year_counts.values, color='skyblue')
plt.xlabel('Updated Year')
plt.ylabel('Count')
plt.title('Number of Apps Updated Each Year')


for i, count in enumerate(year_counts.values):
    plt.text(year_counts.index[i], count, str(count), ha='center', va='bottom', fontsize=12)

plt.xticks(year_counts.index, rotation=45)
plt.tight_layout()
plt.show()
```

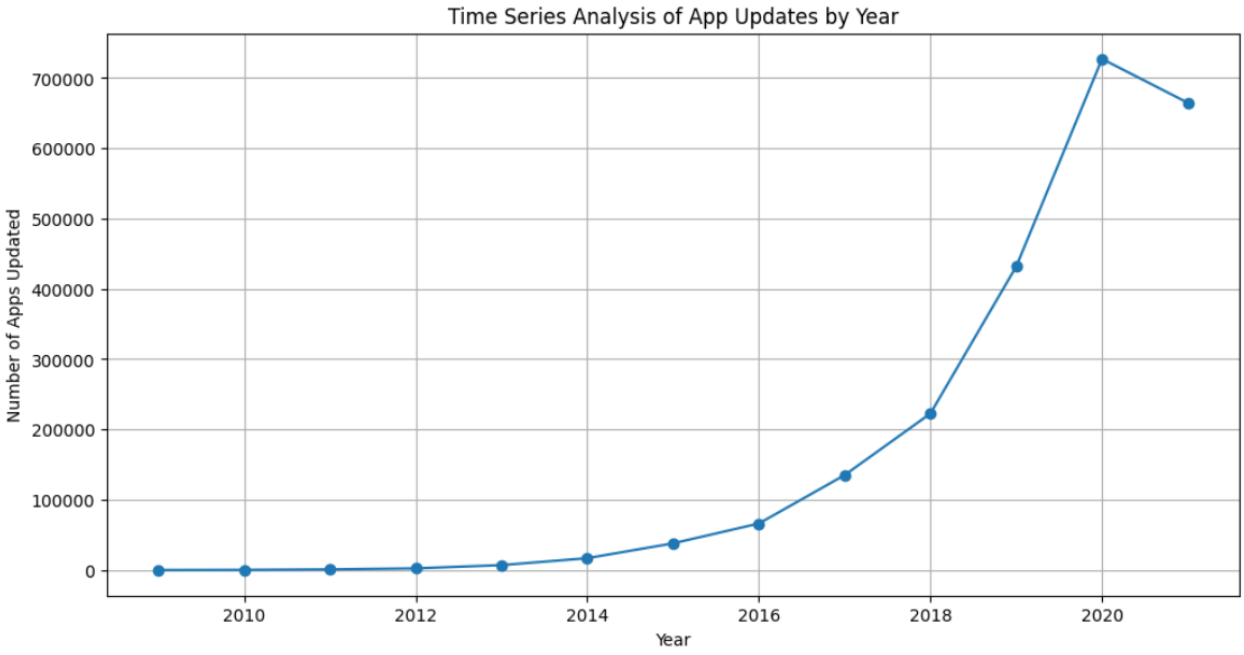


Maximum number of apps have been updated in the year 2020. Already existing applications were updated for better performance and user experience during the peak period of lockdown.

```
data1['Updated_Year'] = pd.to_datetime(data1['Updated_Year'], format='%Y')

yearly_counts = data1['Updated_Year'].dt.year.value_counts().sort_index()
```

```
plt.figure(figsize=(12, 6))
plt.plot(yearly_counts.index, yearly_counts.values, marker='o', linestyle='-')
plt.title('Time Series Analysis of App Updates by Year')
plt.xlabel('Year')
plt.ylabel('Number of Apps Updated')
plt.grid(True)
plt.show()
```



- Exploring Price attribute

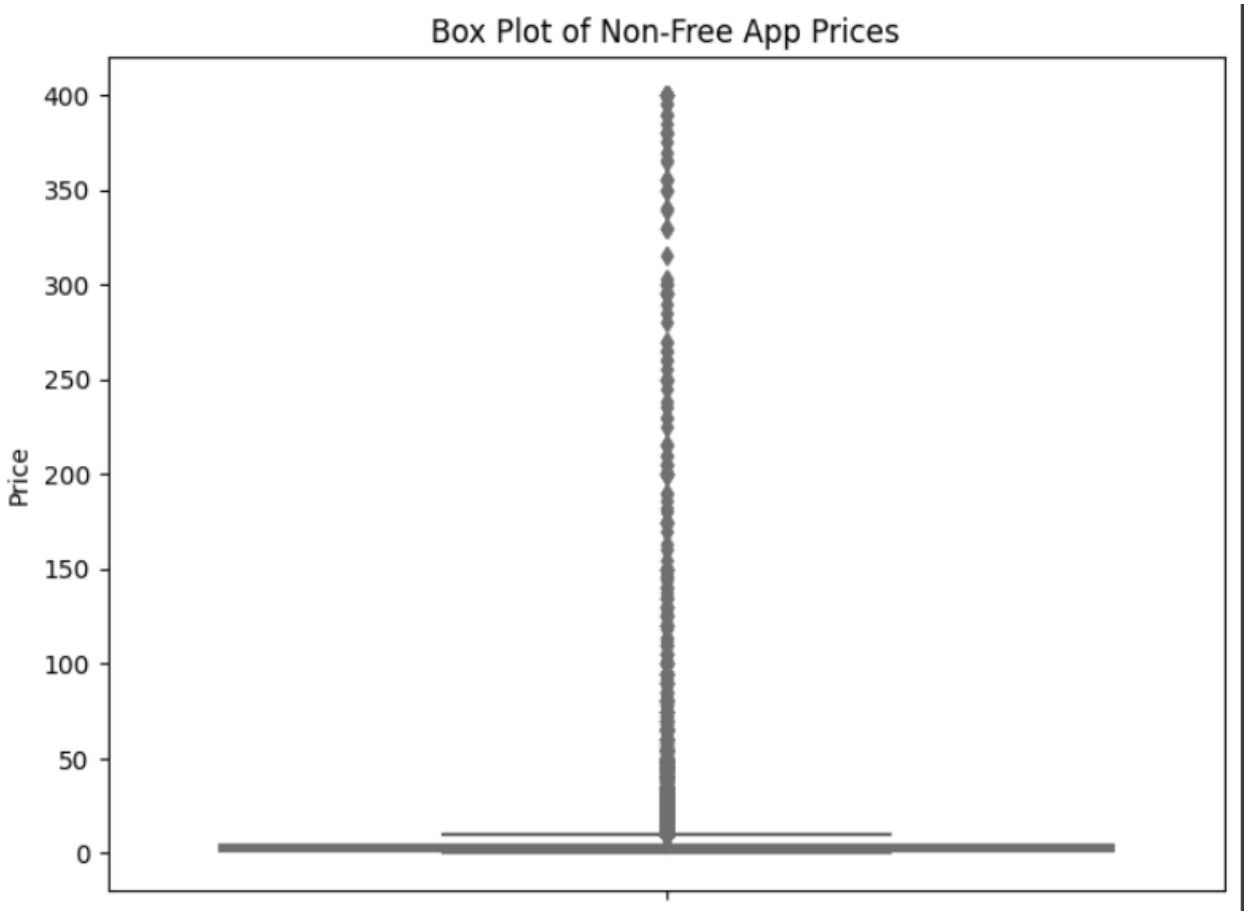
```
# Filter non-free apps
non_free_apps = data1[data1['Price'] > 0.0]

plt.figure(figsize=(8, 6))
sns.boxplot(data=non_free_apps, y='Price', color='skyblue')
plt.ylabel('Price')
plt.title('Box Plot of Non-Free App Prices')
plt.show()

max_price = non_free_apps['Price'].max()
min_price = non_free_apps['Price'].min()
print(max_price)
print(min_price)
```

```
400.0
0.194824
```

The range of prices in Google PlayStore is 0.194824 - 400.0.

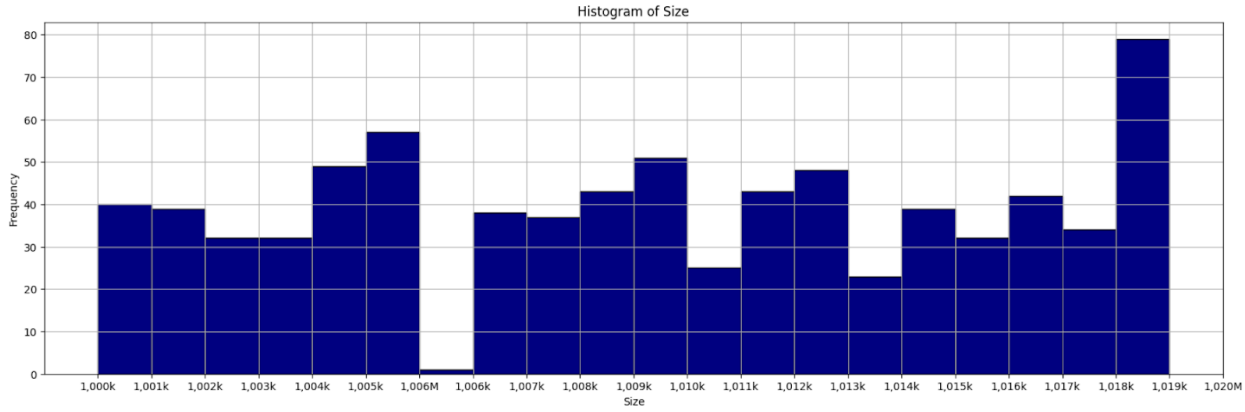


- Exploring Size Attribute

  sorted_sizes = sorted(data1['Size'])

  plt.figure(figsize=(20, 6))
  plt.hist(sorted_sizes, bins=20, range=(0, 20), color='navy', edgecolor='black')
  plt.xlabel('Size')
  plt.ylabel('Frequency')
  plt.title('Histogram of Size')
  plt.grid(True)

  plt.show()



The sizes of apps are clustered the most in the range of 1018k - 1019k, implying that most apps in the dataset have sizes in this range and least in 100

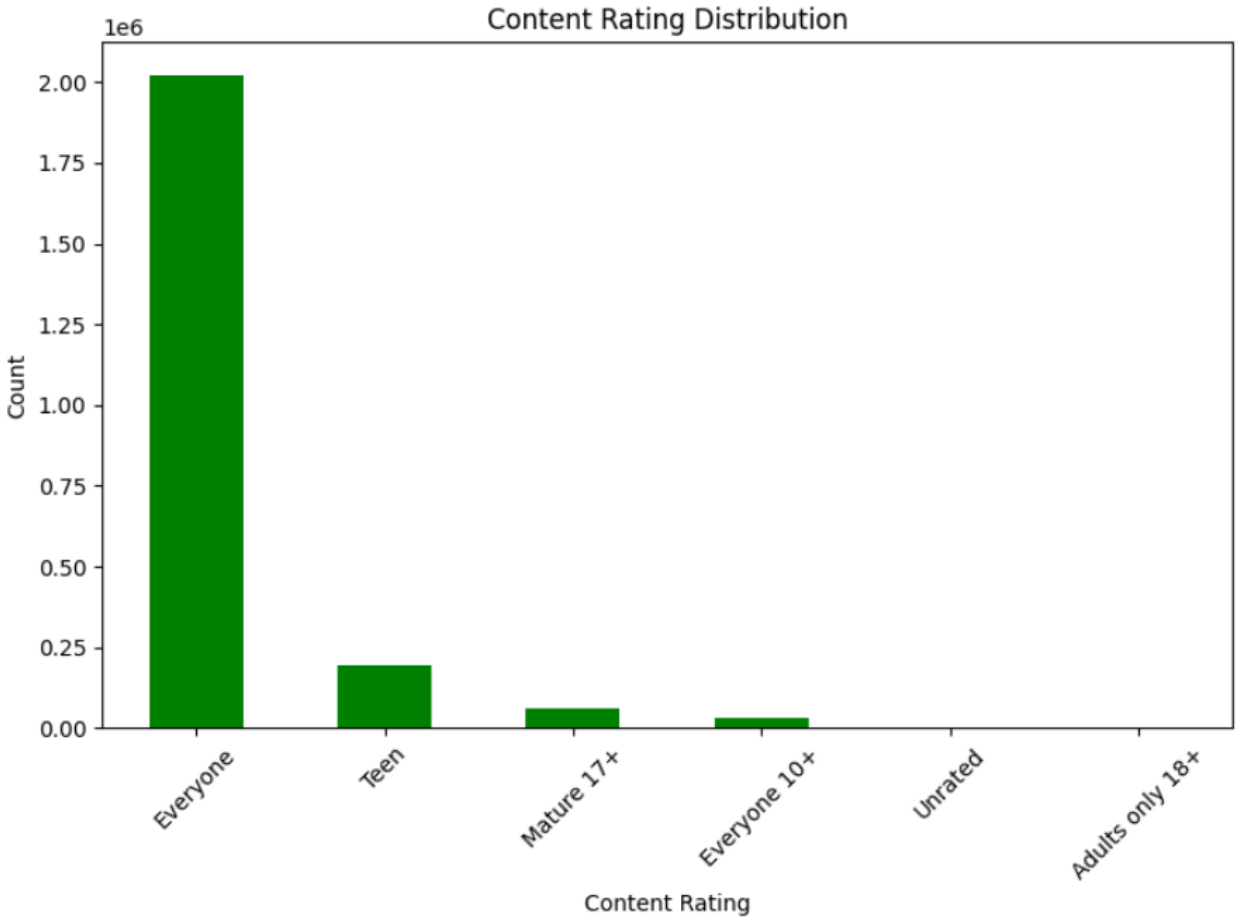
- Exploring Content Rating attribute

  data1['Content Rating'].unique()
  array(['Everyone', 'Teen', 'Mature 17+', 'Everyone 10+',
  'Adults only 18+', 'Unrated'], dtype=object)

  This attribute has the categories that the apps have been classified into as 'Everyone', 'Teen', 'Mature 17+', 'Everyone 10+', 'Adults only 18+', 'Unrated'

  content_rating_counts = data1['Content Rating'].value_counts()
  plt.figure(figsize=(8, 6))
  content_rating_counts.plot(kind='bar', color='green')
  plt.xlabel('Content Rating')
  plt.ylabel('Count')
  plt.title('Content Rating Distribution')

```
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



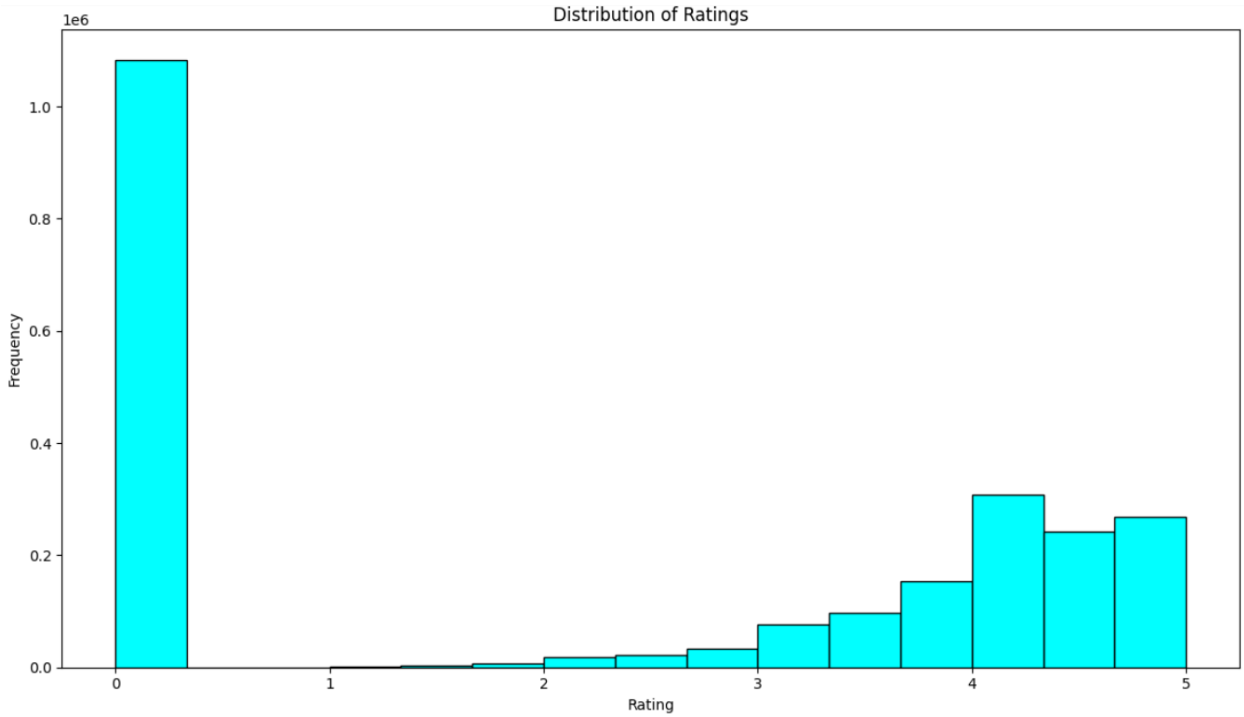
Most applications have Everyone rating and the least fall under Adults only 18+ category rating.

- Exploring Rating attribute

  ```
  data1['Rating'].unique()
  array([0. , 4.4, 5. , 4.5, 2. , 4.7, 4.9, 3.9, 3.7, 4.2, 3.4, 3.8, 4.6,
  4.1, 2.5, 2.3, 2.1, 2.7, 4.3, 3.1, 4.8, 3.3, 4. , 2.4, 3.2, 3.6,
  2.6, 1.5, 3.5, 2.9, 1.3, 2.8, 3. , 2.2, 1.8, 1.4, 1.2, 1.9, 1.7,
  1.6, 1. , 1.1])
  ```

  This reflects the user rating given to each app on the scale of 0.0 to 5.0.

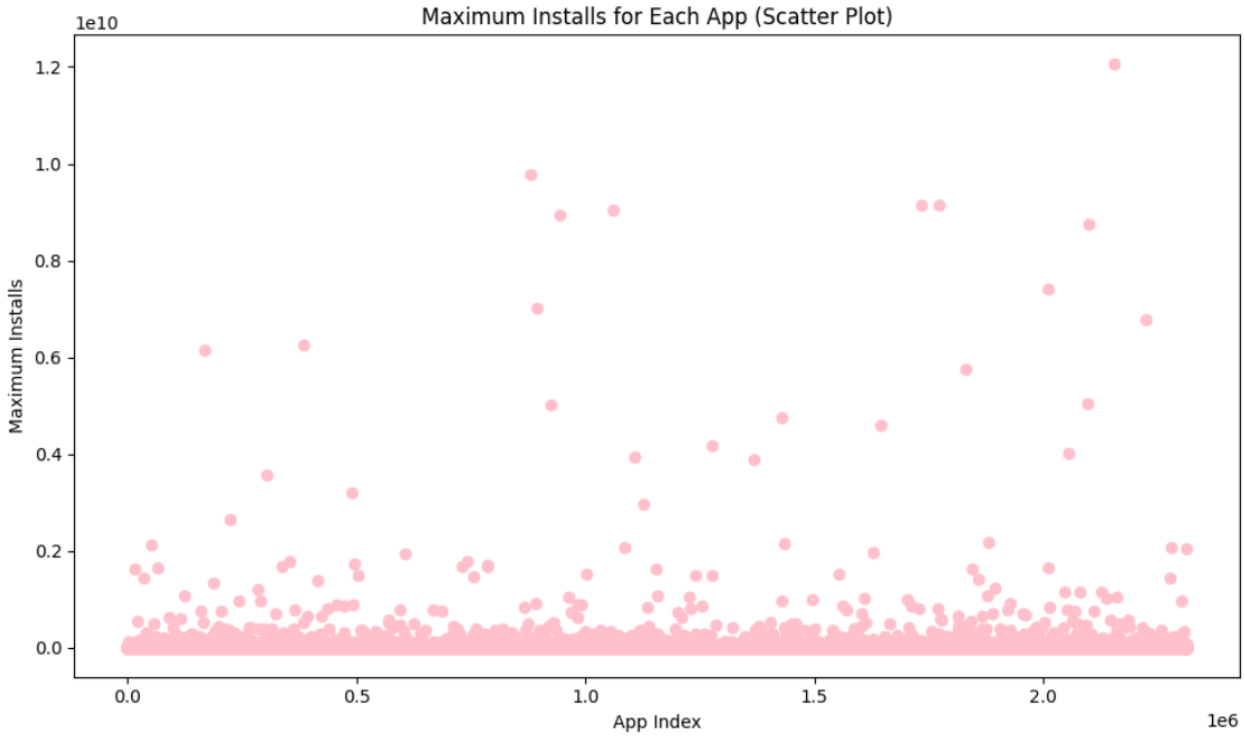
  ```
  plt.figure(figsize=(12, 7))
  plt.hist(data1['Rating'], bins=15, color='aqua', edgecolor='black')
  plt.xlabel('Rating')
  plt.ylabel('Frequency')
  plt.title('Distribution of Ratings')
  plt.tight_layout()
  plt.show()
  ```



Most apps have either not been rated or are rated 0 by the users. Although there is data clustered around 3.0 to 5.0 rating.

- Exploring Maximum Installs attribute

```
data1['Maximum Installs'].value_counts()
3 16815
2 16623
4 16319
6 15842
5 15695
...
1312664 1
679961 1
966697 1
120933 1
814548 1
Name: Maximum Installs, Length: 251563, dtype: int64
plt.figure(figsize=(10, 6))
plt.scatter(range(len(data1)), data1['Maximum Installs'], color='pink', marker='o')
plt.xlabel('App Index')
plt.ylabel('Maximum Installs')
plt.title('Maximum Installs for Each App (Scatter Plot)')
plt.tight_layout()
plt.show()
```

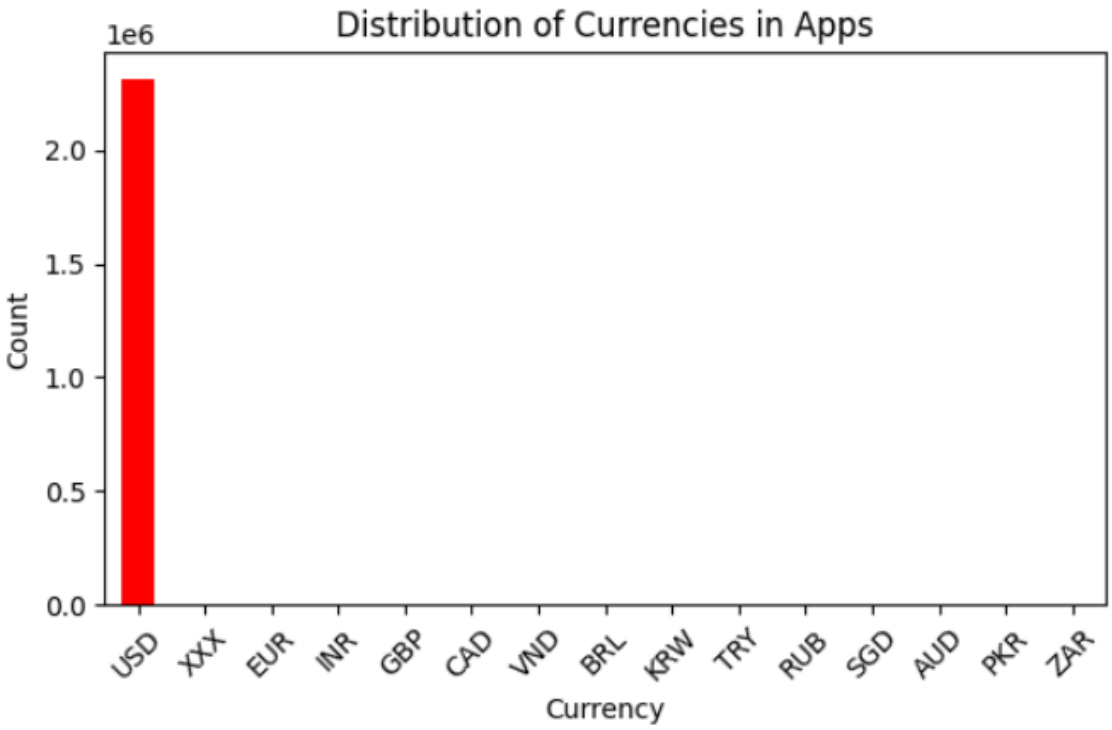


The scatter plot visualises the installs of applications made by users. The installs are clustered between 0.0 x 10^10 to 0.1 x 10^10.

- Exploring Currency Attribute

```
data1['Currency'].unique()
array(['USD', 'XXX', 'CAD', 'EUR', 'INR', 'VND', 'GBP', 'BRL', 'KRW',
'TRY', 'RUB', 'SGD', 'AUD', 'PKR', 'ZAR'], dtype=object)
currency_counts = data1['Currency'].value_counts()
print(currency_counts)
plt.figure(figsize=(6, 4))
currency_counts.plot(kind='bar', color='red')
plt.xlabel('Currency')
plt.ylabel('Count')
plt.title('Distribution of Currencies in Apps')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
USD    2311681
XXX       1236
EUR          6
INR          5
GBP          3
CAD          2
VND          1
BRL          1
KRW          1
TRY          1
RUB          1
SGD          1
AUD          1
PKR          1
ZAR          1
Name: Currency, dtype: int64
```

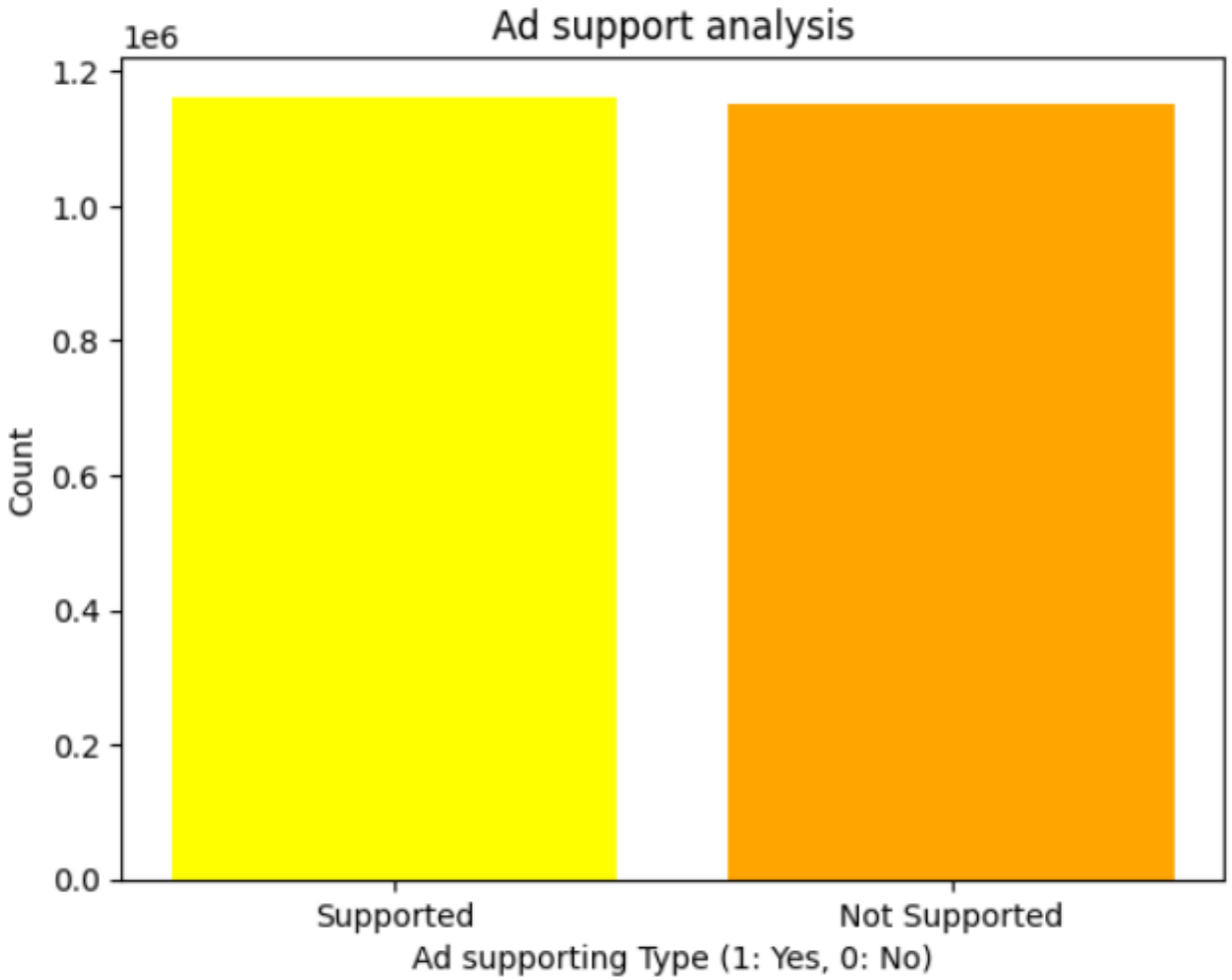


Maximum apps have USD as currency.

- Exploring Ad supported attribute

```
app_counts = data1['Ad Supported'].value_counts()
print(app_counts)
plt.bar(app_counts.index, app_counts.values, color=['yellow', 'orange'])
plt.xlabel('Ad supporting Type (1: Yes, 0: No)')
plt.ylabel('Count')
plt.title('Ad support analysis')
plt.xticks(app_counts.index, ['Supported', 'Not Supported'])
plt.show()
```

```
0    1162170
1    1150772
Name: Ad Supported, dtype: int64
```

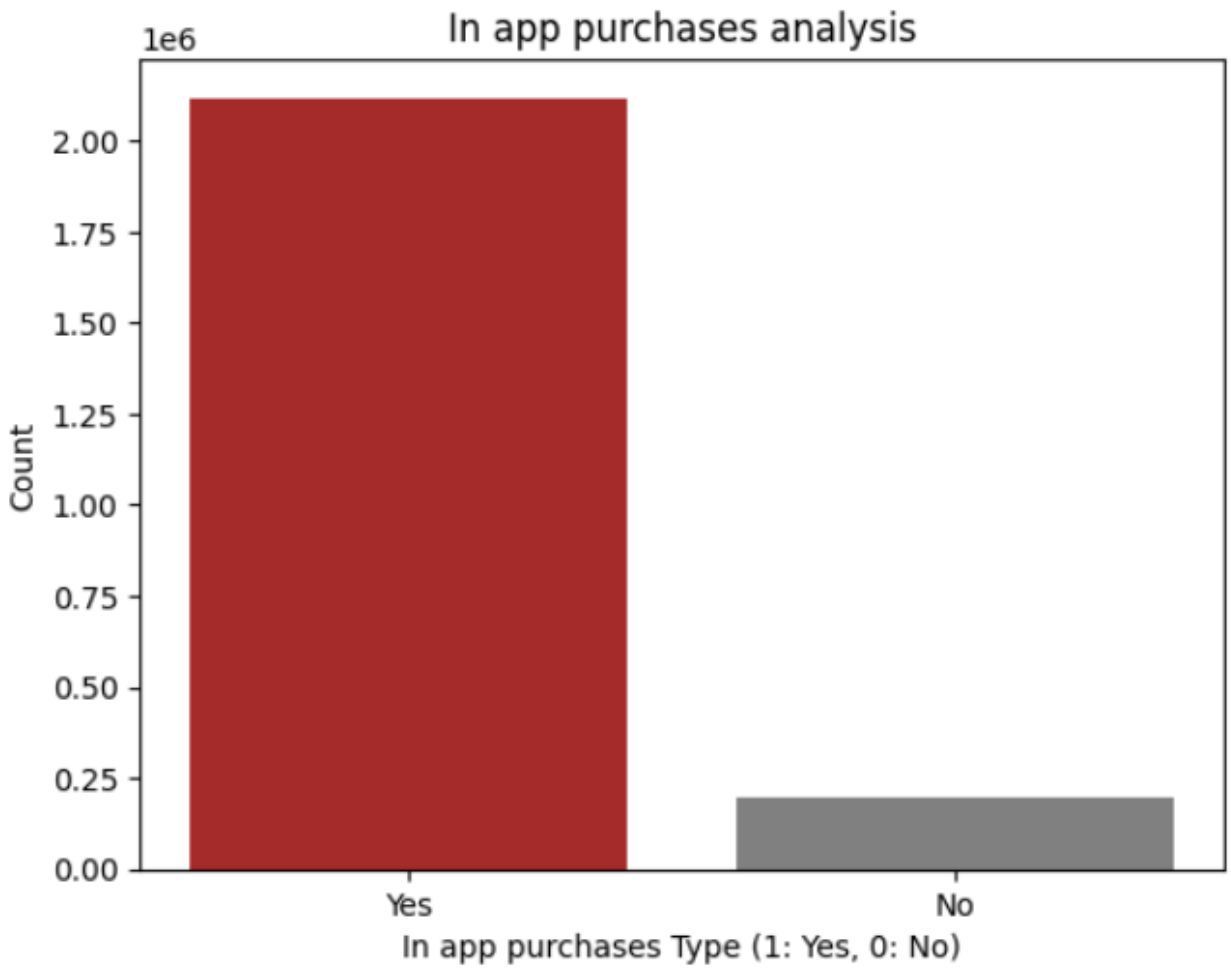


According to the bar plot, apps in the dataset are almost equally ad supported and not supported.

- Exploring In app purchases attribute

```
app_counts = data1['In App Purchases'].value_counts()
print(app_counts)
plt.bar(app_counts.index, app_counts.values, color=['brown', 'grey'])
plt.xlabel('In app purchases Type (1: Yes, 0: No)')
plt.ylabel('Count')
plt.title('In app purchases analysis')
plt.xticks(app_counts.index, ['Yes', 'No'])
plt.show()
```

```
0    2117633
1     195309
Name: In App Purchases, dtype: int64
```

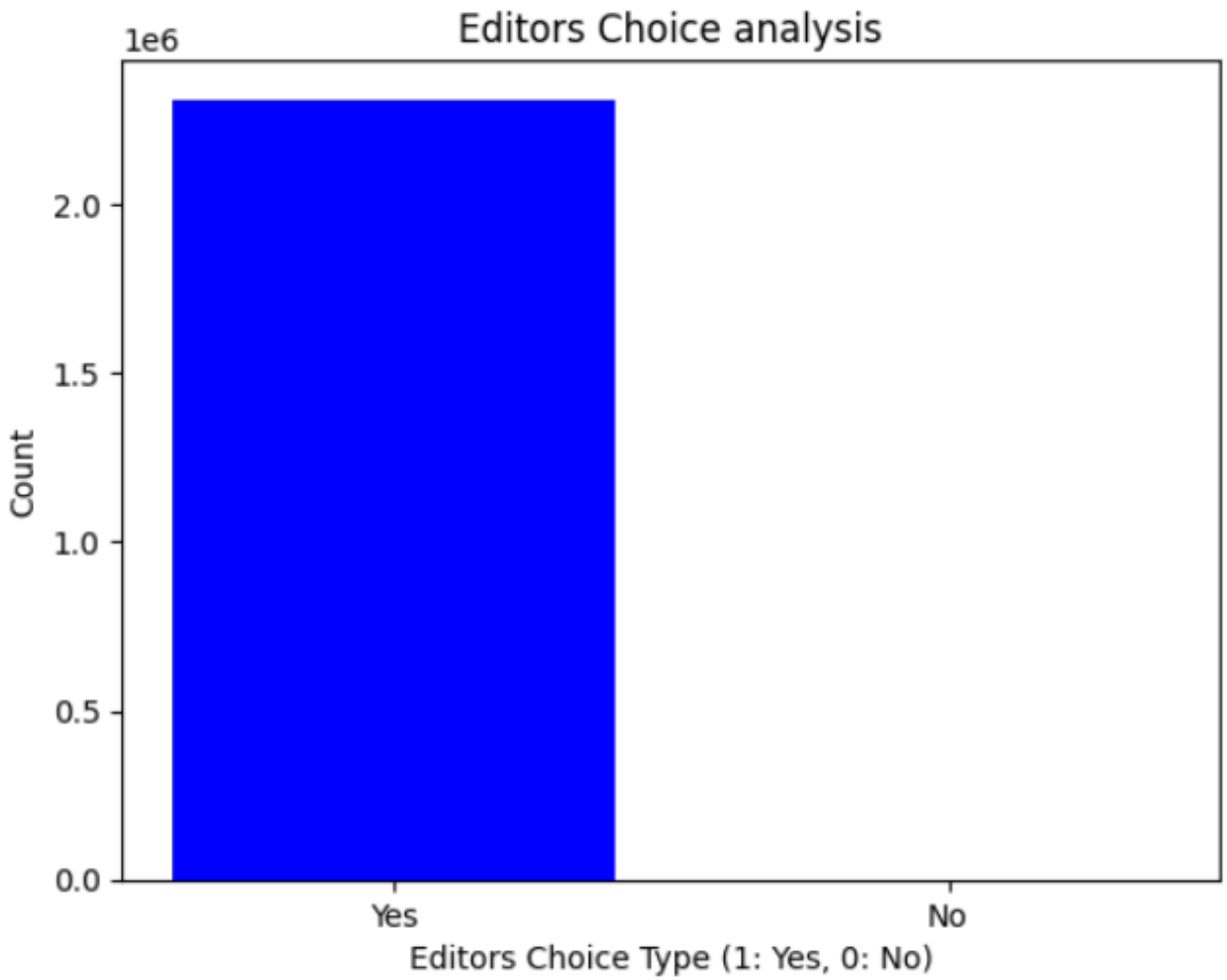


Maximum apps are in the in app purchases of playstore, very few percentage of apps are not in the in app purchases.

- Exploring Editors choice attribute

```
app_counts = data1['Editors Choice'].value_counts()
print(app_counts)
plt.bar(app_counts.index, app_counts.values, color=['blue', 'black'])
plt.xlabel('Editors Choice Type (1: Yes, 0: No)')
plt.ylabel('Count')
plt.title('Editors Choice analysis')
plt.xticks(app_counts.index, ['Yes', 'No'])
plt.show()
```

```
0    2312089
1        853
Name: Editors Choice, dtype: int64
```



Most apps are reviewed and recognized by the platform's editorial team or a group of experts for its quality, innovation, usability, or overall excellence.

BIVARIATE ANALYSIS

```
data1.columns
Index(['App Name', 'App Id', 'Category', 'Rating', 'Rating Count',
'Maximum Installs', 'Free', 'Price', 'Currency', 'Size',
'Minimum Android', 'Developer Id', 'Content Rating', 'Ad Supported',
'In App Purchases', 'Editors Choice', 'Released_Year', 'Updated_Year'],
dtype='object')
```

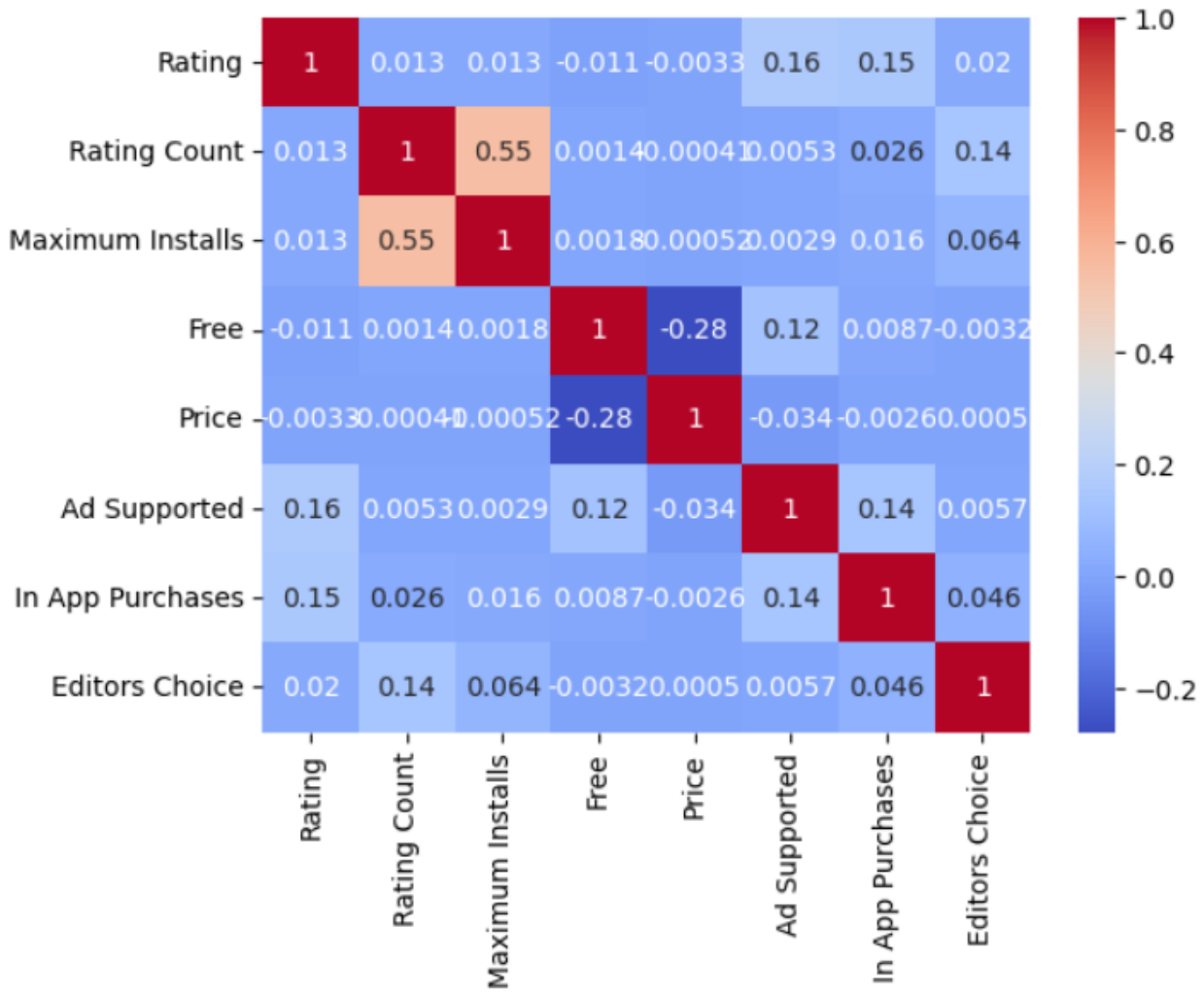
Above are the attributes of apps that will be considered for bivariate analysis according to the results of correlation matrix and logical correlation.

```
data1['Rating Count'] = pd.to_numeric(data1['Rating Count'], errors='coerce')
corr = data1.corr()
sns.heatmap(corr,cbar= True, cmap='coolwarm', annot=True)
print(corr)
```

```
<ipython-input-119-56fe0dc7f2a1>:3: FutureWarning: The default value of numeric_only in
DataFrame.corr is deprecated. In a future version, it will default to False. Select only valid
columns or specify the value of numeric_only to silence this warning.
  corr = data1.corr()
```

```
                  Rating  Rating Count  Maximum Installs      Free  \
Rating            1.000000      0.013107          0.012671 -0.011387
Rating Count      0.013107      1.000000          0.547548  0.001397
Maximum Installs  0.012671      0.547548          1.000000  0.001815
Free             -0.011387      0.001397          0.001815  1.000000
Price            -0.003309     -0.000406         -0.000515 -0.278831
Ad Supported      0.162949      0.005321          0.002882  0.123617
In App Purchases  0.150997      0.025558          0.016101  0.008669
Editors Choice    0.019579      0.137313          0.064206 -0.003157

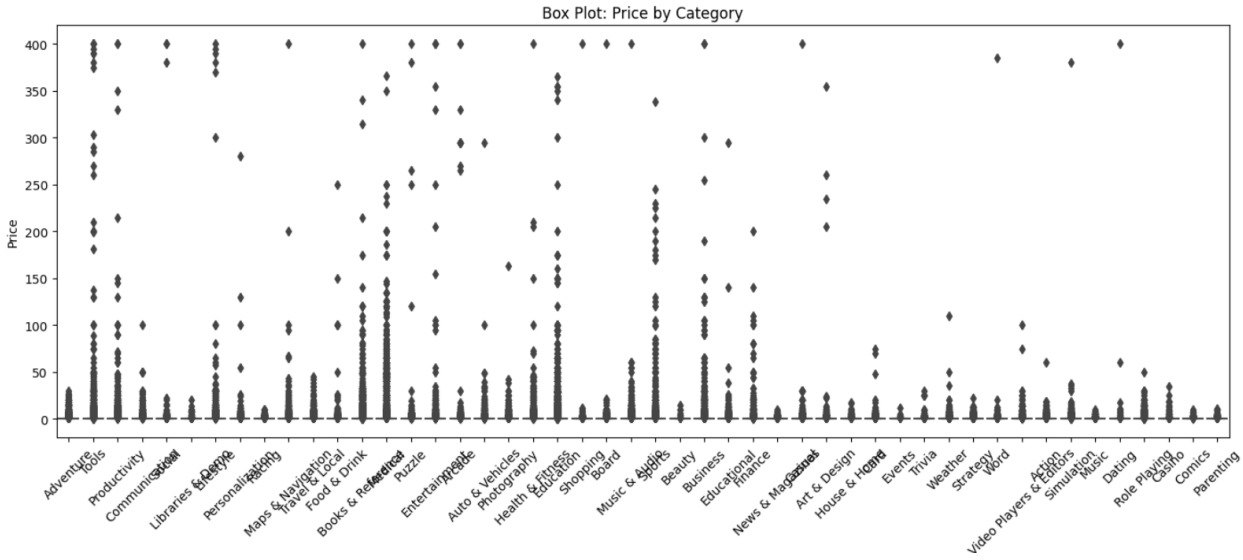
                   Price  Ad Supported  In App Purchases  Editors Choice
Rating          -0.003309      0.162949          0.150997        0.019579
Rating Count    -0.000406      0.005321          0.025558        0.137313
Maximum Installs -0.000515     0.002882          0.016101        0.064206
Free            -0.278831      0.123617          0.008669       -0.003157
Price            1.000000     -0.034276         -0.002634        0.000503
Ad Supported    -0.034276      1.000000          0.138313        0.005702
In App Purchases -0.002634     0.138313          1.000000        0.046078
Editors Choice   0.000503      0.005702          0.046078        1.000000
```



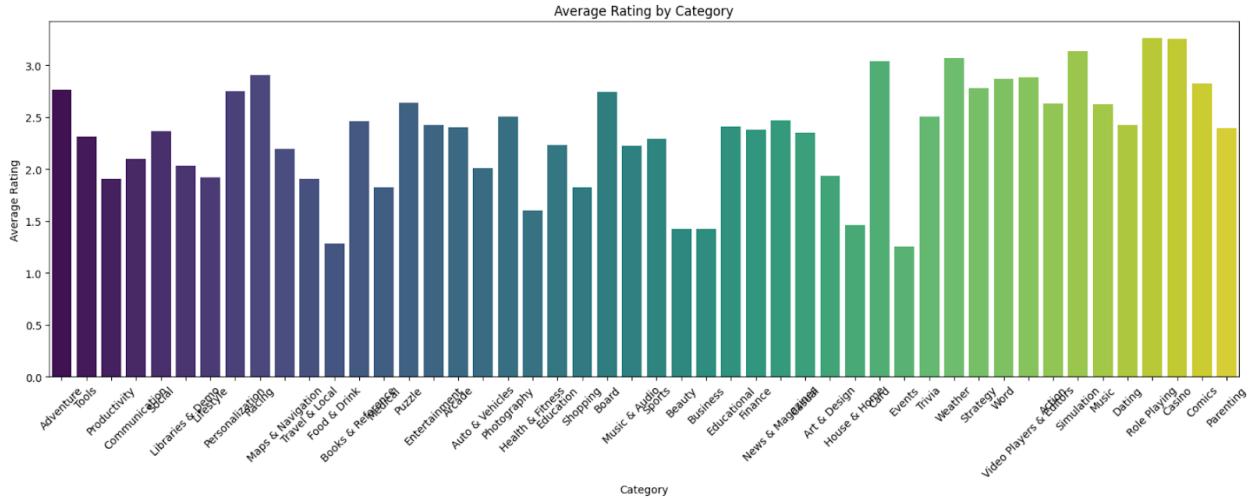
1. Maximum positive correlation has been found in Maximum Installs and Rating Count with a correlation coefficient of 0.547. This implies that apps that have higher rating count are installed the most.
2. Maximum negative correlation has been found in Price and Free with a correlation coefficient of -0.28. This implies that apps that as the probablity of app being free increases, the price decreases which is rightly justified by the data.

- Analysing Category attribute with Price, Rating, Maximum installs, Size, Content Rating, Released Year, Free, Ad supported, In app purchases, Editors choice and Minimum Android.

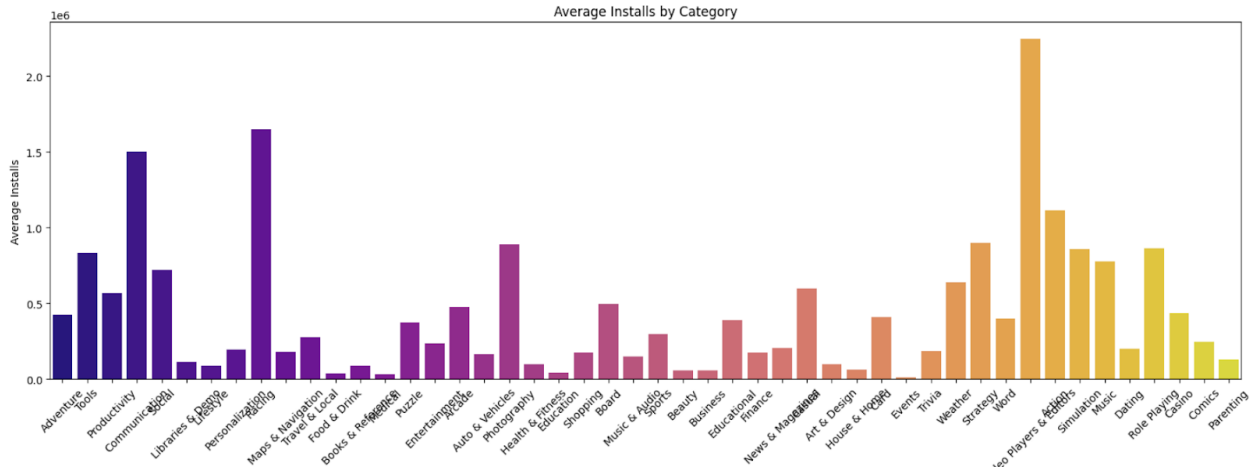
```
plt.figure(figsize=(17, 6))
sns.boxplot(data=data1, x='Category', y='Price')
plt.xlabel('Category')
plt.ylabel('Price')
plt.title('Box Plot: Price by Category')
plt.xticks(rotation=45)
plt.show()
```




```
plt.figure(figsize=(20, 6))
sns.barplot(data=data1, x='Category', y='Rating', ci=None, palette='viridis')
plt.xlabel('Category')
plt.ylabel('Average Rating')
plt.title('Average Rating by Category')
plt.title('Average Rating by Category')
plt.xticks(rotation=45)
plt.show()
```




```
plt.figure(figsize=(20, 6))
sns.barplot(data=df, x='Category', y='Maximum Installs', ci=None, palette='plasma')
plt.xlabel('Category')
plt.ylabel('Average Installs')
plt.title('Average Installs by Category')
plt.xticks(rotation=45)
plt.show()
```



```
data1['Size'].unique()
array(['10M', '2.9M', '3.7M', ..., '405M', '3.2k', '512M'], dtype=object)

df = pd.DataFrame(data1)
def find_unexpected_characters(size_str):
expected_characters = ['M', 'k']
last_char = size_str[-1]
if last_char not in expected_characters:
return last_char
df['Unexpected Character'] = df['Size'].apply(find_unexpected_characters)
df_filtered = df.dropna(subset=['Unexpected Character'])
unique_unexpected_chars = df_filtered['Unexpected Character'].unique()
print("Unique Unexpected Characters at the End of 'Size' Values:")
print(unique_unexpected_chars)

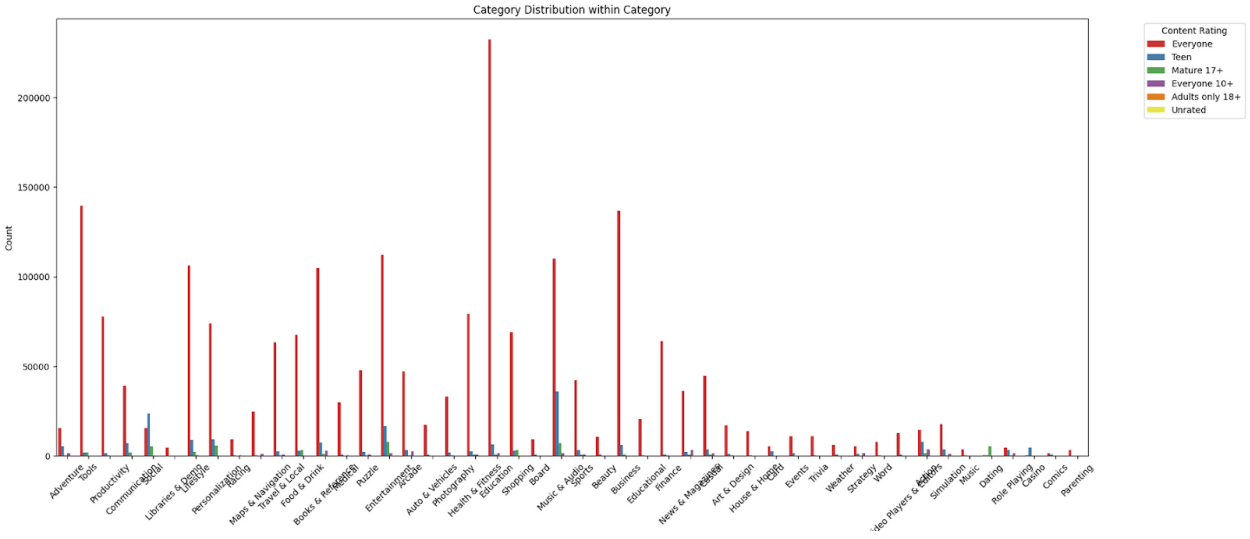
Unique Unexpected Characters at the End of 'Size' Values:
['e' 'G']

def convert_size_to_mb(size):
size = size.replace(',', '')
if size == "Varies with device":
return None
if 'M' in size:
if 'M' in size:
return float(size.rstrip('M'))
elif 'k' in size:
return float(size.rstrip('k')) / 1024
elif 'G' in size:
return float(size.rstrip('G')) * 1024
else:
return float(size) / 1024 / 1024
data1['Size (MB)'] = data1['Size'].apply(convert_size_to_mb)
data1['Size (MB)']

0 10.0
1 2.9
2 3.7
3 1.8
4 6.2
...
2312939 77.0
2312940 44.0
2312941 29.0
2312942 10.0
```

2312943 5.2
Name: Size (MB), Length: 2312942, dtype: float64

```
filtered_data = data1[data1['Category'].isin(data1['Category'].unique()) & data1['Content Rating'
plt.figure(figsize=(20, 9))
sns.countplot(data=filtered_data, x='Category', hue='Content Rating', palette='Set1')
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Category Distribution within Category')
plt.xticks(rotation=45)
plt.legend(title='Content Rating', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```




```
mode_category_by_content_rating = filtered_data.groupby('Content Rating')['Category'].agg(lambda
print(mode_category_by_content_rating)
```

Content Rating Category
0 Adults only 18+ Sports
1 Everyone Education
2 Everyone 10+ Action
3 Mature 17+ Entertainment
4 Teen Music & Audio
5 Unrated Entertainment

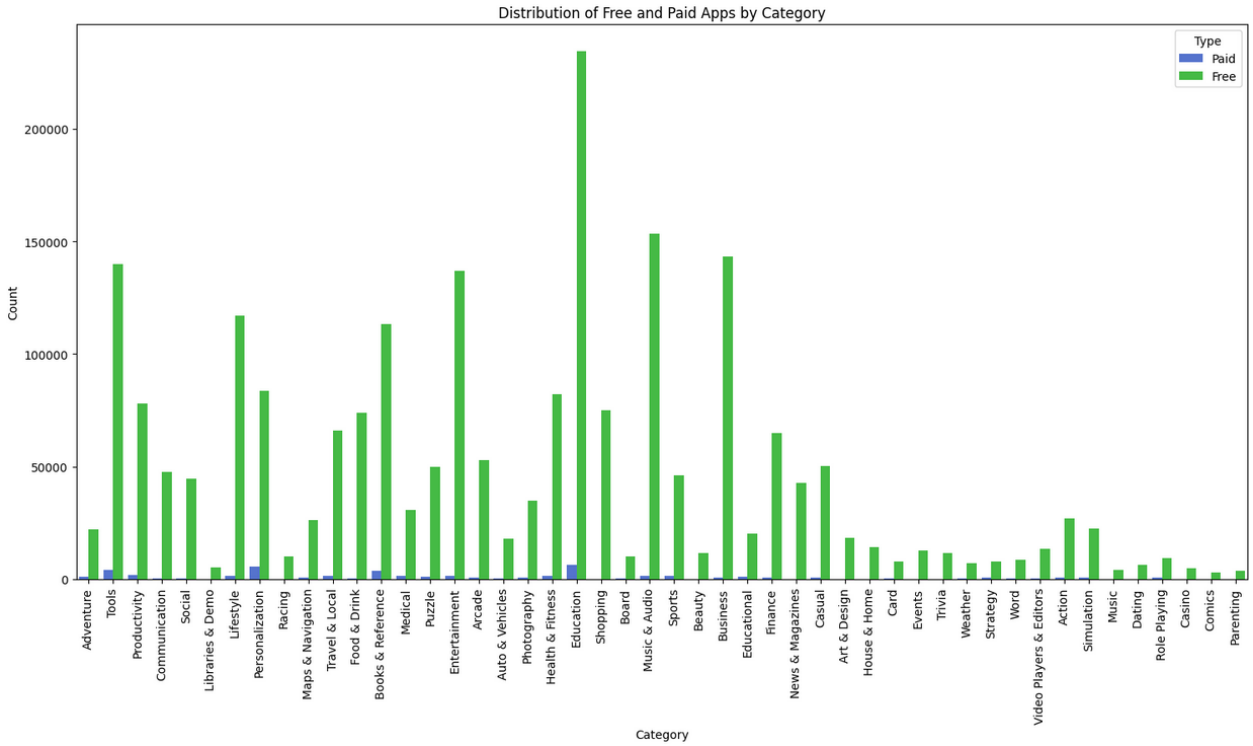
```
data1.columns
data1['Released_Year'].unique()
```

array(['2020', '2019', '2018', '2016', '2021', '2017', '2015', '2013',
'2012', '2014', '2011', '2010', '2009'], dtype=object)

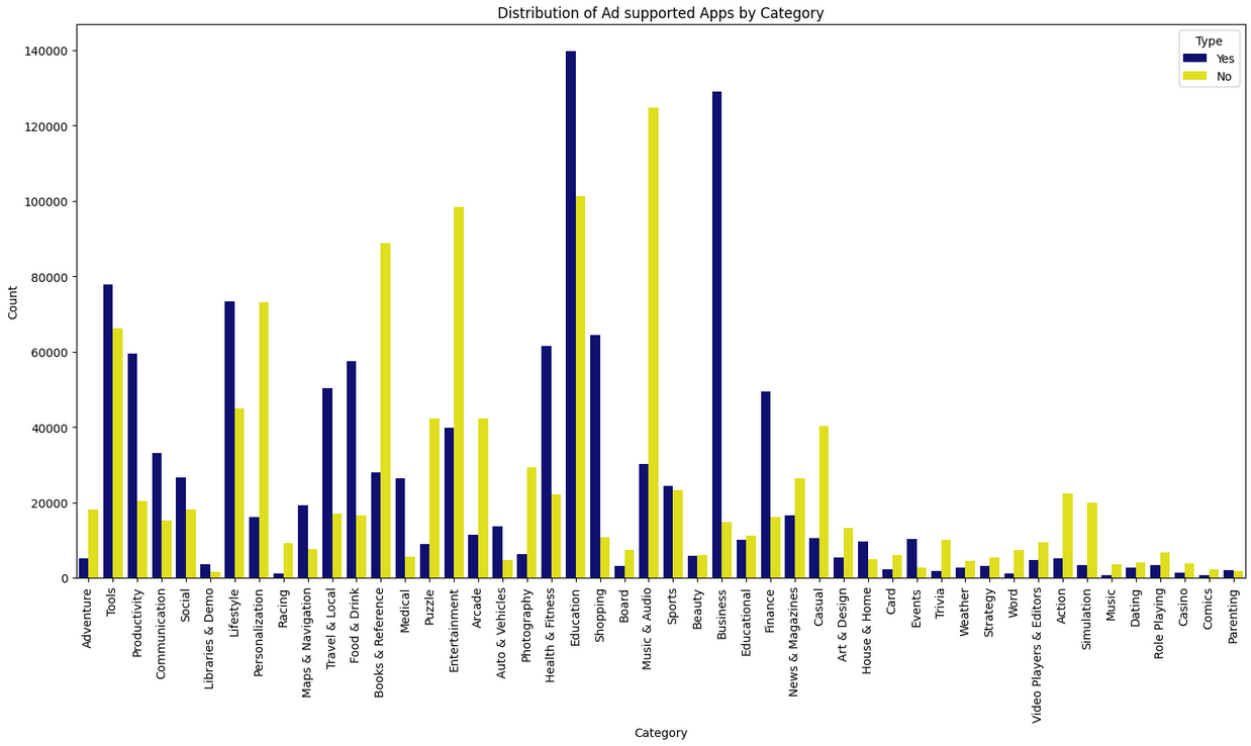
```
filtered_data = data1[data1['Released_Year'].isin(data1['Released_Year'].unique())]
most_common_category_by_year =
filtered_data.groupby('Released_Year')['Category'].agg(lambda x:
print(most_common_category_by_year)
```

```
Released_Year Category
0 2009 Tools
1 2010 Tools
2 2011 Tools
3 2012 Tools
4 2013 Education
5 2014 Education
6 2015 Education
7 2016 Education
8 2017 Education
9 2018 Education
10 2019 Education
11 2020 Education
12 2021 Education
```

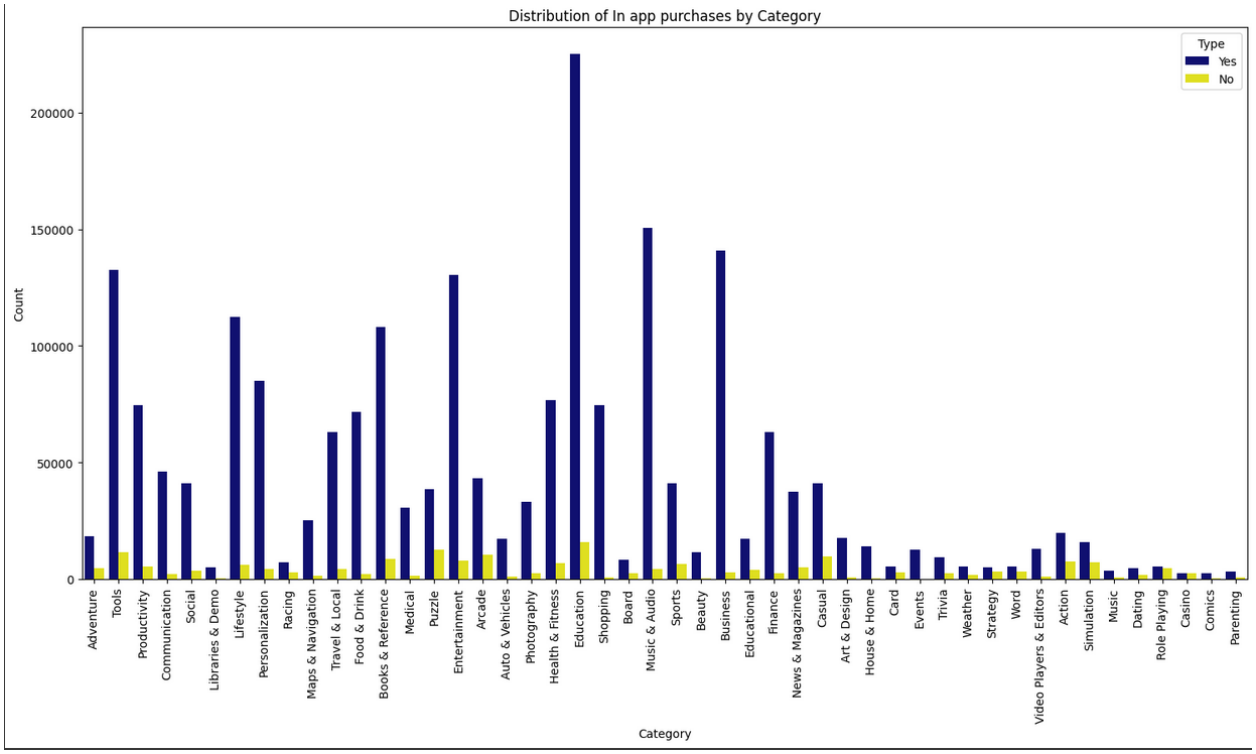
```
plt.figure(figsize=(15, 9))
sns.countplot(data=data1, x='Category', hue='Free', palette={0: 'royalblue', 1:
'limegreen'})
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Distribution of Free and Paid Apps by Category')
plt.xticks(rotation=90)
plt.legend(title='Type', labels=['Paid', 'Free'])
plt.tight_layout()
plt.show()
```



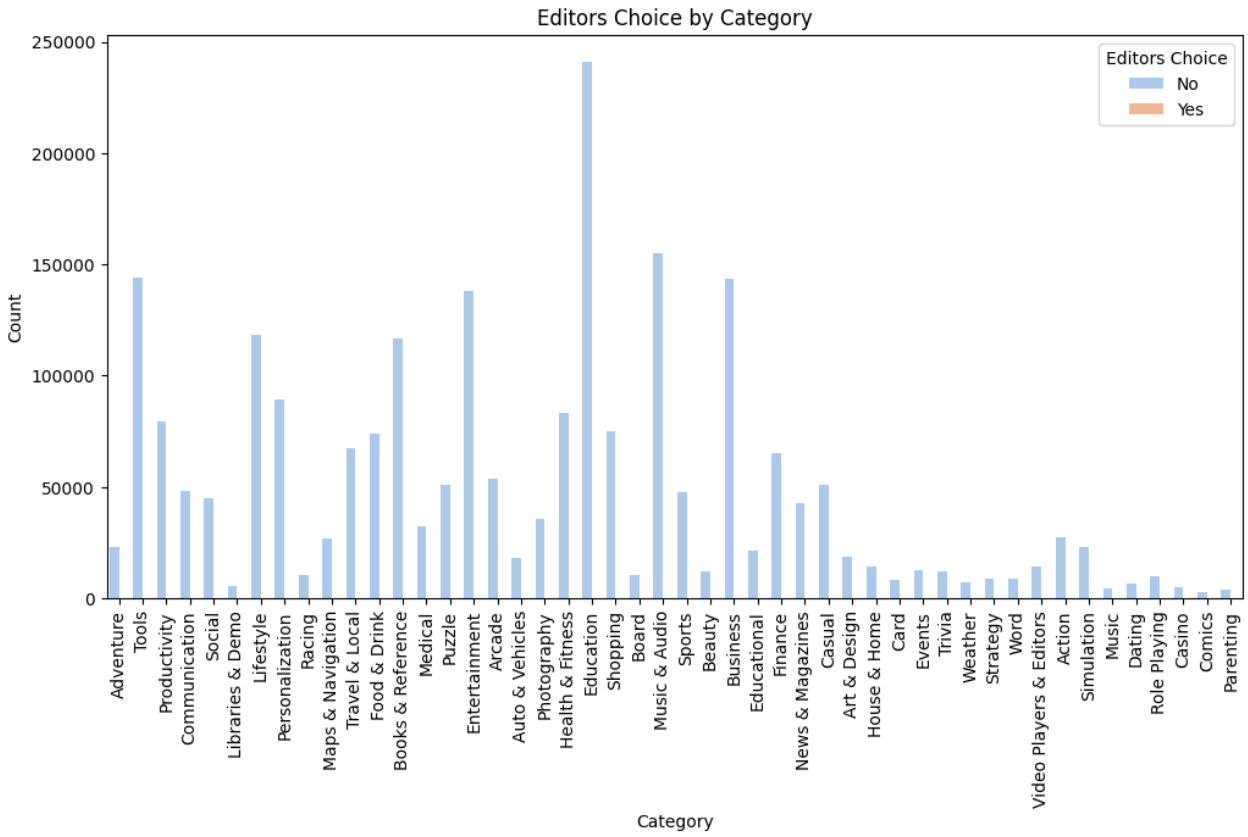

```
plt.figure(figsize=(15, 9))
sns.countplot(data=data1, x='Category', hue='Ad Supported', palette={0: 'navy', 1: 'yellow'})
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Distribution of Ad supported Apps by Category')
plt.xticks(rotation=90)
plt.legend(title='Type', labels=['Yes', 'No'])
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(15, 9))
sns.countplot(data=data1, x='Category', hue='In App Purchases', palette={0: 'navy', 1: 'yellow'
plt.xlabel('Category')
plt.ylabel('Count')
plt.title('Distribution of In app purchases by Category')
plt.xticks(rotation=90)
plt.legend(title='Type', labels=['Yes', 'No'])
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(12, 6))
sns.countplot(data=df, x='Category', hue='Editors Choice', palette='pastel')
plt.xticks(rotation=90)
plt.title('Editors Choice by Category')
plt.xlabel('Category')
plt.ylabel('Count')
plt.legend(title='Editors Choice', loc='upper right', labels=['No', 'Yes'])
plt.show()
```




```
filtered_data = data1[data1['Category'].isin(data1['Category'].unique())]
most_common_version_by_category = filtered_data.groupby('Category')['Minimum
Android'].agg(lambda
print(most_common_version_by_category)
```

Category Minimum Android
0 Action 1.0
1 Adventure 1.0
2 Arcade 1.0
3 Art & Design 1.0
4 Auto & Vehicles 1.0
5 Beauty 1.0
6 Board 1.0
7 Books & Reference 1.0
8 Business 1.0
9 Card 1.0
10 Casino 1.0
11 Casual 1.0
12 Comics 1.0
13 Communication 1.0
14 Dating 1.0
15 Education 1.0

16 Educational 1.0
17 Entertainment 1.0
18 Events 1.0
19 Finance 1.0
20 Food & Drink 1.0
21 Health & Fitness 1.0
22 House & Home 1.0
23 Libraries & Demo 1.0
24 Lifestyle 1.0
25 Maps & Navigation 1.0
26 Medical 1.0
27 Music 1.0
28 Music & Audio 1.0
29 News & Magazines 1.0
30 Parenting 1.0
31 Personalization 1.0
32 Photography 1.0
33 Productivity 1.0
34 Puzzle 1.0
35 Racing 1.0
36 Role Playing 1.0
37 Shopping 1.0
38 Simulation 1.0
39 Social 1.0
40 Sports 1.0
41 Strategy 1.0
42 Tools 1.0
43 Travel & Local 1.0
44 Trivia 1.0
45 Video Players & Editors 1.0
46 Weather 1.0
47 Word 1.0

```
data1['Size (MB)'].unique()
```
```
array([1.000e+01, 2.900e+00, 3.700e+00, ..., 4.050e+02, 3.125e-03, 5.120e+02])
```

```
average_size_by_category = data1.groupby('Category')['Size (MB)'].mean()
print(average_size_by_category)
```
```
Category
Action 42.251477
Adventure 36.847072
Arcade 26.873582
Art & Design 15.568861
```

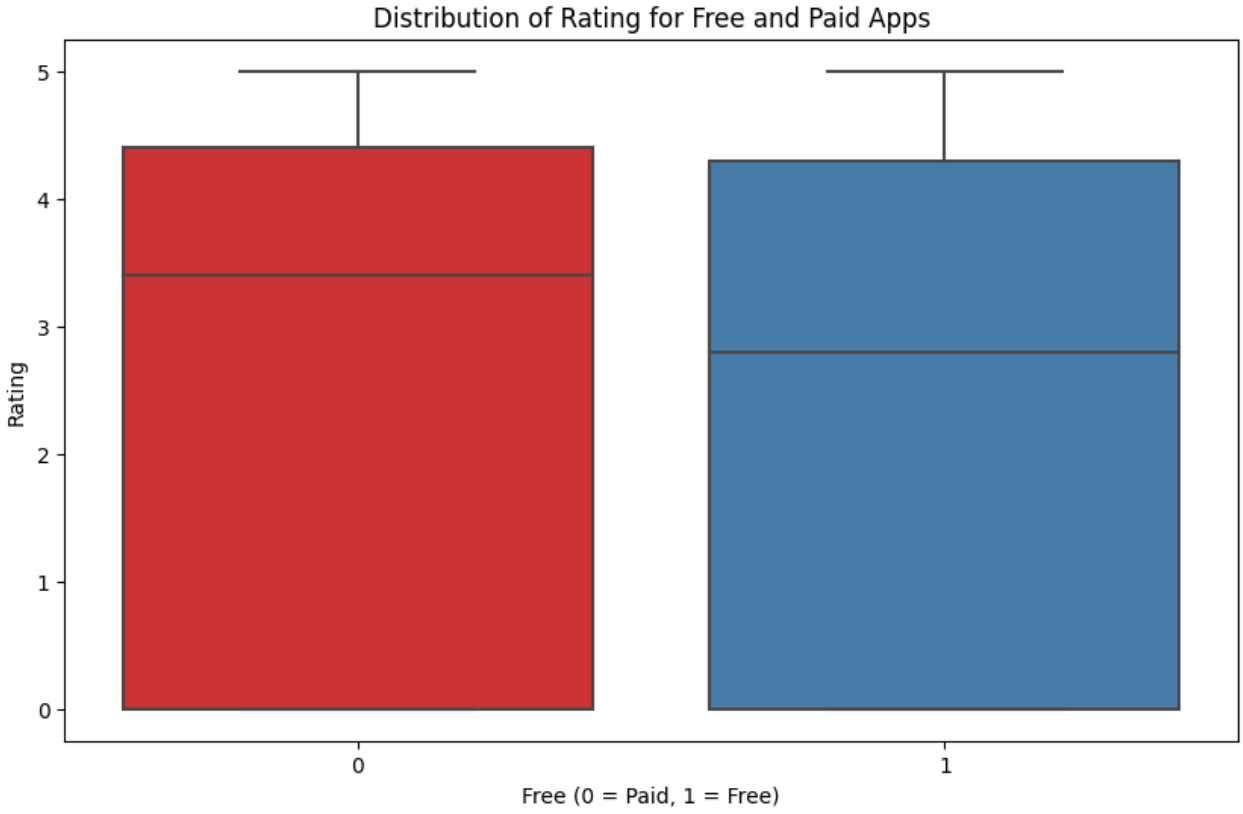
Auto & Vehicles 17.916531
Beauty 14.877986
Board 21.265030
Books & Reference 12.428592
Business 18.088310
Card 27.802284
Casino 38.167712
Casual 27.800519
Comics 26.689134
Communication 16.695281
Dating 16.710119
Education 19.649914
Educational 30.382581
Entertainment 15.194502
Events 19.201215
Finance 18.917456
Food & Drink 15.018047
Health & Fitness 28.800684
House & Home 17.774856
Libraries & Demo 14.503446
Lifestyle 19.740585
Maps & Navigation 15.170561
Medical 20.444543
Music 34.091197
Music & Audio 17.923060
News & Magazines 13.984990
Parenting 21.795012
Personalization 12.403394
Photography 17.199825
Productivity 15.054537
Puzzle 25.403490
Racing 43.967810
Role Playing 51.218730
Shopping 15.096549
Simulation 45.195717
Social 18.567127
Sports 20.534333
Strategy 41.311633
Tools 10.946132
Travel & Local 20.365576
Trivia 21.252847
Video Players & Editors 23.168931
Weather 22.600399
Word 23.494040

Name: Size (MB), dtype: float64

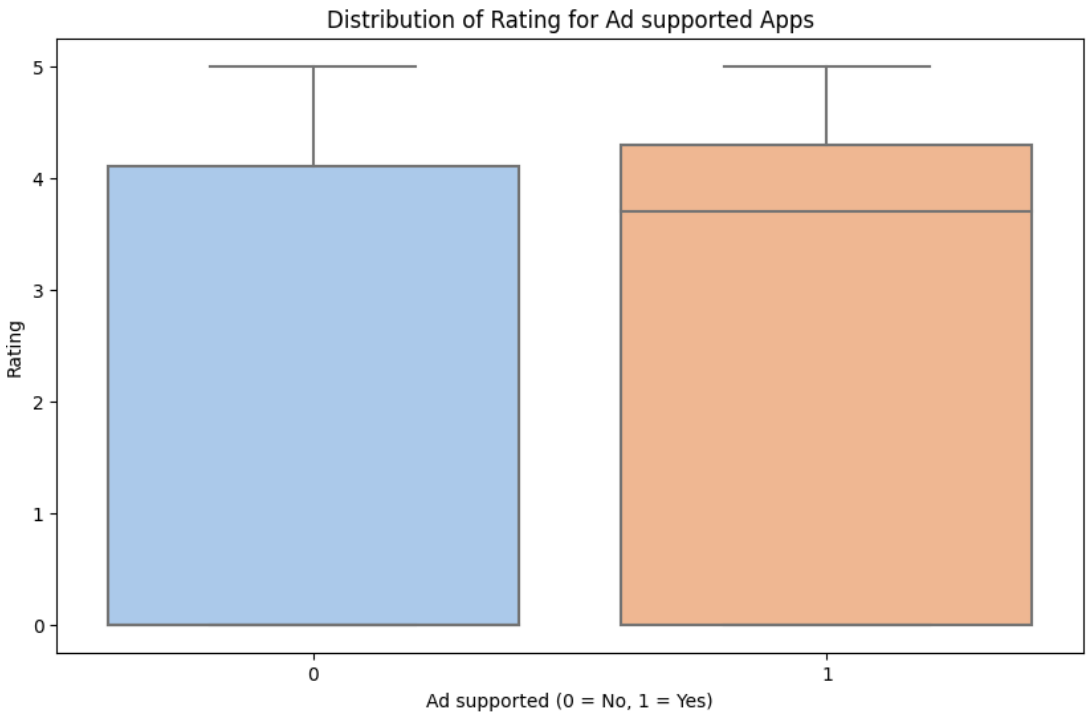
```
data1['Rating'].unique()
```

```
array([0. , 4.4, 5. , 4.5, 2. , 4.7, 4.9, 3.9, 3.7, 4.2, 3.4, 3.8, 4.6,
4.1, 2.5, 2.3, 2.1, 2.7, 4.3, 3.1, 4.8, 3.3, 4. , 2.4, 3.2, 3.6,
2.6, 1.5, 3.5, 2.9, 1.3, 2.8, 3. , 2.2, 1.8, 1.4, 1.2, 1.9, 1.7,
1.6, 1. , 1.1])
```

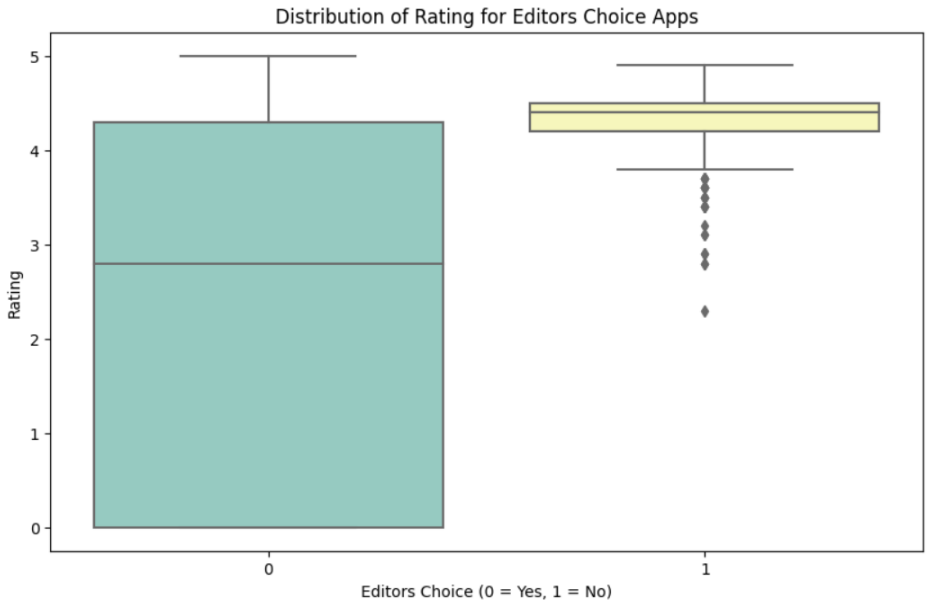
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data1, x='Free', y='Rating', palette='Set1')
plt.title('Distribution of Rating for Free and Paid Apps')
plt.xlabel('Free (0 = Paid, 1 = Free)')
plt.ylabel('Rating')
plt.show()
```



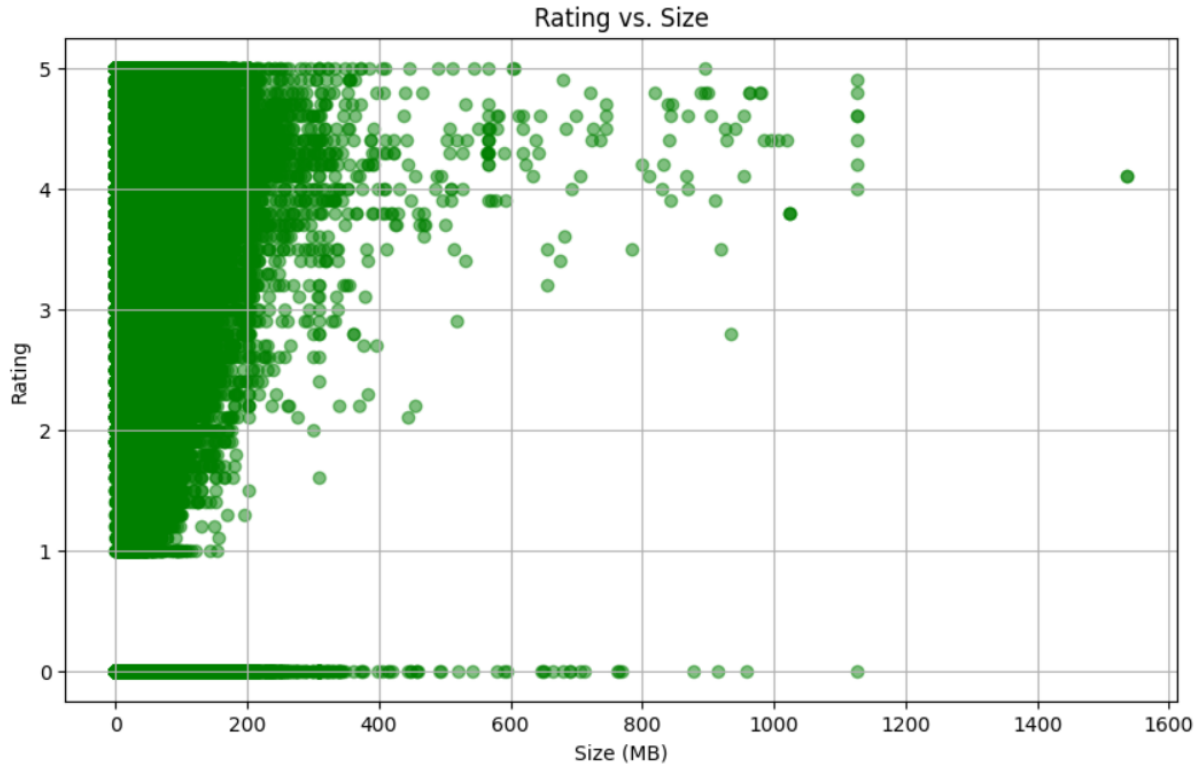

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data1, x='Ad Supported', y='Rating', palette='pastel')
plt.title('Distribution of Rating for Ad supported Apps')
plt.xlabel('Ad supported (0 = No, 1 = Yes)')
plt.ylabel('Rating')
plt.show()
```



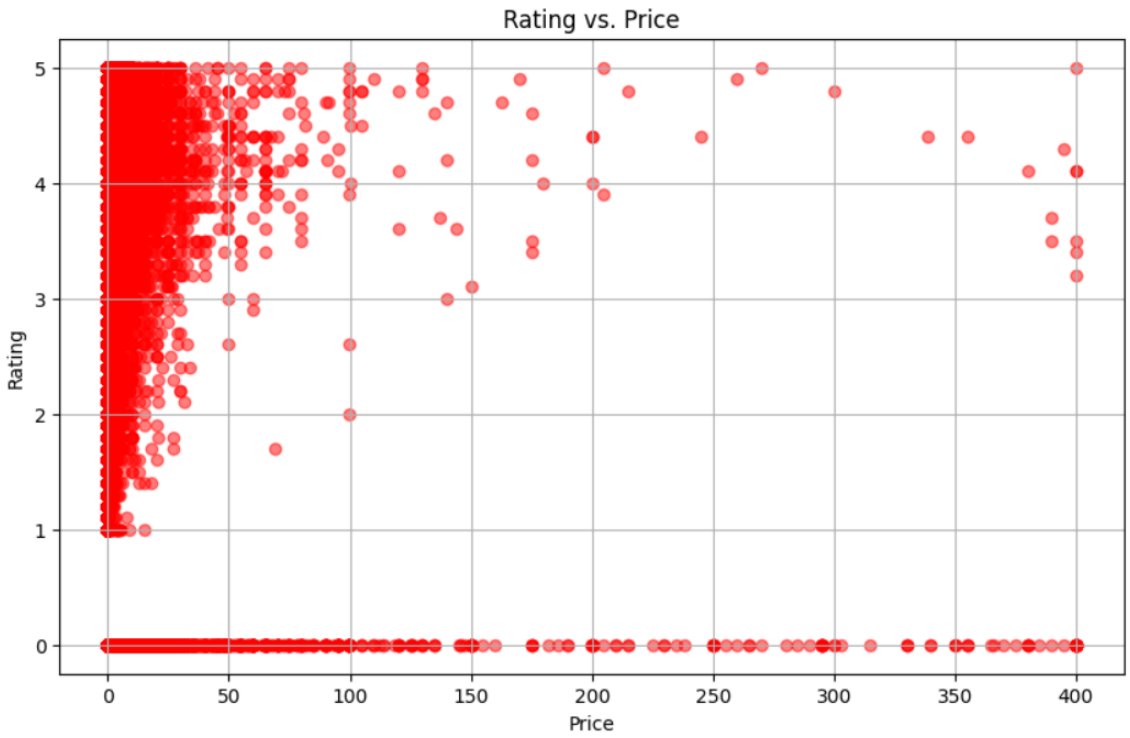

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data1, x='Editors Choice', y='Rating', palette='Set3')
plt.title('Distribution of Rating for Editors Choice Apps')
plt.xlabel('Editors Choice (0 = Yes, 1 = No)')
plt.ylabel('Rating')
plt.show()
```



```
plt.figure(figsize=(10, 6))
plt.scatter(data1['Size (MB)'], data1['Rating'], alpha=0.5, color = 'green')
plt.title('Rating vs. Size')
plt.xlabel('Size (MB)')
plt.ylabel('Rating')
plt.grid(True)
plt.show()
```

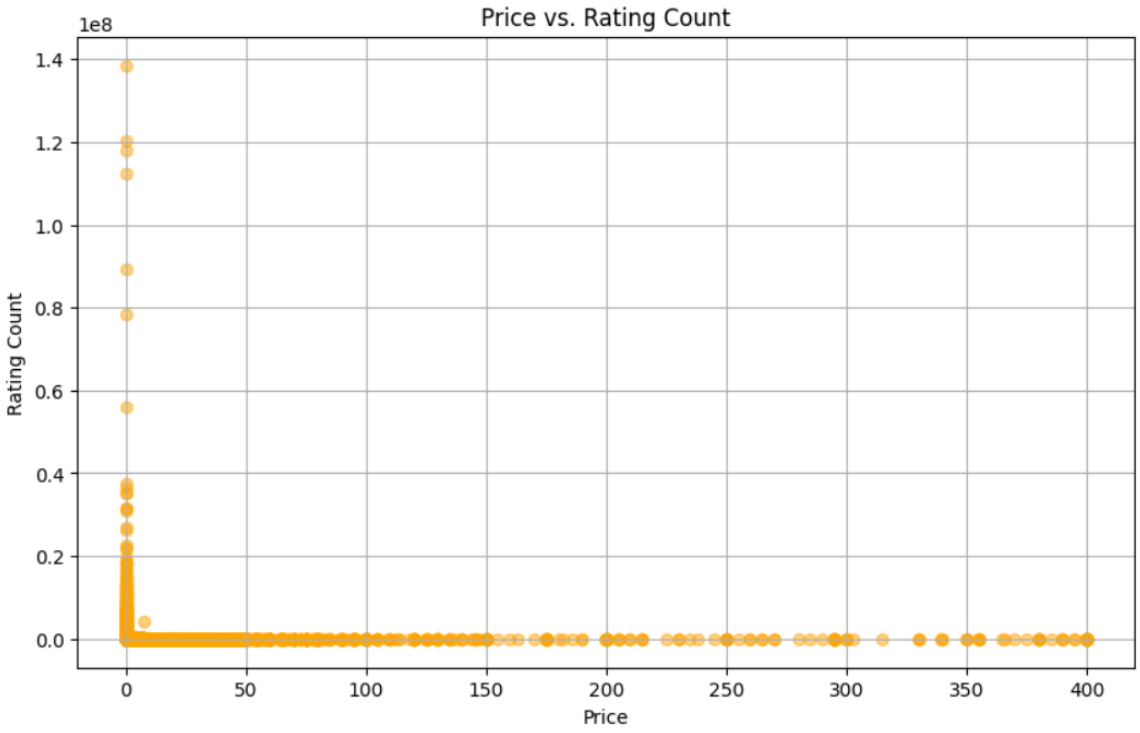



```
plt.figure(figsize=(10, 6))
plt.scatter(data1['Price'], df['Rating'], alpha=0.5, color = 'red')
plt.title('Rating vs. Price')
plt.xlabel('Price')
plt.ylabel('Rating')
plt.grid(True)
plt.show()
```

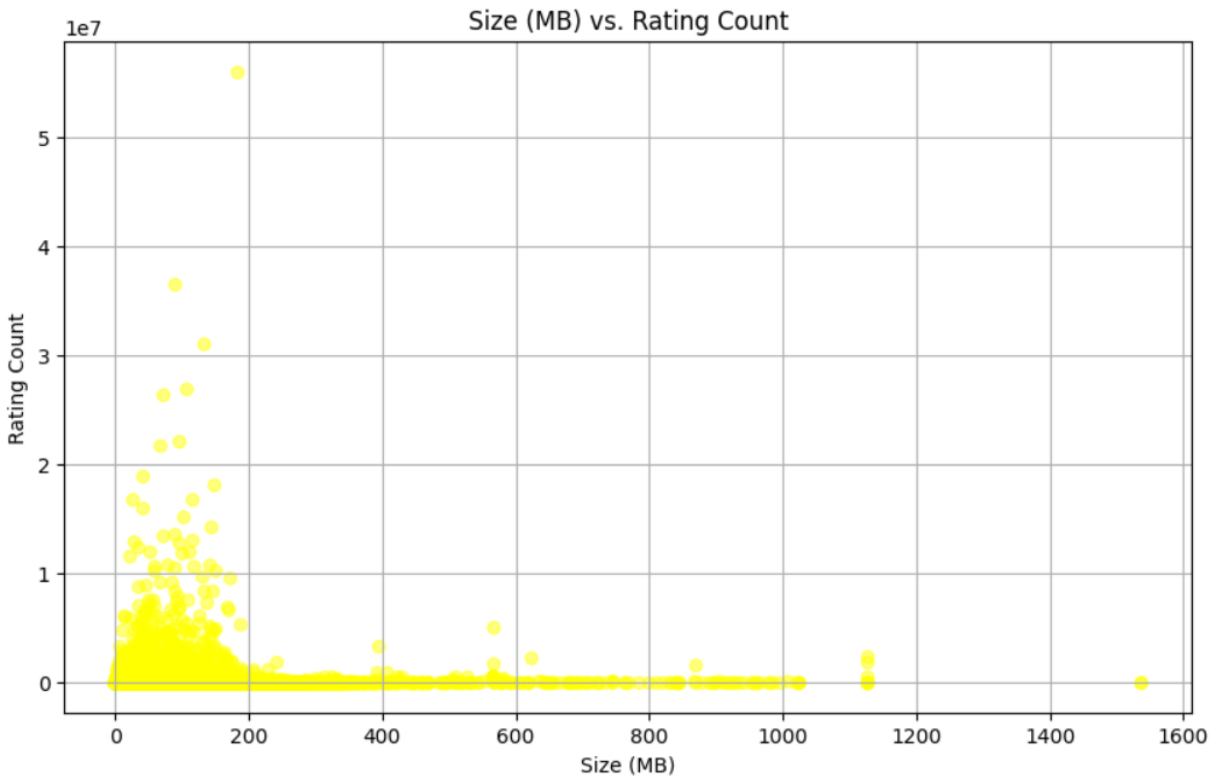



```
data1['Content Rating'].unique()
array(['Everyone', 'Teen', 'Mature 17+', 'Everyone 10+',
 'Adults only 18+', 'Unrated'], dtype=object)
filtered_df = data1[data1['Rating'] > 0]
mode_rating_by_content_rating = filtered_df.groupby('Content Rating')['Rating'].agg(lambda x:
x.mode().iloc[0])
print(mode_rating_by_content_rating)
Content Rating
Adults only 18+ 4.3
Everyone 5.0
Everyone 10+ 4.2
Mature 17+ 4.2
Teen 4.2
Unrated 4.4
Name: Rating, dtype: float64
data1['Rating Count'].unique()
array([0.0000e+00, 6.4000e+01, 5.0000e+00, ..., 8.7553e+04, 7.5960e+04,
 7.8351e+04])
output
plt.figure(figsize=(10, 6))
plt.scatter(data1['Price'], df['Rating Count'], alpha=0.5, color='orange')
plt.title('Price vs. Rating Count')
plt.xlabel('Price')
```

```
plt.ylabel('Rating Count')
plt.grid(True)
plt.show()
```



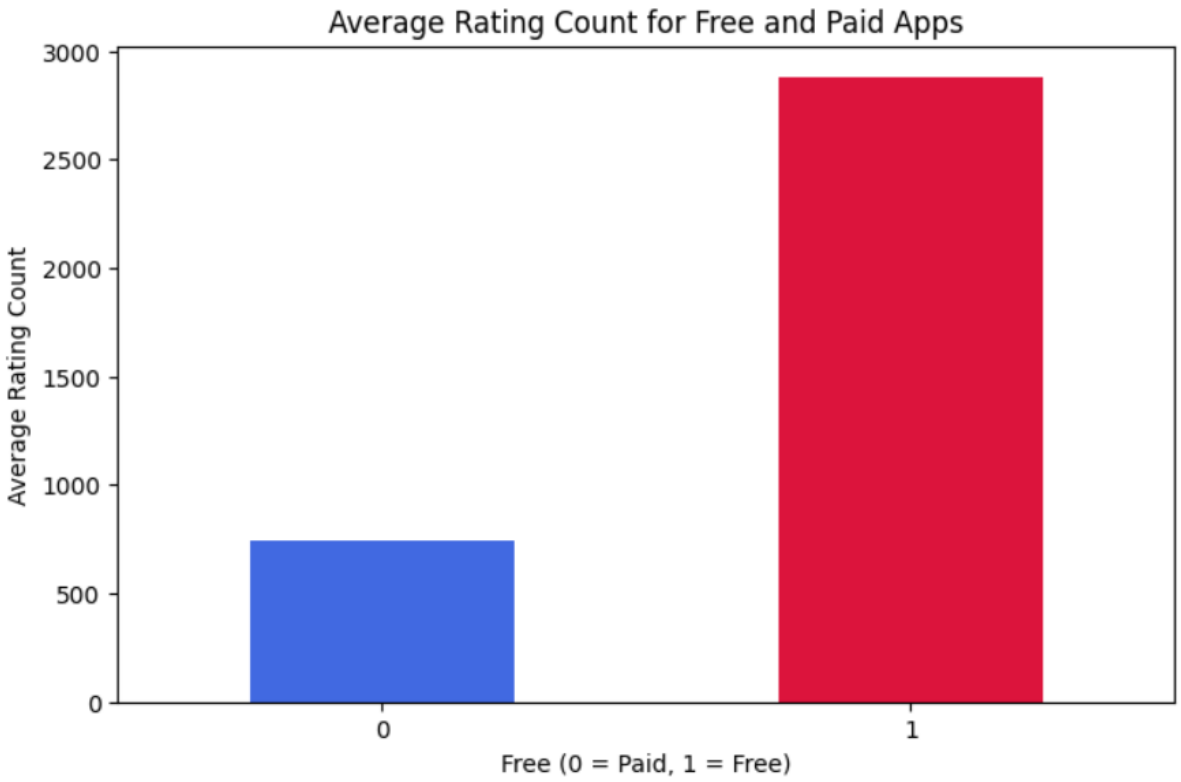

```
plt.figure(figsize=(10, 6))
plt.scatter(data1['Size (MB)'], df['Rating Count'], alpha=0.5, color='yellow')
plt.title('Size (MB) vs. Rating Count')
plt.xlabel('Size (MB)')
plt.ylabel('Rating Count')
plt.grid(True)
plt.show()
```




```
data1['Rating Count'] = pd.to_numeric(data1['Rating Count'], errors='coerce')

average_rating_count_by_free = data1.groupby('Free')['Rating Count'].mean()

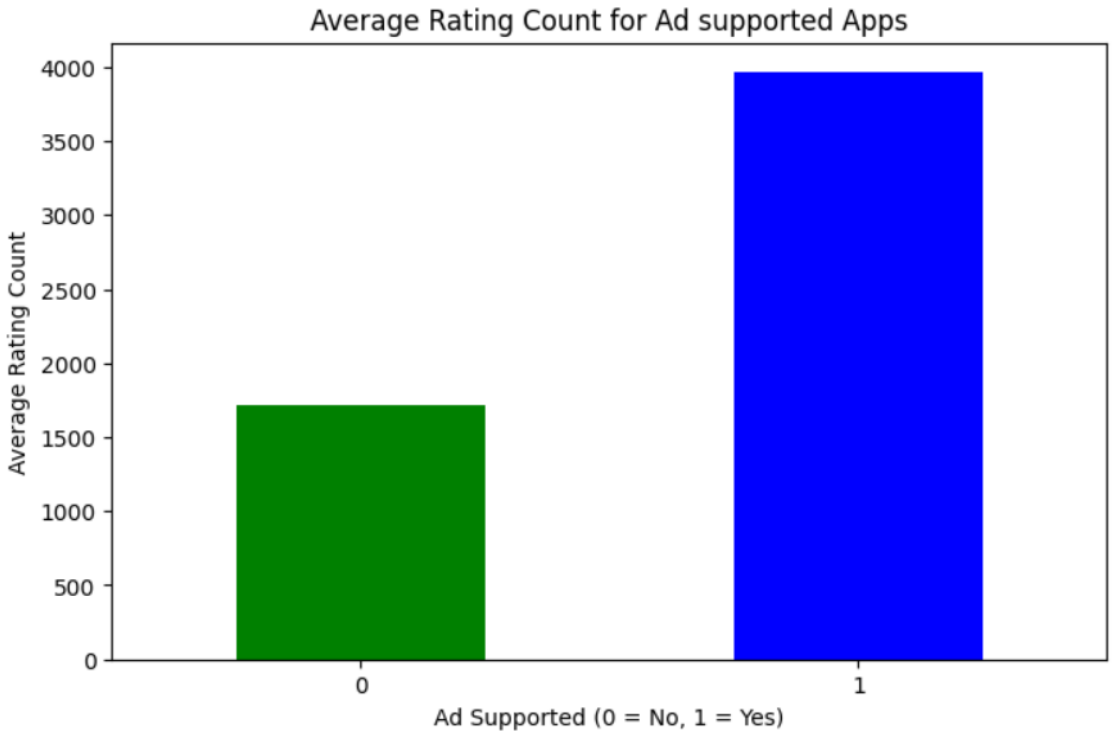

plt.figure(figsize=(8, 5))
average_rating_count_by_free.plot(kind='bar', color=['royalblue', 'crimson'])
plt.title('Average Rating Count for Free and Paid Apps')
plt.xlabel('Free (0 = Paid, 1 = Free)')
plt.ylabel('Average Rating Count')
plt.xticks(rotation=0)
plt.show()
```




```
data1['Rating Count'] = pd.to_numeric(data1['Rating Count'], errors='coerce')

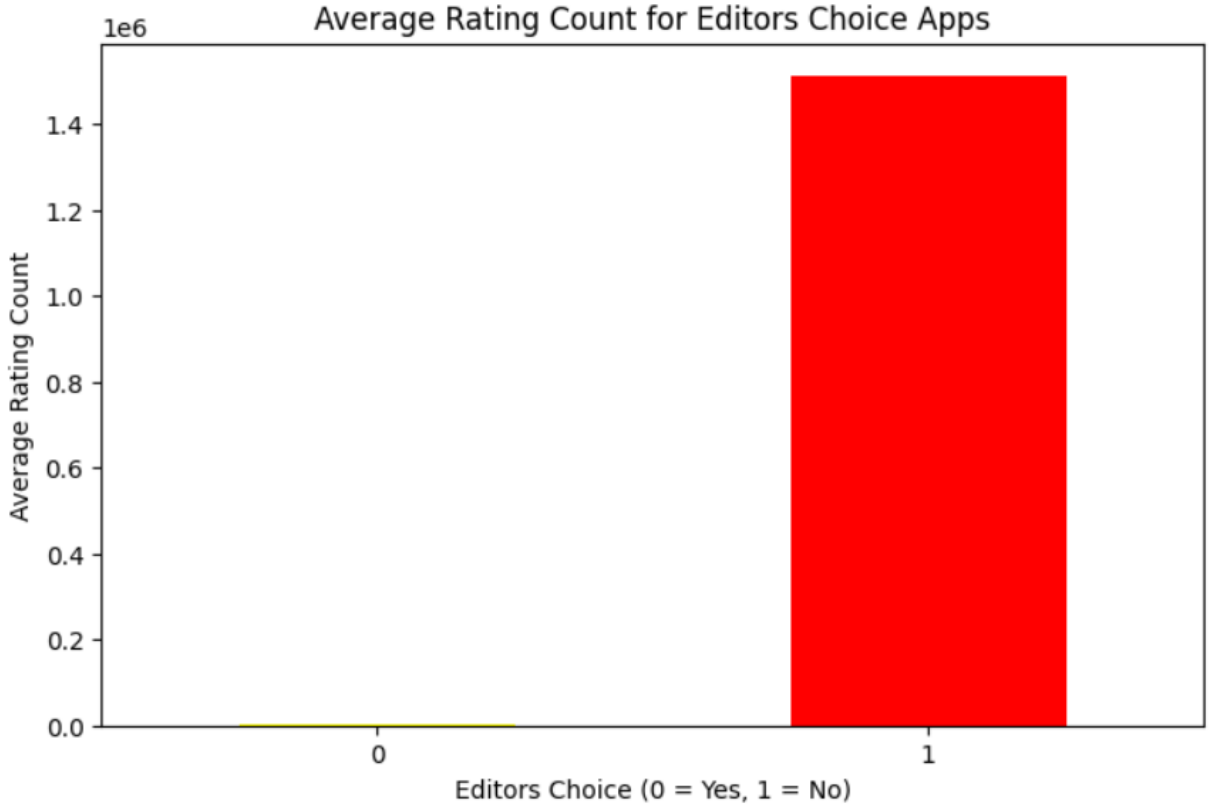
average_rating_count_by_ad = data1.groupby('Ad Supported')['Rating Count'].mean()


plt.figure(figsize=(8, 5))
average_rating_count_by_ad.plot(kind='bar', color=['green', 'blue'])
plt.title('Average Rating Count for Ad supported Apps')
plt.xlabel('Ad Supported (0 = No, 1 = Yes)')
plt.ylabel('Average Rating Count')
plt.xticks(rotation=0)
plt.show()
```



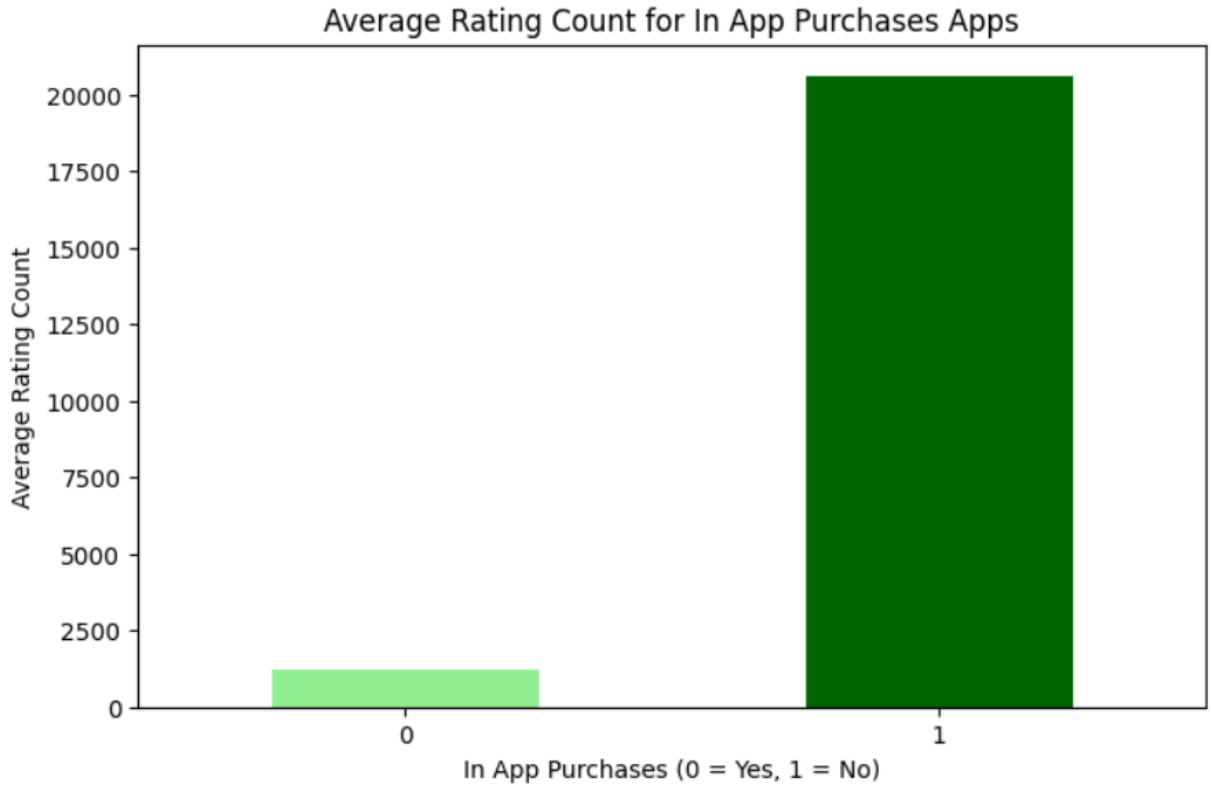

```
average_rating_count_by_ad = data1.groupby('Editors Choice')['Rating Count'].mean()


plt.figure(figsize=(8, 5))
average_rating_count_by_ad.plot(kind='bar', color=['yellow', 'red'])
plt.title('Average Rating Count for Editors Choice Apps')
plt.xlabel('Editors Choice (0 = Yes, 1 = No)')
plt.ylabel('Average Rating Count')
plt.xticks(rotation=0)
plt.show()
```




```
average_rating_count_by_ad = data1.groupby('In App Purchases')['Rating Count'].mean()


plt.figure(figsize=(8, 5))
average_rating_count_by_ad.plot(kind='bar', color=['lightgreen', 'darkgreen'])
plt.title('Average Rating Count for In App Purchases Apps')
plt.xlabel('In App Purchases (0 = Yes, 1 = No)')
plt.ylabel('Average Rating Count')
plt.xticks(rotation=0)
plt.show()
```




```
max_rating_count_by_category = data1.groupby('Category')['Rating Count'].max()
print(max_rating_count_by_category)
```

```
Category
Action                     89177097.0
Adventure                  22148032.0
Arcade                      7213140.0
Art & Design                5213141.0
Auto & Vehicles             1824861.0
Beauty                       780880.0
Board                       7630921.0
Books & Reference           4224876.0
Business                    3498904.0
Card                        2587465.0
Casino                      1830022.0
Casual                     31476637.0
Comics                      1746185.0
Communication            138557570.0
Dating                      5503151.0
Education                  11103561.0
Educational                 1788412.0
Entertainment              12016421.0
Events                        77398.0
```

```
Finance                      8851930.0
Food & Drink                 6260813.0
Health & Fitness             5949473.0
House & Home                  542826.0
Libraries & Demo              455955.0
Lifestyle                    8017109.0
Maps & Navigation            8649676.0
Medical                       386152.0
Music                        2488885.0
Music & Audio               22752840.0
News & Magazines             1402711.0
Parenting                    1366658.0
Personalization              8798409.0
Photography                 35369236.0
Productivity                11961069.0
Puzzle                       4908403.0
Racing                      10205496.0
Role Playing                 3187733.0
Shopping                    18956756.0
Simulation                   8424191.0
Social                     120206190.0
Sports                      21754025.0
Strategy                    56025424.0
Tools                       35128398.0
Travel & Local              13576045.0
Trivia                       7345935.0
Video Players & Editors    112440547.0
Weather                      2528696.0
Word                         2251455.0
Name: Rating Count, dtype: float64
```

```
category_total_counts = data1.groupby('Category')['Rating Count'].sum()
print(category_total_counts)
```

```
Category
Action                558142904.0
Adventure              130442286.0
Arcade                 245086756.0
Art & Design            15949376.0
Auto & Vehicles         12674079.0
Beauty                   4484425.0
Board                   60648614.0
Books & Reference        77207604.0
Business                62321995.0
Card                    50865494.0
Casino                  48804026.0
```

```
Casual                  359404856.0
Comics                    6892265.0
Communication           470077239.0
Dating                  13857505.0
Education               104813333.0
Educational              29404189.0
Entertainment           224992777.0
Events                    903566.0
Finance                 191582077.0
Food & Drink             49788184.0
Health & Fitness         79150819.0
House & Home              6921894.0
Libraries & Demo          3498401.0
Lifestyle               94107518.0
Maps & Navigation       63655153.0
Medical                 12453334.0
Music                   24808668.0
Music & Audio           195761238.0
News & Magazines         32345825.0
Parenting                6562245.0
Personalization         123449553.0
Photography             213045620.0
Productivity            150293205.0
Puzzle                  210993016.0
Racing                  181138685.0
Role Playing            140598286.0
Shopping                210427782.0
Simulation              230768842.0
Social                  470373435.0
Sports                  217547049.0
Strategy                233340521.0
Tools                   472782203.0
Travel & Local           73350019.0
Trivia                  37403235.0
Video Players & Editors  264972790.0
Weather                  37450762.0
Word                     55134542.0
Name: Rating Count, dtype: float64

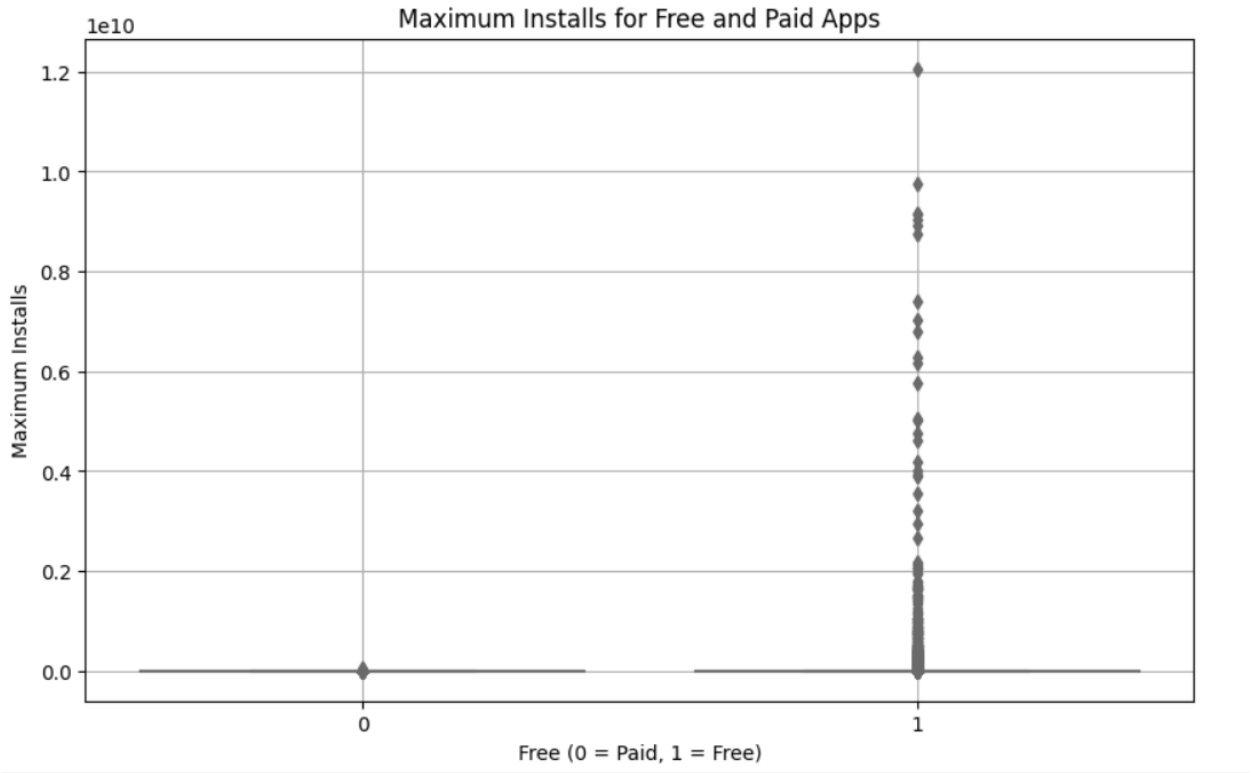
content_total_counts = data1.groupby('Content Rating')['Rating Count'].sum()
formatted_numbers = {key: f'{value:,.0f}' for key, value in content_total_counts.items()}
for category, count in formatted_numbers.items():
print(f"{category}: {count}")
Adults only 18+: 861,940
```

Everyone: 4,059,690,672
Everyone 10+: 524,055,627
Mature 17+: 332,787,413
Teen: 1,643,242,513
Unrated: 40,025

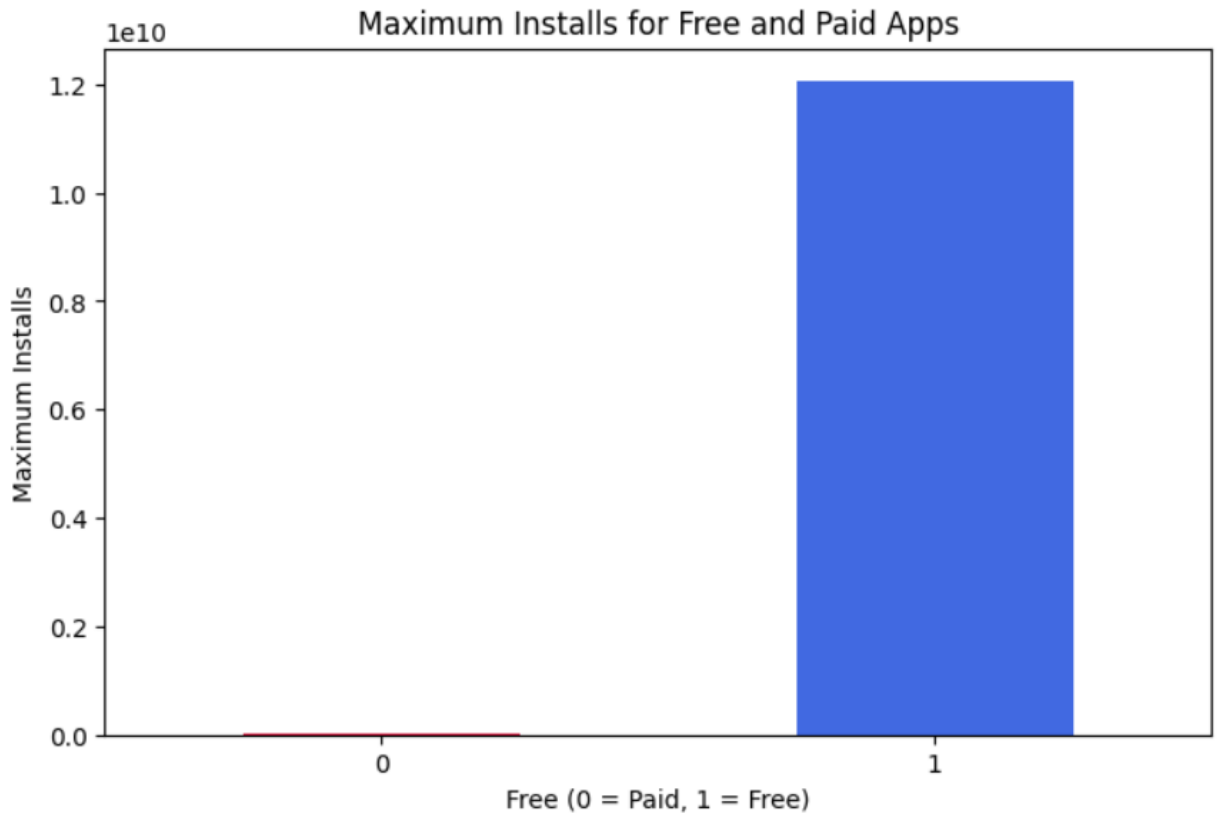
```
data1['Maximum Installs'].unique()
```

```
array([ 15, 7662, 58, ..., 590469, 7274660, 814548])
```

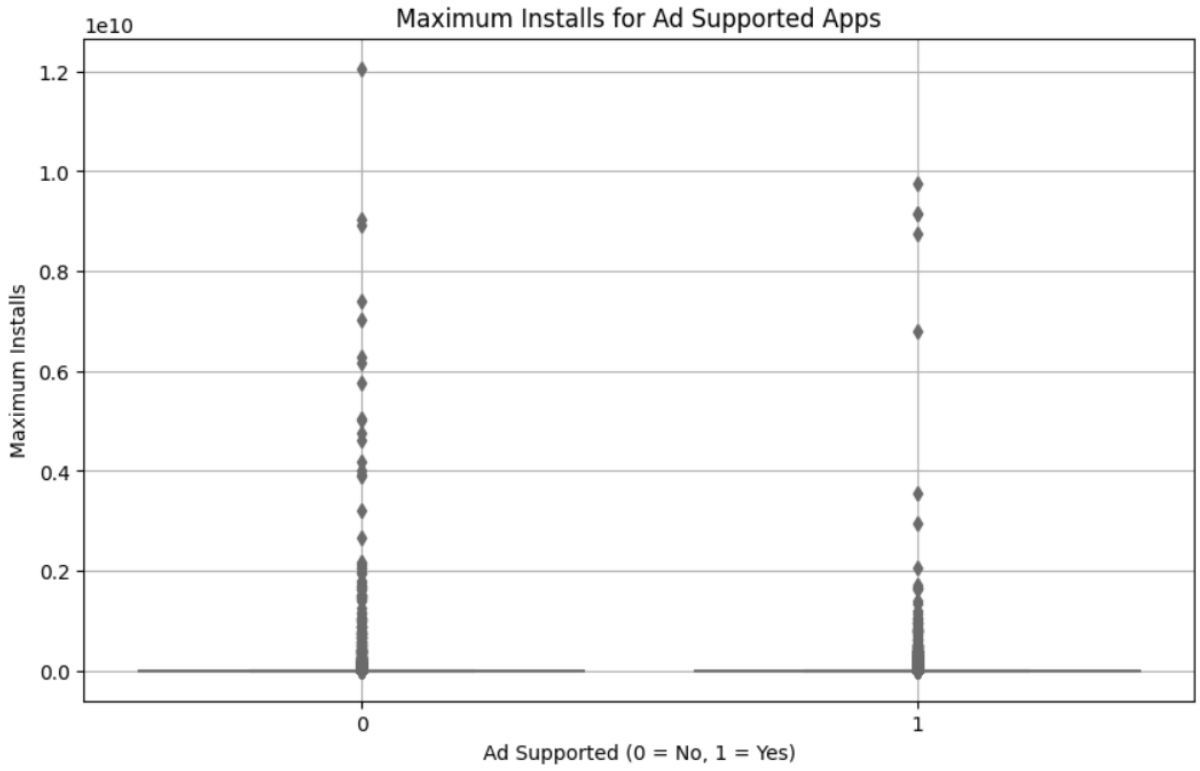
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='Free', y='Maximum Installs', palette='Set3')
plt.title('Maximum Installs for Free and Paid Apps')
plt.xlabel('Free (0 = Paid, 1 = Free)')
plt.ylabel('Maximum Installs')
plt.grid(True)
plt.show()
```



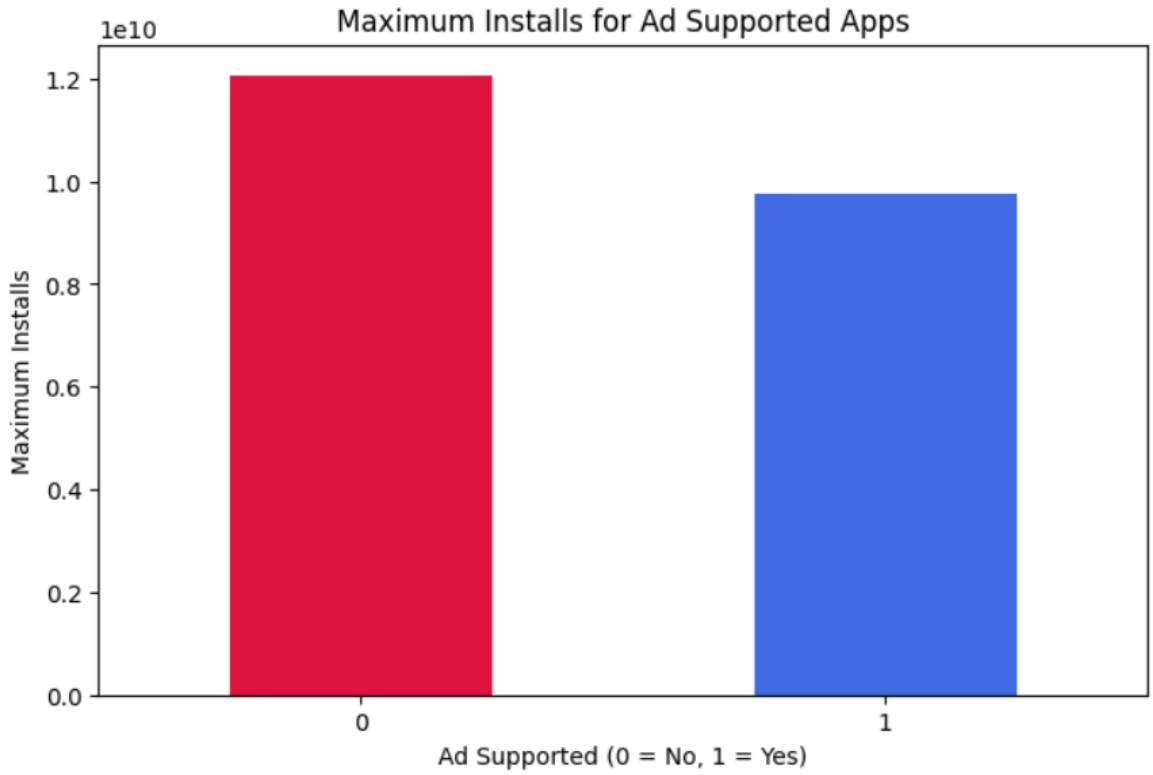

```
max_installs_by_free = data1.groupby('Free')['Maximum Installs'].max()
plt.figure(figsize=(8, 5))
max_installs_by_free.plot(kind='bar', color=[ 'crimson', 'royalblue'])
plt.title('Maximum Installs for Free and Paid Apps')
plt.xlabel('Free (0 = Paid, 1 = Free)')
plt.ylabel('Maximum Installs')
plt.xticks(rotation=0)
plt.show()
```



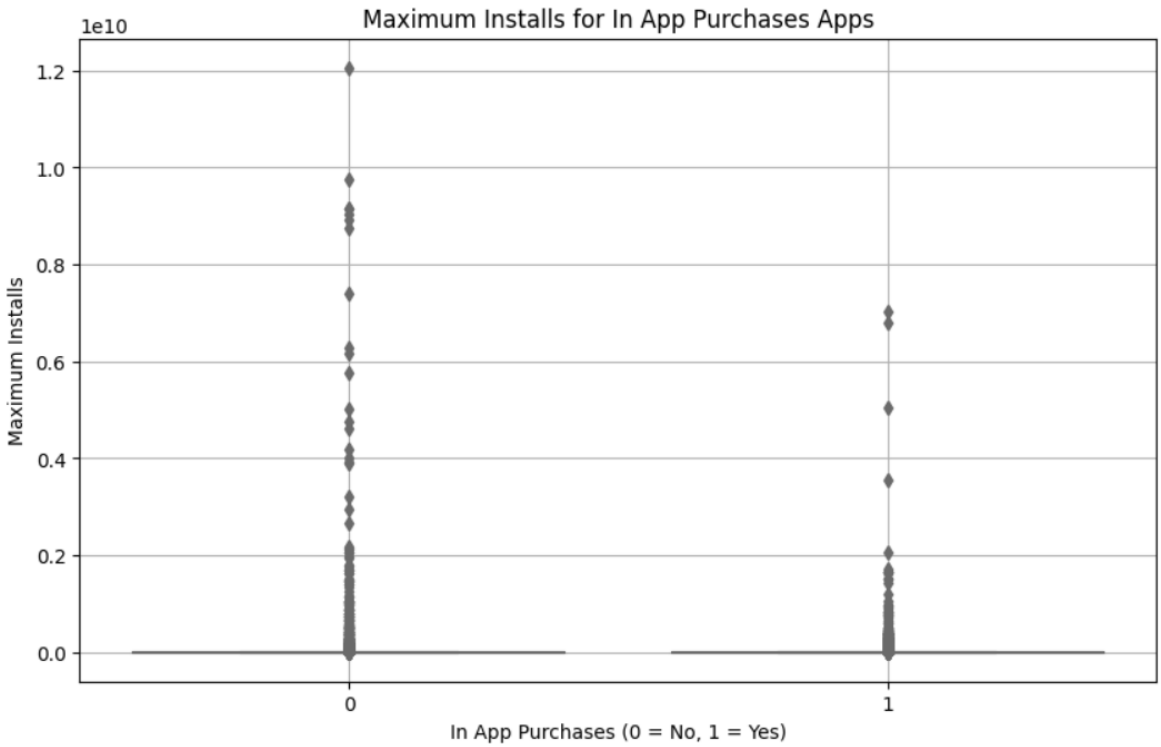

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='Ad Supported', y='Maximum Installs', palette='Set3')
plt.title('Maximum Installs for Ad Supported Apps')
plt.xlabel('Ad Supported (0 = No, 1 = Yes)')
plt.ylabel('Maximum Installs')
plt.grid(True)
plt.show()
```



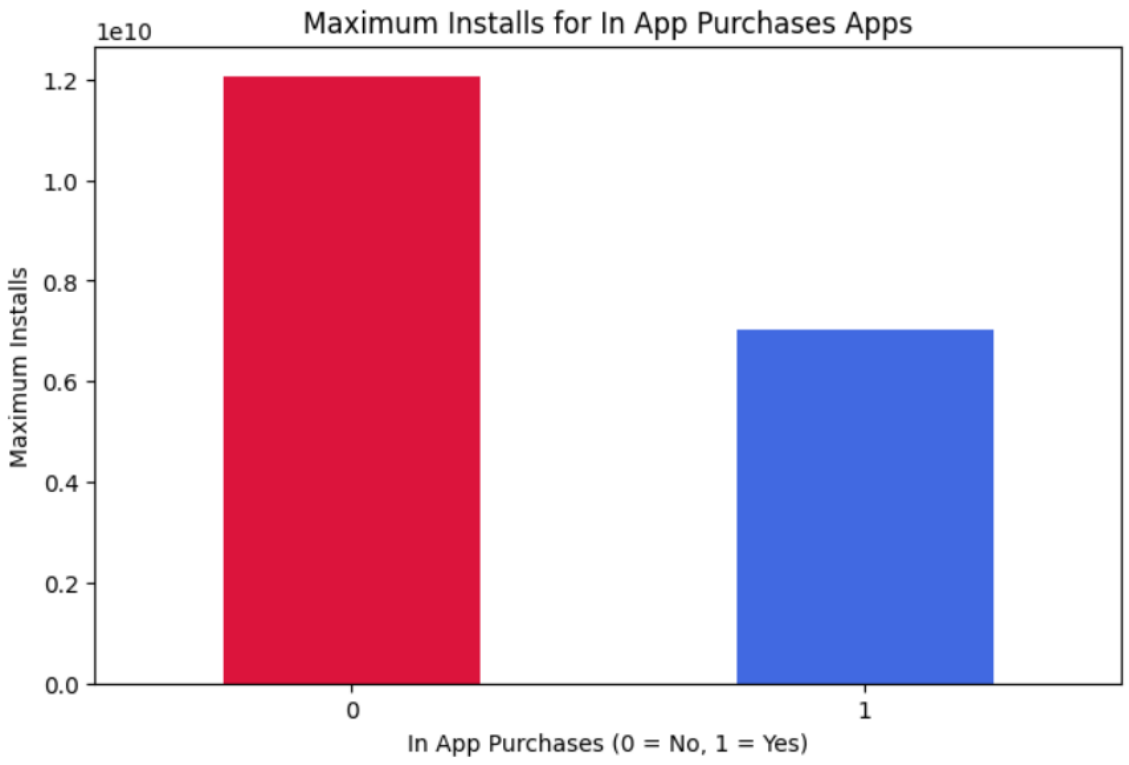

```
max_installs_by_free = data1.groupby('Ad Supported')['Maximum Installs'].max()
plt.figure(figsize=(8, 5))
max_installs_by_free.plot(kind='bar', color=[ 'crimson', 'royalblue'])
plt.title('Maximum Installs for Ad Supported Apps')
plt.xlabel('Ad Supported (0 = No, 1 = Yes)')
plt.ylabel('Maximum Installs')
plt.xticks(rotation=0)
plt.show()
```



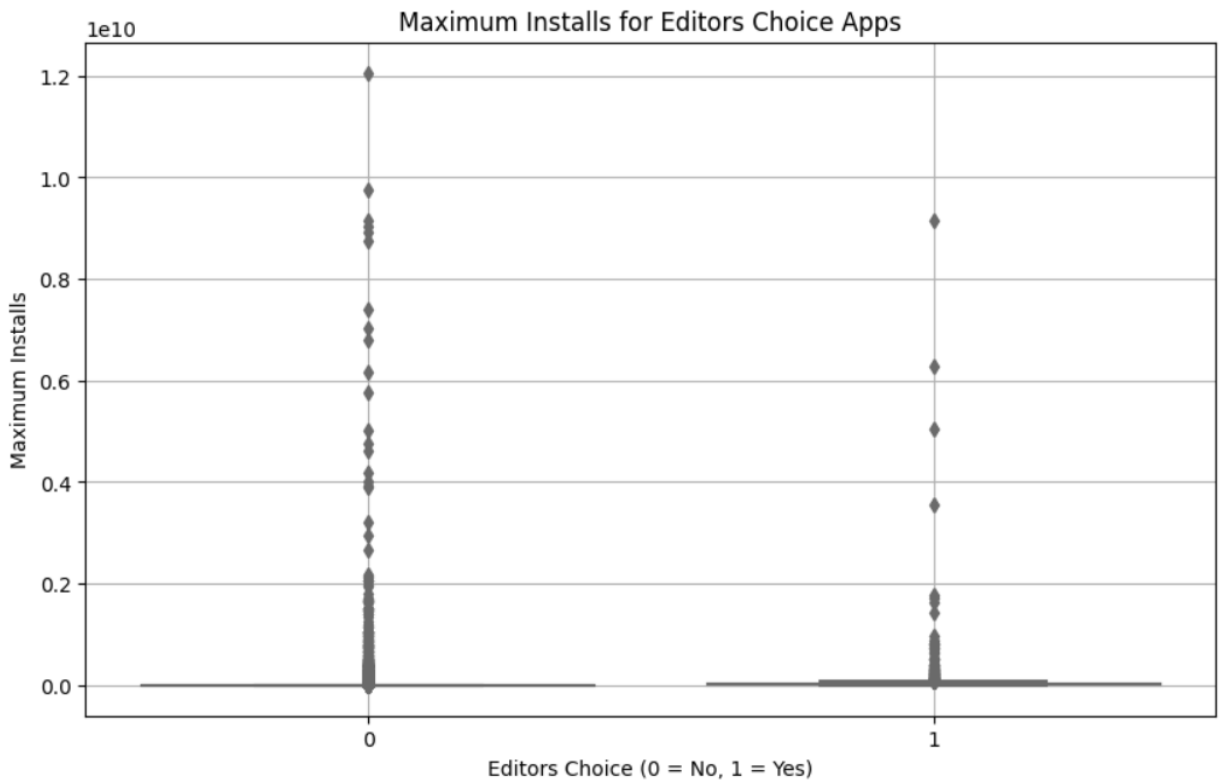

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data1, x='In App Purchases', y='Maximum Installs', palette='Set3')
plt.title('Maximum Installs for In App Purchases Apps')
plt.xlabel('In App Purchases (0 = No, 1 = Yes)')
plt.ylabel('Maximum Installs')
plt.grid(True)
plt.show()
```



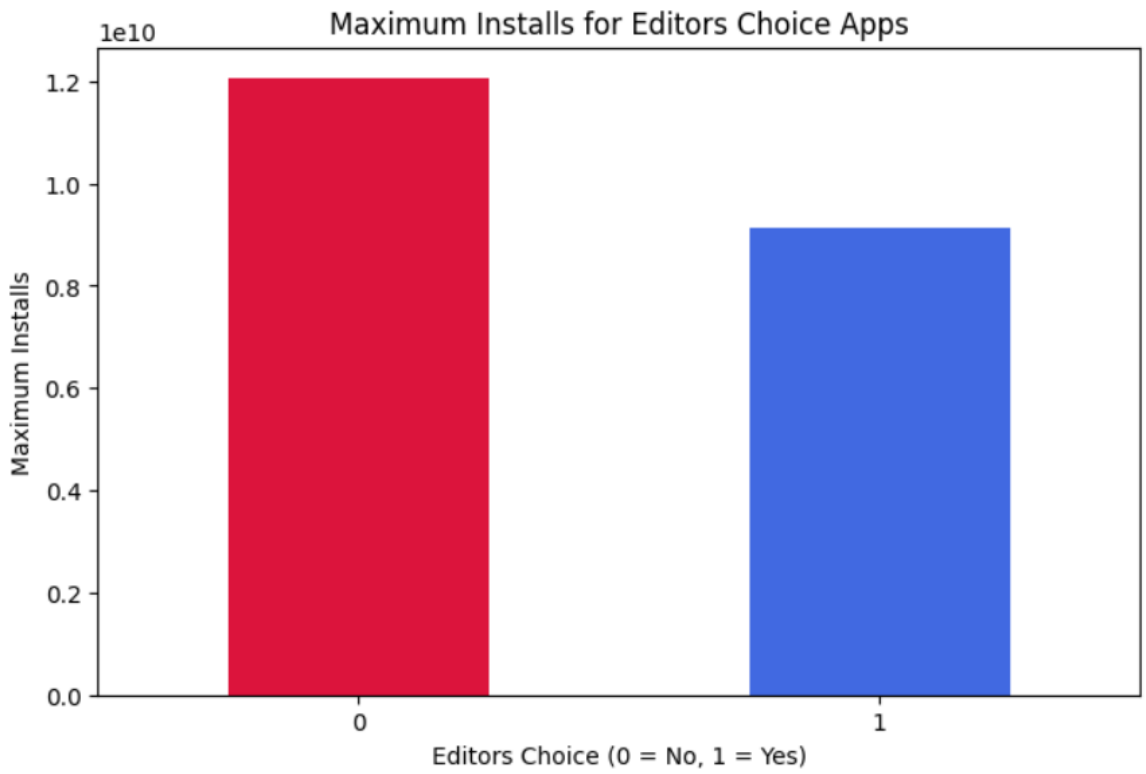

```
plt.figure(figsize=(8, 5))
max_installs_by_free.plot(kind='bar', color=[ 'crimson', 'royalblue'])
plt.title('Maximum Installs for In App Purchases Apps')
plt.xlabel('In App Purchases (0 = No, 1 = Yes)')
plt.ylabel('Maximum Installs')
plt.xticks(rotation=0)
plt.show()
```



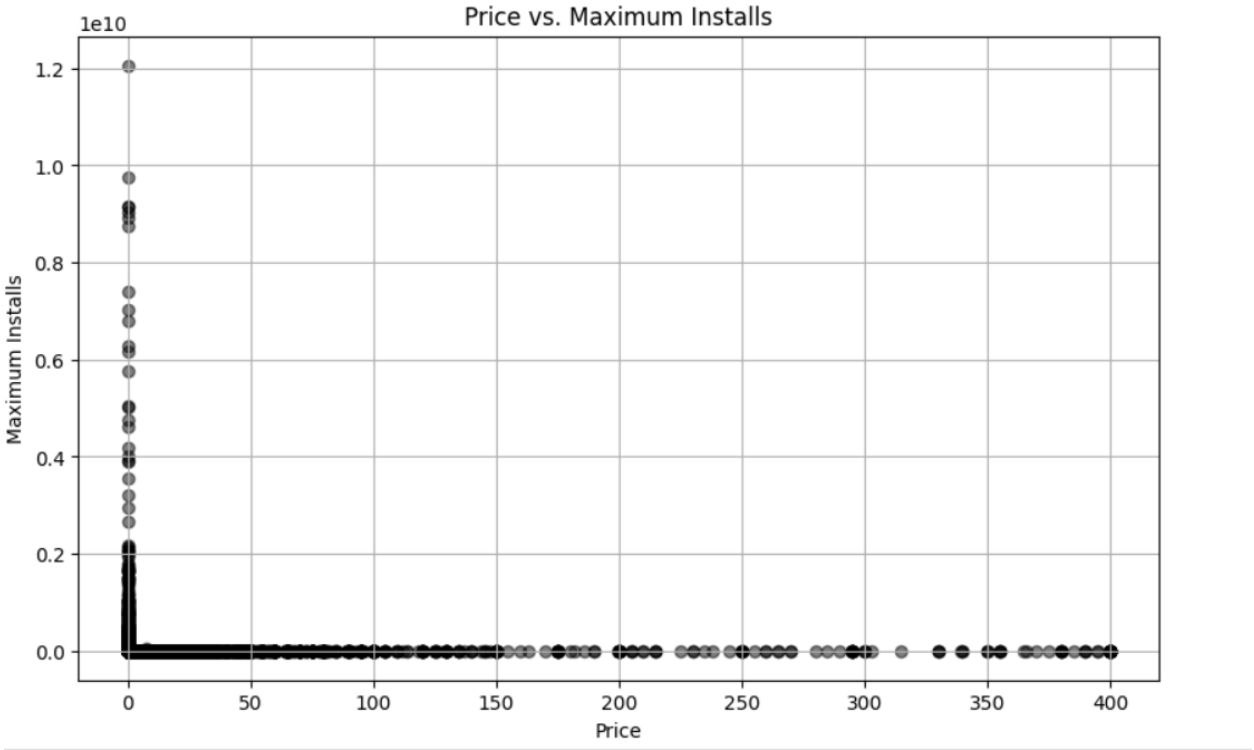

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data1, x='Editors Choice', y='Maximum Installs', palette='Set3')
plt.title('Maximum Installs for Editors Choice Apps')
plt.xlabel('Editors Choice (0 = No, 1 = Yes)')
plt.ylabel('Maximum Installs')
plt.grid(True)
plt.show()
```



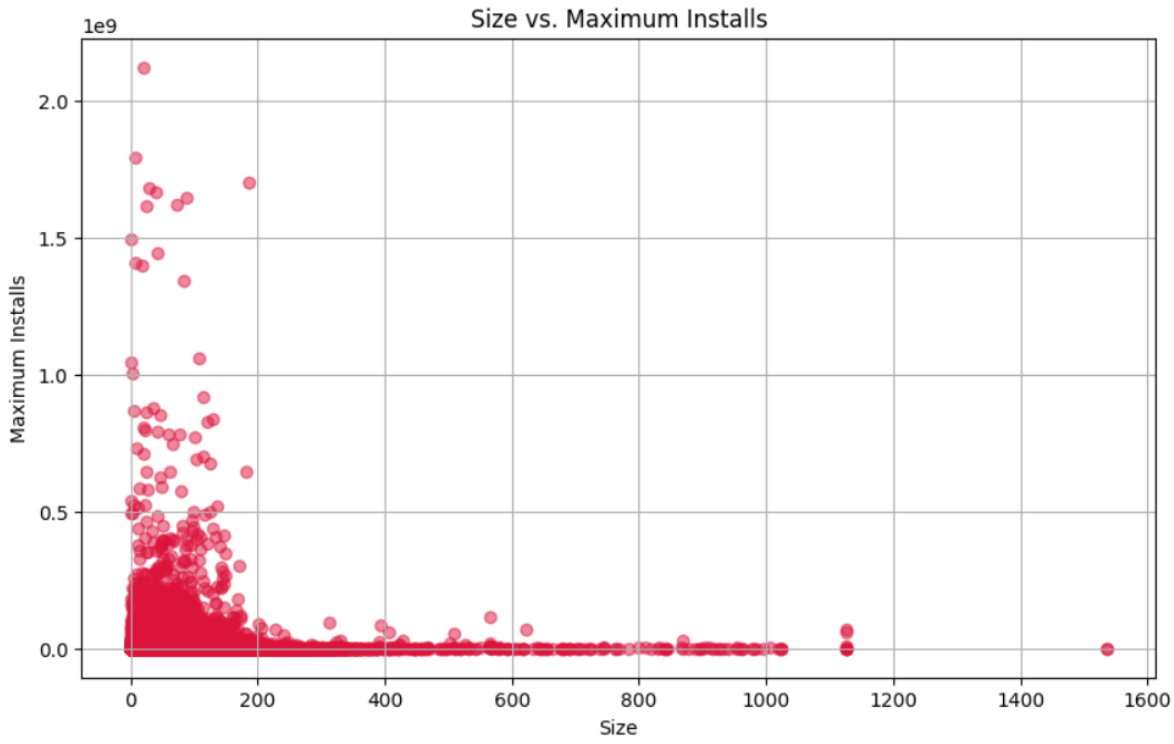

```
max_installs_by_free = data1.groupby('Editors Choice')['Maximum Installs'].max()
plt.figure(figsize=(8, 5))
max_installs_by_free.plot(kind='bar', color=[ 'crimson', 'royalblue'])
plt.title('Maximum Installs for Editors Choice Apps')
plt.xlabel('Editors Choice (0 = No, 1 = Yes)')
plt.ylabel('Maximum Installs')
plt.xticks(rotation=0)
plt.show()
```




```
plt.figure(figsize=(10, 6))
plt.scatter(data1['Price'], data1['Maximum Installs'], alpha=0.5, color = 'black')
plt.title('Price vs. Maximum Installs')
plt.xlabel('Price')
plt.ylabel('Maximum Installs')
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(10, 6))
plt.scatter(data1['Size (MB)'], data1['Maximum Installs'], alpha=0.5, color = 'crimson')
plt.title('Size vs. Maximum Installs')
plt.xlabel('Size')
plt.ylabel('Maximum Installs')
plt.grid(True)
plt.show()
```



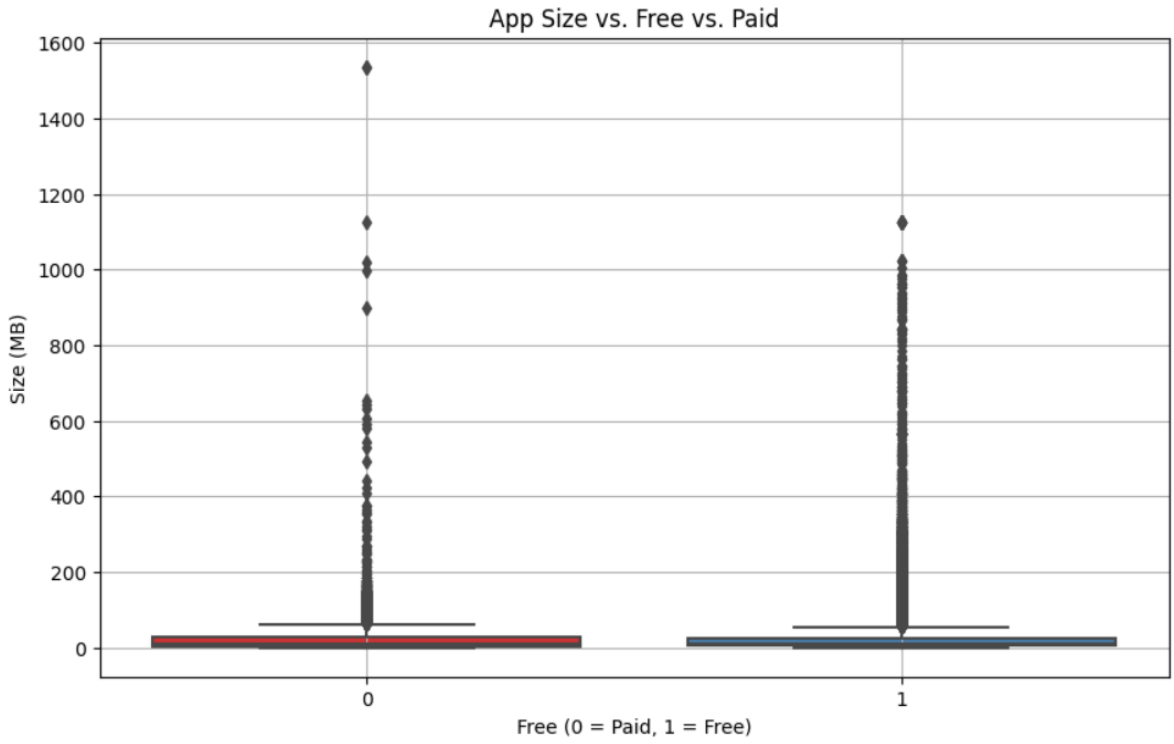

```
total_installs_by_content_rating = data1.groupby('Content Rating')['Maximum Installs'].sum()
print(total_installs_by_content_rating)
Content Rating
Adults only 18+ 28542966
Everyone 541403361237
Everyone 10+ 31428070851
Mature 17+ 21459471726
Teen 146282874305
Unrated 6225911
Name: Maximum Installs, dtype: int64
total_installs_by_year = data1.groupby('Released_Year')['Maximum Installs'].sum()
print(total_installs_by_year)
Released_Year
2009 2479149
2010 65883297222
2011 47547052565
2012 76464060954
2013 78061252836
2014 63407921855
2015 77461661758
2016 64676065638
2017 72382272496
2018 61657979801
```

2019 49258160793
2020 36584087490
2021 47222254439
Name: Maximum Installs, dtype: int64

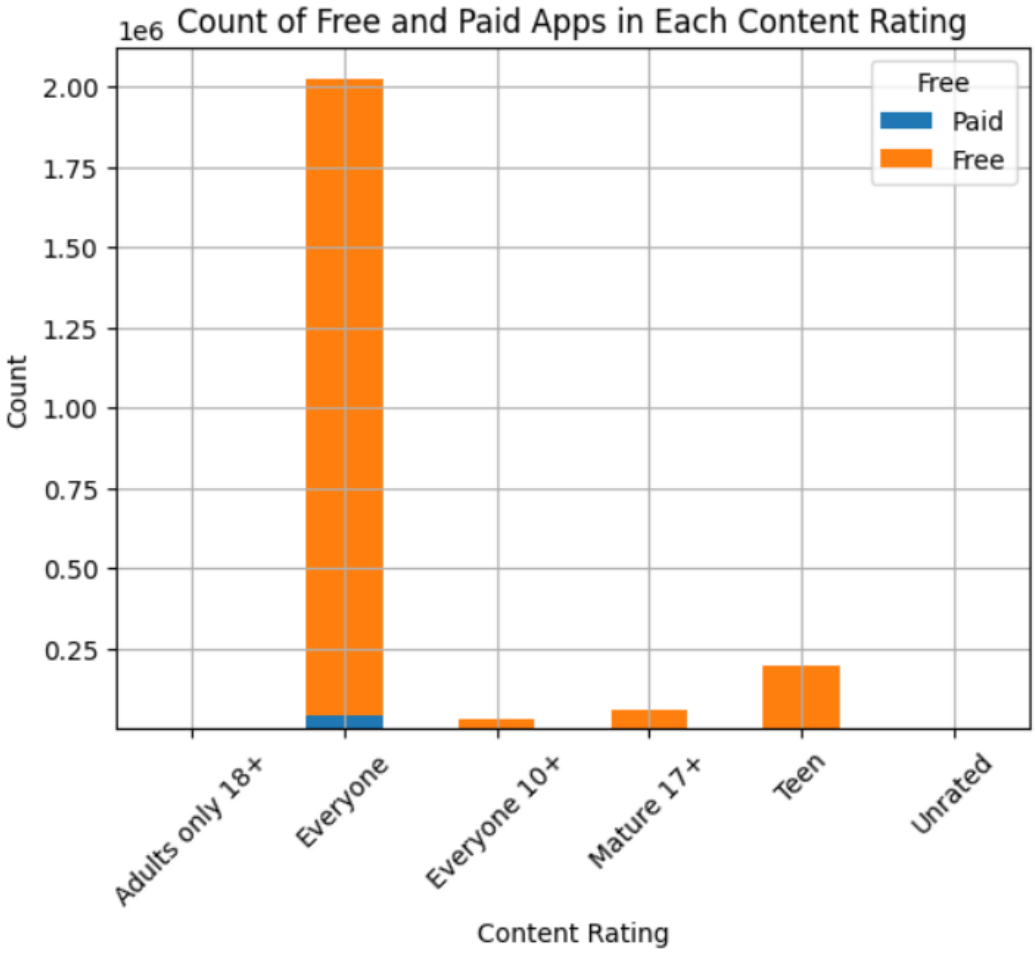
```
data1['Free'].unique()
```

```
array([1, 0])
```

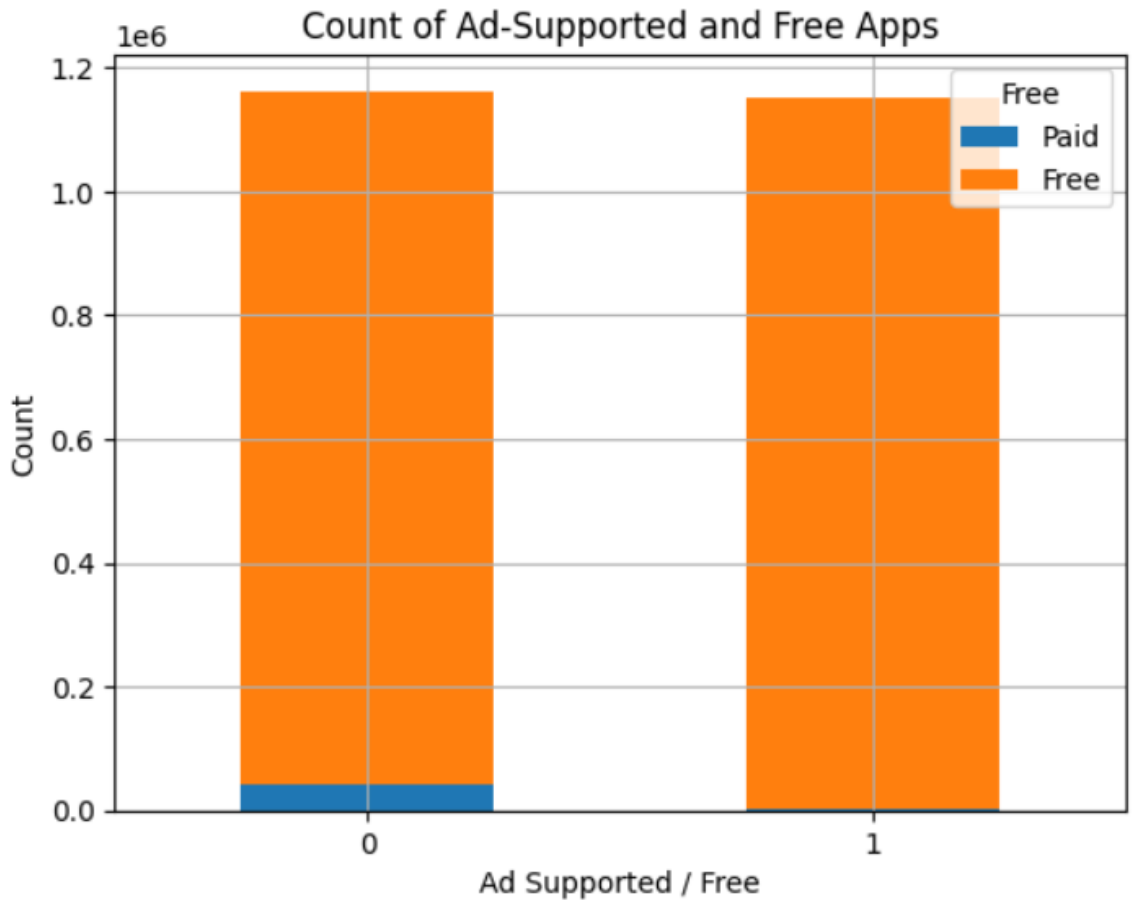
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data1, x='Free', y='Size (MB)', palette='Set1')
plt.title('App Size vs. Free vs. Paid')
plt.xlabel('Free (0 = Paid, 1 = Free)')
plt.ylabel('Size (MB)')
plt.grid(True)
plt.show()
```



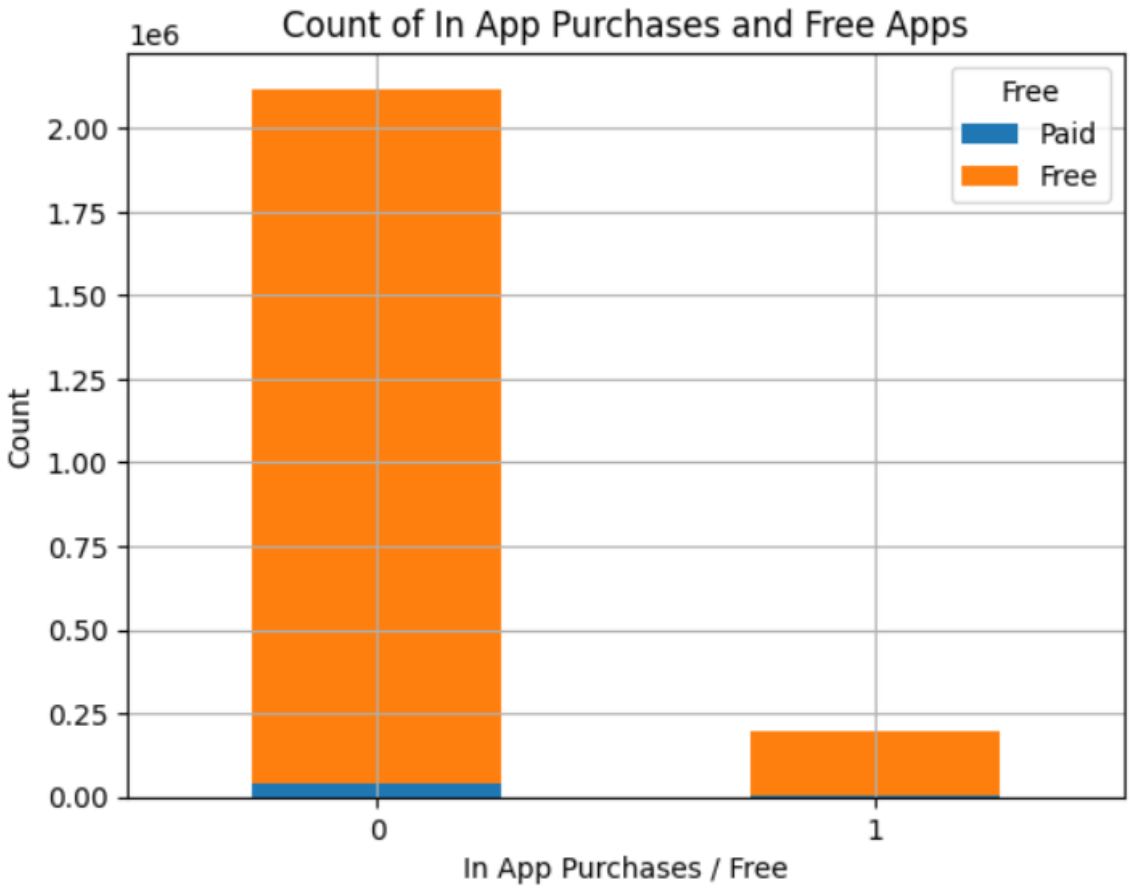

```
plt.figure(figsize=(10, 6))
count_by_content_rating.plot(kind='bar', stacked=True)
plt.title('Count of Free and Paid Apps in Each Content Rating')
plt.xlabel('Content Rating')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.legend(title='Free', loc='upper right', labels=['Paid', 'Free'])
plt.grid(True)
plt.show()
```




```
count_by_attributes = data1.groupby(['Ad Supported', 'Free']).size().unstack(fill_value=0)
plt.figure(figsize=(8, 6))
count_by_attributes.plot(kind='bar', stacked=True)
plt.title('Count of Ad-Supported and Free Apps')
plt.xlabel('Ad Supported / Free')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.legend(title='Free', loc='upper right', labels=['Paid', 'Free'])
plt.grid(True)
plt.show()
```



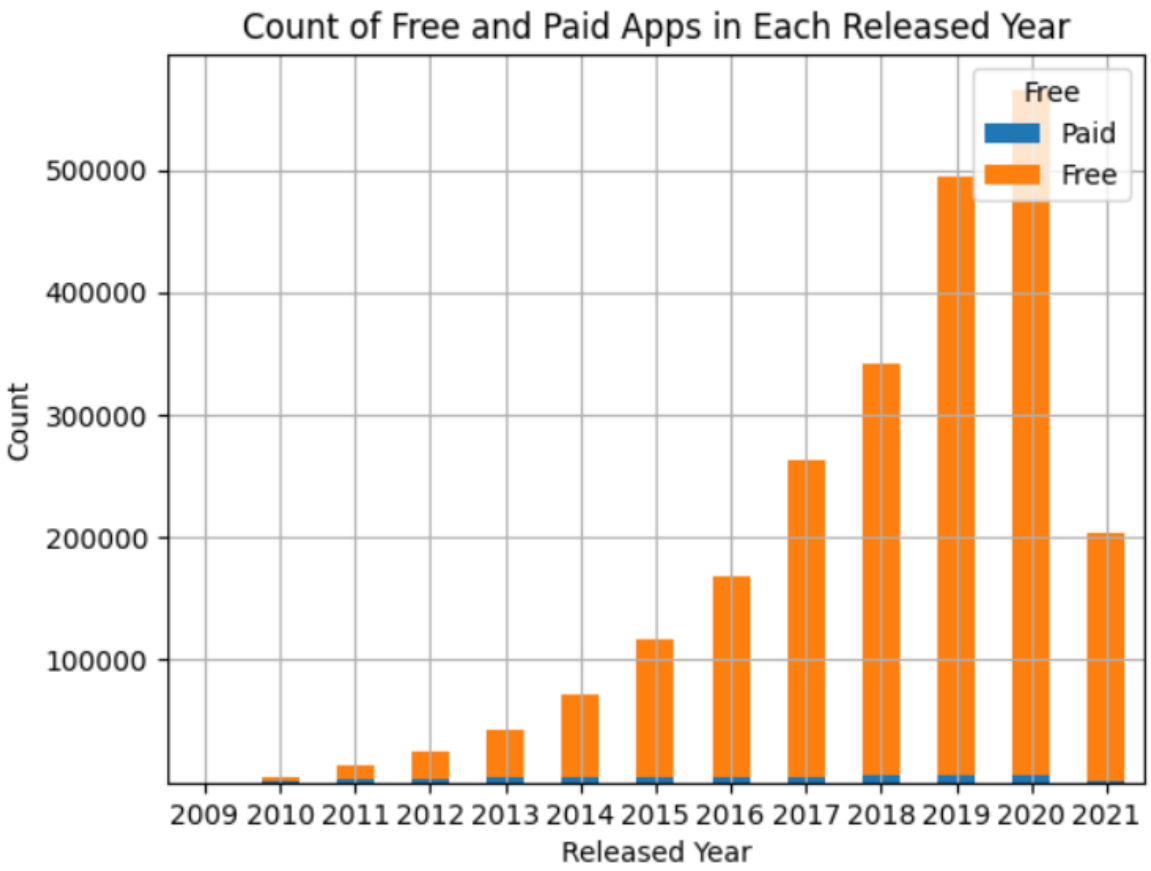

```
count_by_attributes = data1.groupby(['In App Purchases', 'Free']).size().unstack(fill_value=0)
plt.figure(figsize=(8, 6))
count_by_attributes.plot(kind='bar', stacked=True)
plt.title('Count of In App Purchases and Free Apps')
plt.xlabel('In App Purchases / Free')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.legend(title='Free', loc='upper right', labels=['Paid', 'Free'])
plt.grid(True)
plt.show()
```



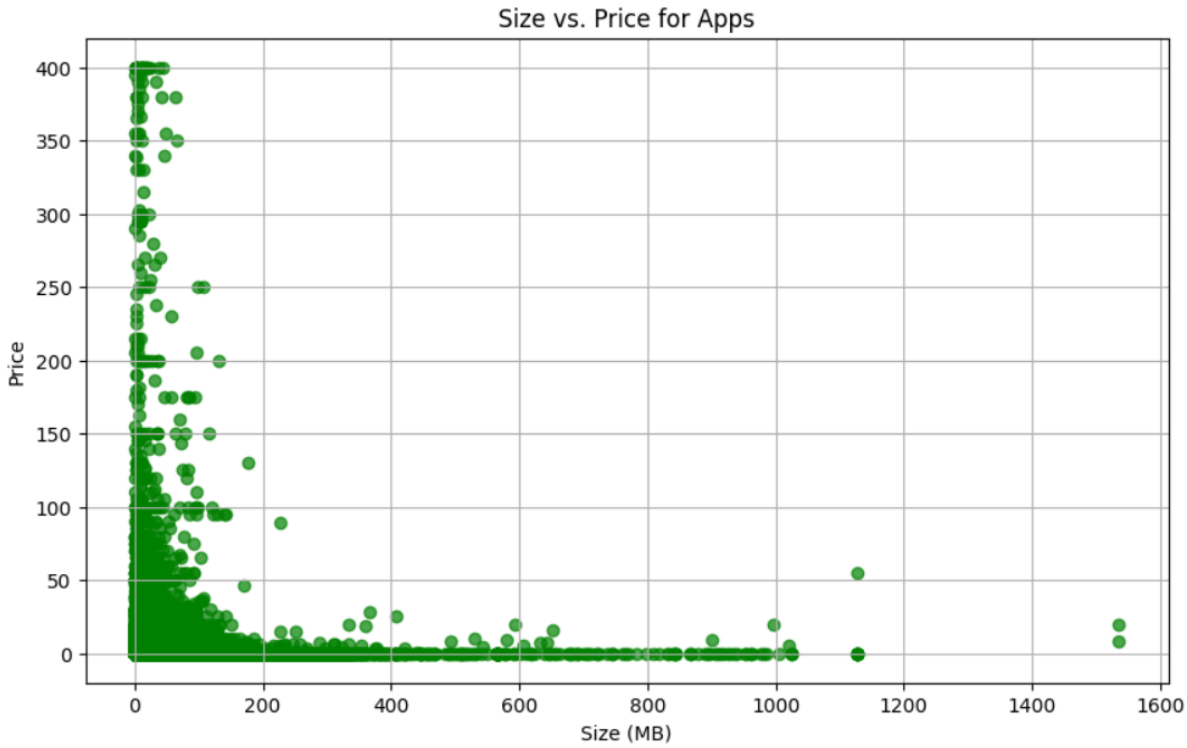

```
count_by_attributes = data1.groupby(['Editors Choice', 'Free']).size().unstack(fill_value=0)
plt.figure(figsize=(8, 6))
count_by_attributes.plot(kind='bar', stacked=True)
plt.title('Count of Editors Choice and Free Apps')
plt.xlabel('Editors Choice / Free')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.legend(title='Free', loc='upper right', labels=['Paid', 'Free'])
plt.grid(True)
plt.show()
```




```
count_by_released_year = df.groupby(['Released_Year', 'Free']).size().unstack(fill_value=0)
plt.figure(figsize=(12, 6))
count_by_released_year.plot(kind='bar', stacked=True)
plt.title('Count of Free and Paid Apps in Each Released Year')
plt.xlabel('Released Year')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.legend(title='Free', loc='upper right', labels=['Paid', 'Free'])
plt.grid(True)
plt.show()
```




```
data1['Price'].unique()
array([ 0. , 1.99 , 4.99 , ..., 3.041816, 26.746362,
 18.903596])
plt.figure(figsize=(10, 6))
plt.scatter(data1['Size (MB)'], data1['Price'], marker='o', color='green', alpha=0.7)
plt.title('Size vs. Price for Apps')
plt.xlabel('Size (MB)')
plt.ylabel('Price')
plt.grid(True)
plt.show()
```



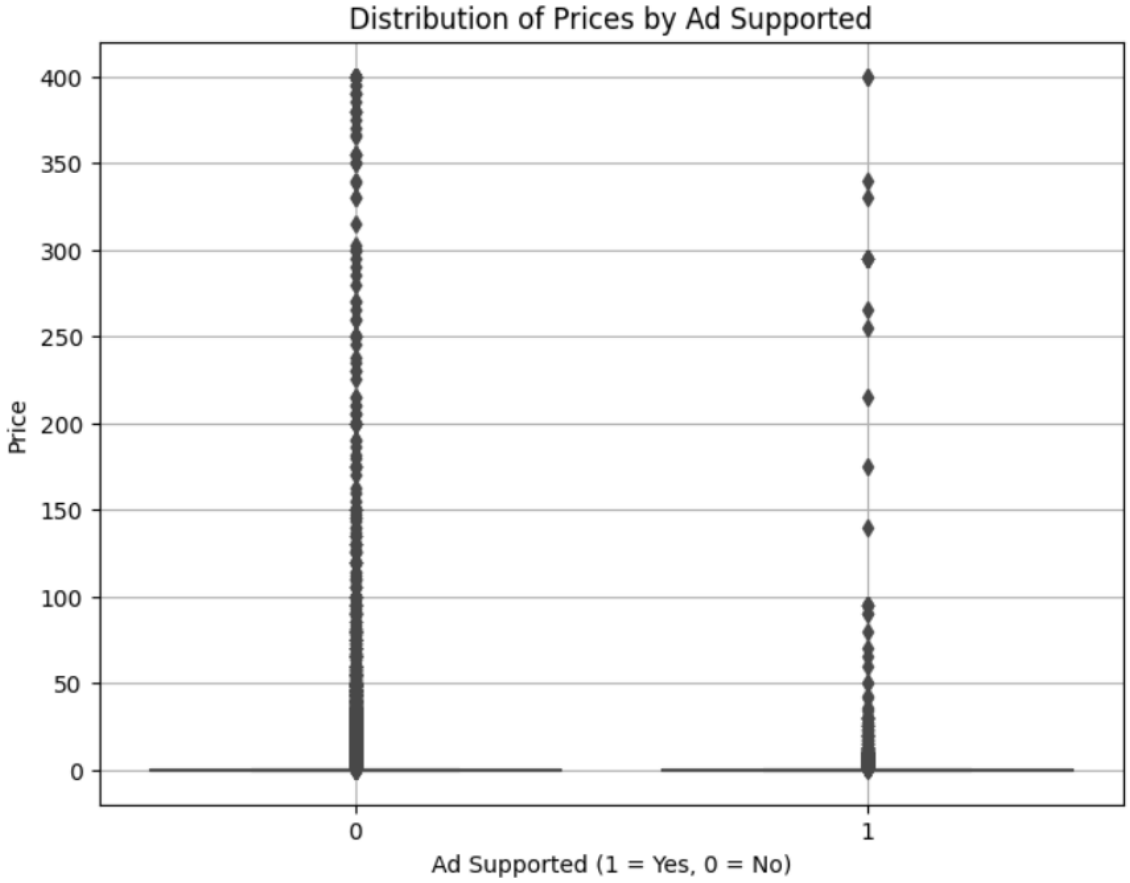

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=data1, x='Content Rating', y='Price', palette='Set1', color = 'red')
plt.title('Distribution of Prices by Content Rating')
plt.xlabel('Content Rating')
plt.ylabel('Price')
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```




```
average_price_by_rating = data1.groupby('Content Rating')['Price'].mean()
print(average_price_by_rating)
print()
print()
max_price_by_rating = data1.groupby('Content Rating')['Price'].max()
print(max_price_by_rating)
```

```
Content Rating
Adults only 18+ 0.000000
Everyone 0.105652
Everyone 10+ 0.208988
Mature 17+ 0.075369
Teen 0.071883
Unrated 0.036169
Name: Price, dtype: float64
Content Rating
Adults only 18+ 0.00
Everyone 400.00
Everyone 10+ 354.99
Mature 17+ 399.99
Teen 399.99
Unrated 2.99
Name: Price, dtype: float64
```

```
plt.figure(figsize=(8, 6))
sns.boxplot(data=data1, x='Ad Supported', y='Price', palette='Set1')
plt.title('Distribution of Prices by Ad Supported')
plt.xlabel('Ad Supported (1 = Yes, 0 = No)')
plt.ylabel('Price')
plt.grid(True)
plt.show()
```



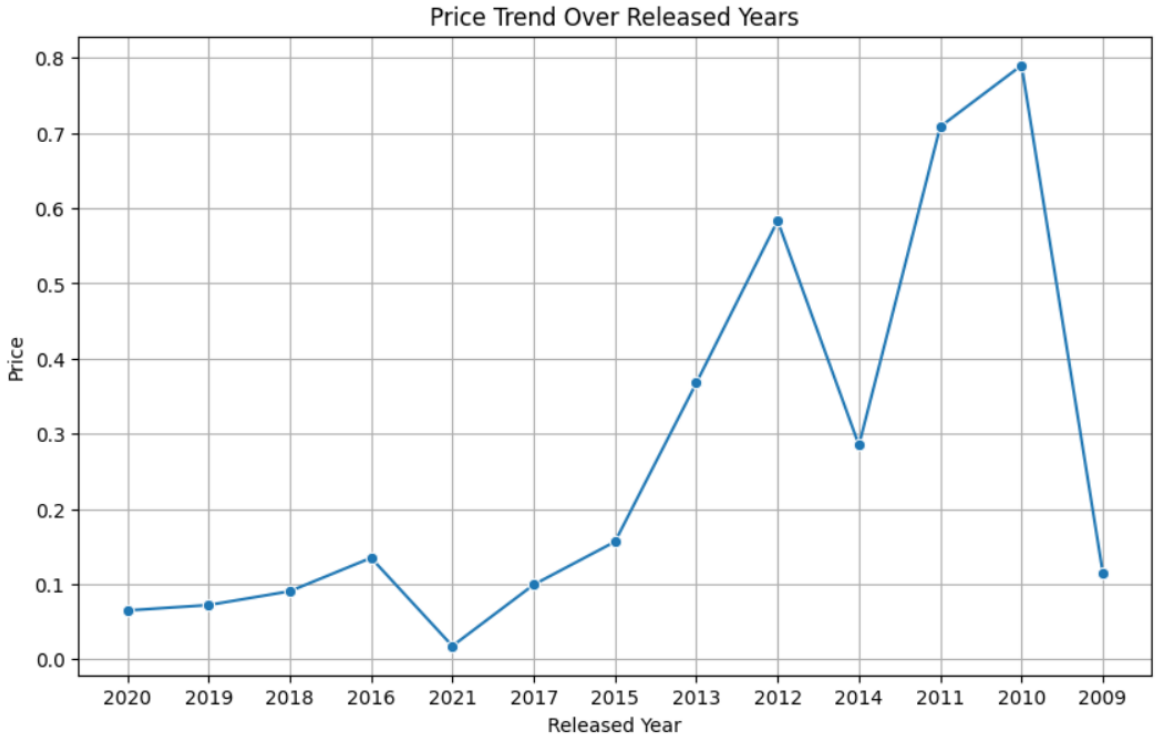

```
plt.figure(figsize=(8, 6))
sns.boxplot(data=data1, x='Editors Choice', y='Price', palette='Set1')
plt.title('Distribution of Prices by Editors Choice')
plt.xlabel('Editors Choice (1 = Yes, 0 = No)')
plt.ylabel('Price')
plt.grid(True)
plt.show()
```



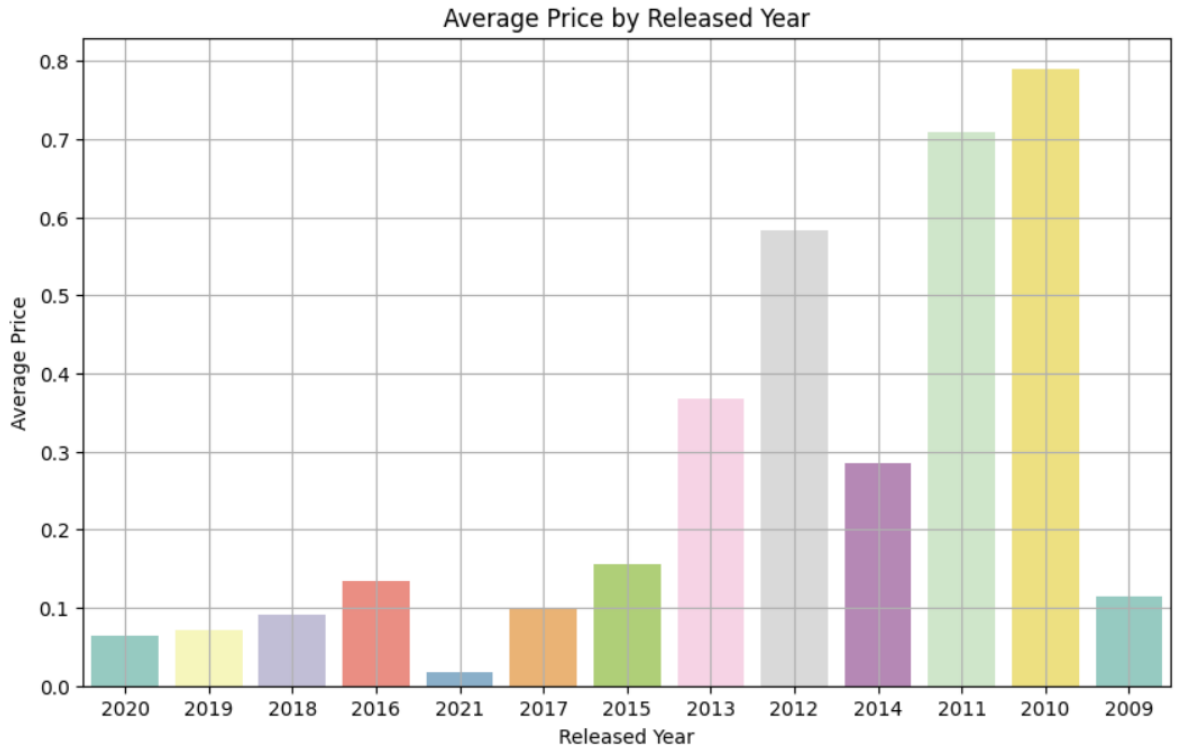

```
plt.figure(figsize=(8, 6))
sns.boxplot(data=data1, x='In App Purchases', y='Price', palette='Set1')
plt.title('Distribution of Prices by In App Purchases')
plt.xlabel('In App Purchases (1 = Yes, 0 = No)')
plt.ylabel('Price')
plt.grid(True)
plt.show()
```



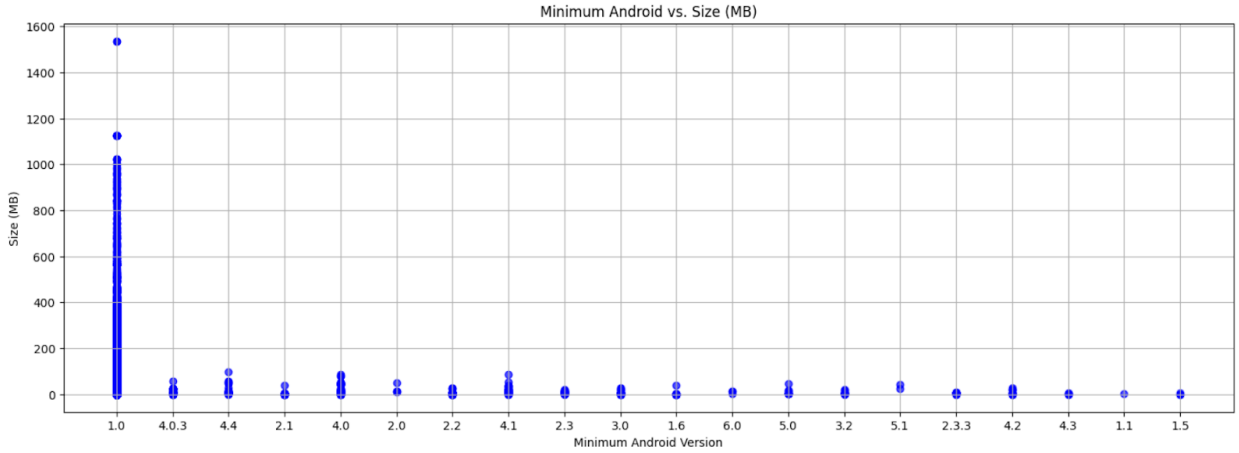

```
plt.figure(figsize=(10, 6))
sns.lineplot(data=data1, x='Released_Year', y='Price', marker='o', ci=None, palette='Set1')
plt.title('Price Trend Over Released Years')
plt.xlabel('Released Year')
plt.ylabel('Price')
plt.grid(True)
plt.show()
<ipython-input-186-7c1b1993fa3c>:2: FutureWarning:
The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.
 sns.lineplot(data=data1, x='Released_Year', y='Price', marker='o', ci=None, palette='Set1')
<ipython-input-186-7c1b1993fa3c>:2: UserWarning: Ignoring `palette` because no `hue`
variable has been assigned.
 sns.lineplot(data=data1, x='Released_Year', y='Price', marker='o', ci=None, palette='Set1')
```




```
plt.figure(figsize=(10, 6))
sns.barplot(data=data1, x='Released_Year', y='Price', ci=None, palette='Set3')
plt.title('Average Price by Released Year')
plt.xlabel('Released Year')
plt.ylabel('Average Price')
plt.grid(True)
plt.show()
```




```
data1['Size (MB)'].unique()
array([1.000e+01, 2.900e+00, 3.700e+00, ..., 4.050e+02, 3.125e-03,
 5.120e+02])
data1['Size'].unique()
array(['10M', '2.9M', '3.7M', ..., '405M', '3.2k', '512M'], dtype=object)
plt.figure(figsize=(18, 6))
plt.scatter(data1['Minimum Android'], data1['Size (MB)'], marker='o', color='blue', alpha=0.7)
plt.title('Minimum Android vs. Size (MB)')
plt.xlabel('Minimum Android Version')
plt.ylabel('Size (MB)')
plt.grid(True)
plt.show()
```

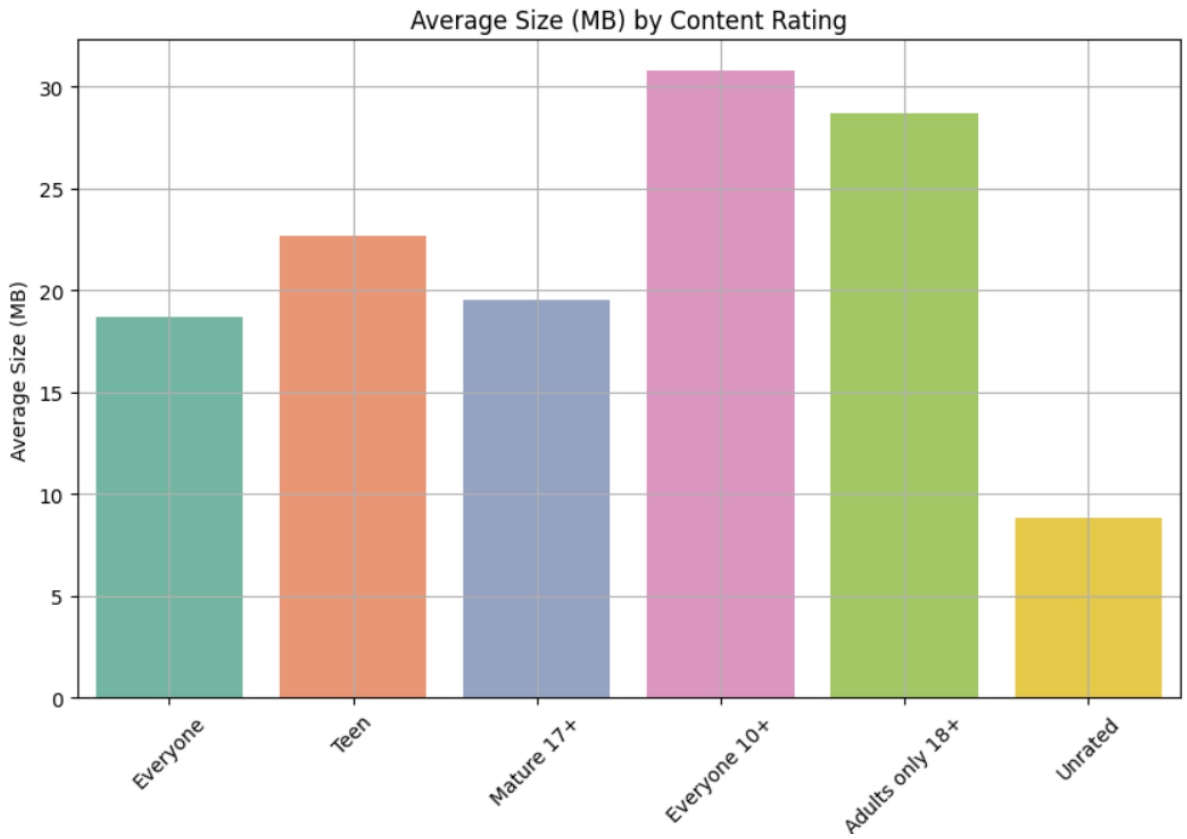



```
average_size_by_android = data1.groupby('Minimum Android')['Size (MB)'].mean()
print(average_size_by_android)
```

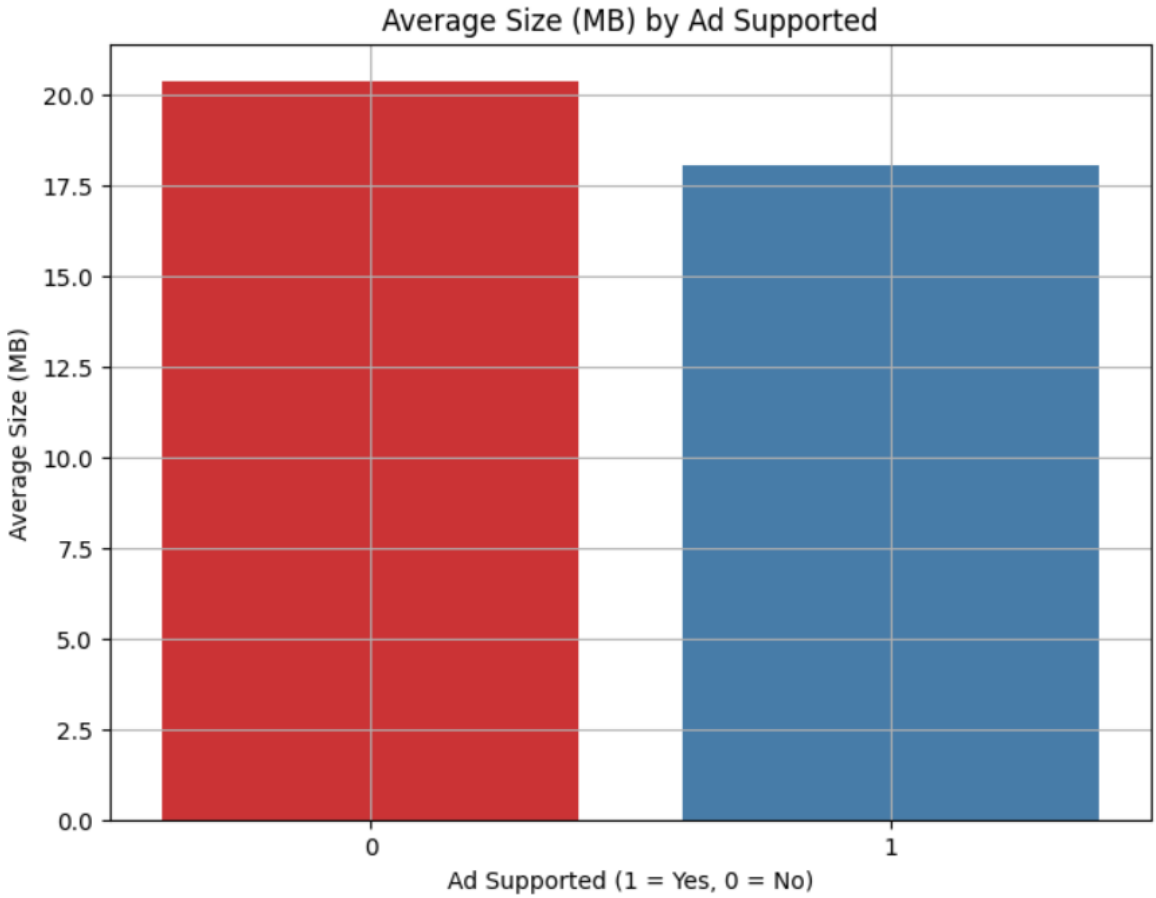
```
Minimum Android
1.0 19.210039
1.1 2.800000
1.5 2.626172
1.6 3.513266
2.0 22.425000
2.1 3.468906
2.2 3.906222
2.3 5.267055
2.3.3 4.029340
3.0 6.176426
3.2 6.110511
4.0 15.724814
4.0.3 10.250898
4.1 11.525651
4.2 9.959197
4.3 2.078164
4.4 20.565681
5.0 10.536364
5.1 34.000000
6.0 10.366667
Name: Size (MB), dtype: float64
```

```
plt.figure(figsize=(10, 6))
sns.barplot(data=data1, x='Content Rating', y='Size (MB)', ci=None, palette='Set2')
plt.title('Average Size (MB) by Content Rating')
plt.xlabel('Content Rating')
plt.ylabel('Average Size (MB)')
plt.xticks(rotation=45)
plt.grid(True)
```

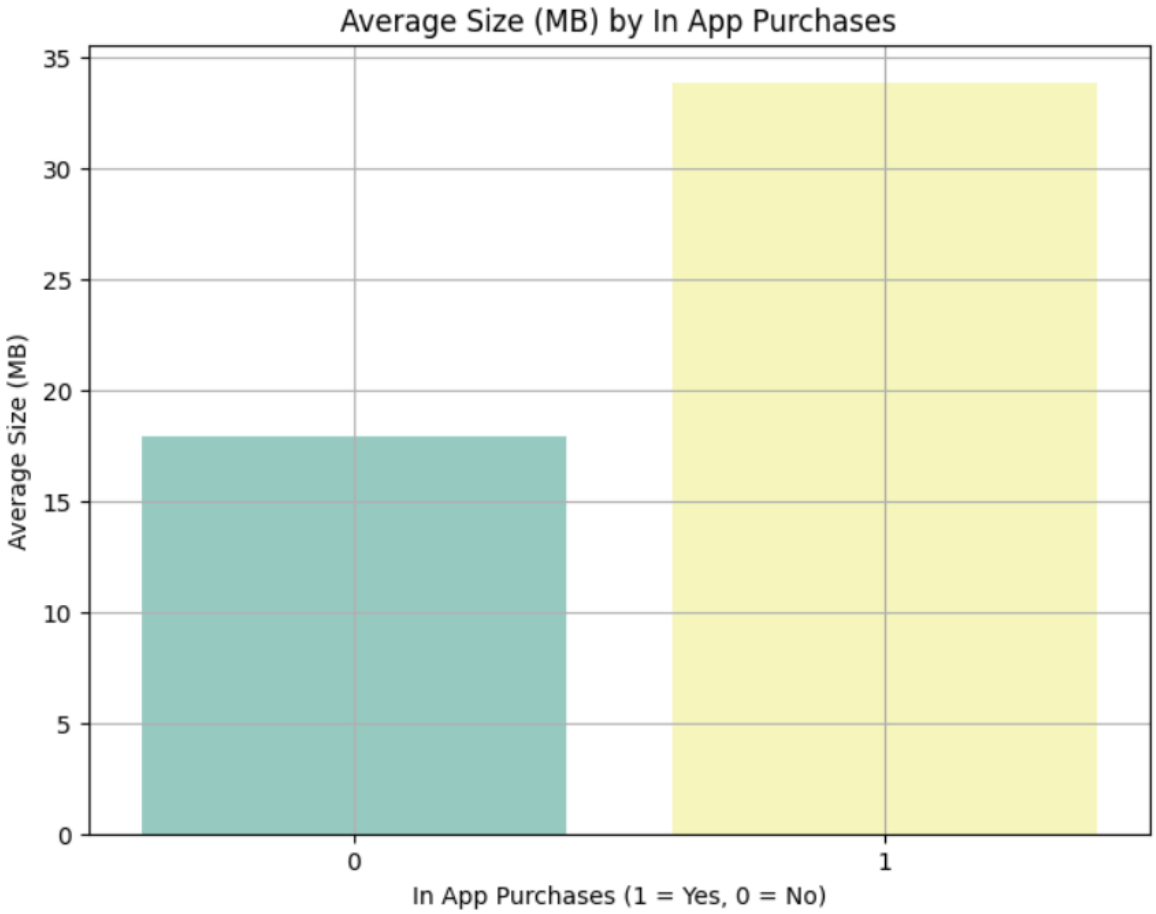
plt.show()



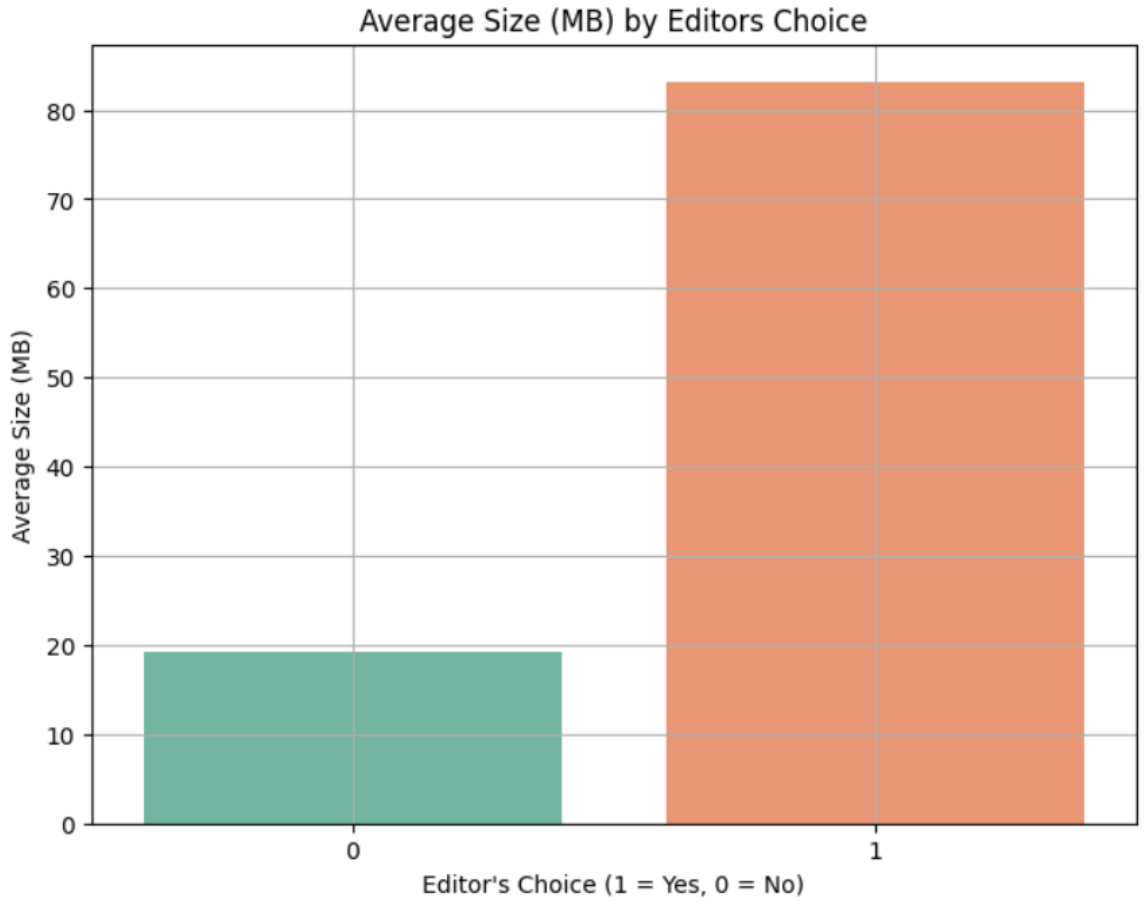

```
plt.figure(figsize=(8, 6))
sns.barplot(data=data1, x='Ad Supported', y='Size (MB)', ci=None, palette='Set1')
plt.title('Average Size (MB) by Ad Supported')
plt.xlabel('Ad Supported (1 = Yes, 0 = No)')
plt.ylabel('Average Size (MB)')
plt.grid(True)
plt.show()
```



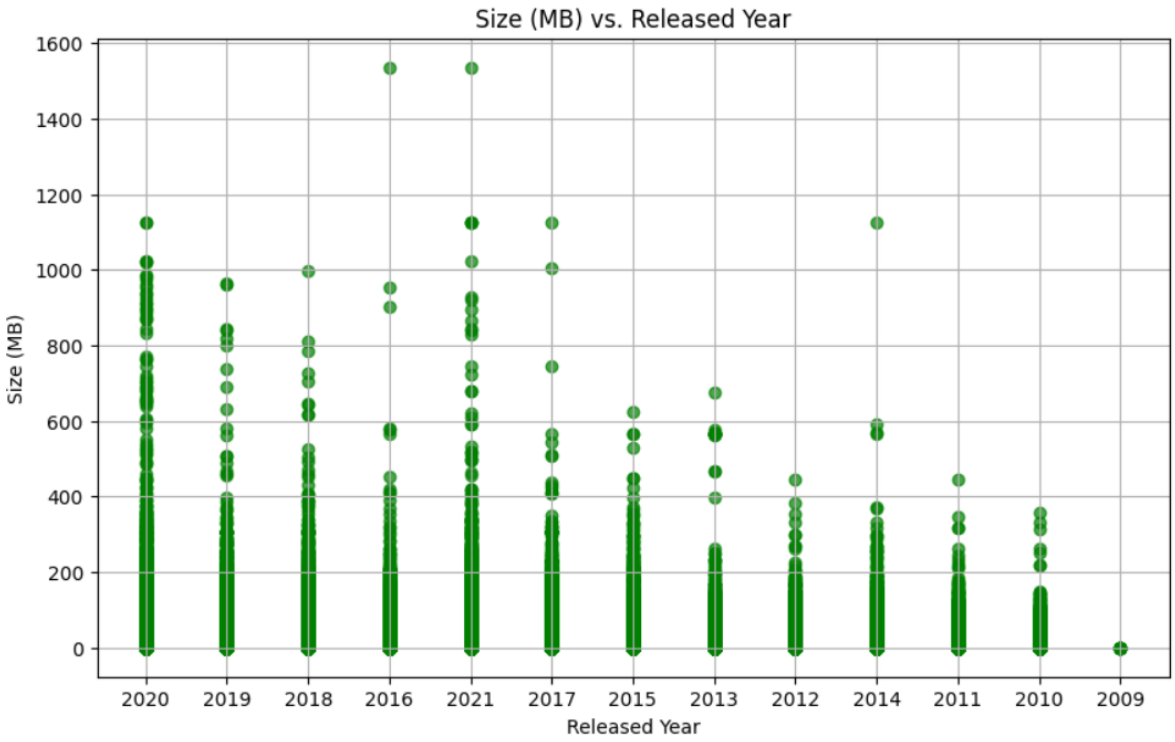

```
plt.figure(figsize=(8, 6))
sns.barplot(data=data1, x='In App Purchases', y='Size (MB)', ci=None, palette='Set3')
plt.title('Average Size (MB) by In App Purchases')
plt.xlabel('In App Purchases (1 = Yes, 0 = No)')
plt.ylabel('Average Size (MB)')
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(8, 6))
sns.barplot(data=data1, x='Editors Choice', y='Size (MB)', ci=None, palette='Set2')
plt.title('Average Size (MB) by Editors Choice')
plt.xlabel("Editor's Choice (1 = Yes, 0 = No)")
plt.ylabel('Average Size (MB)')
plt.grid(True)
plt.show()
```

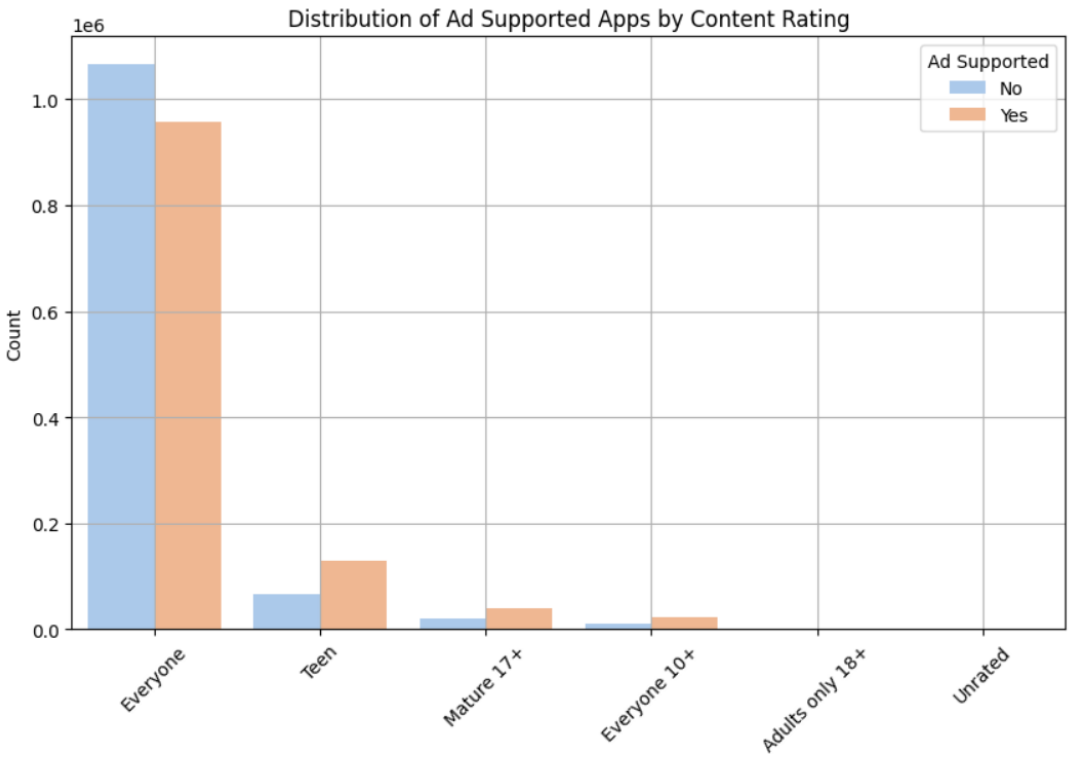



```
plt.figure(figsize=(10, 6))
plt.scatter(data1['Released_Year'], data1['Size (MB)'], marker='o', color='green', alpha=0.7)
plt.title('Size (MB) vs. Released Year')
plt.xlabel('Released Year')
plt.ylabel('Size (MB)')
plt.grid(True)
plt.show()
```

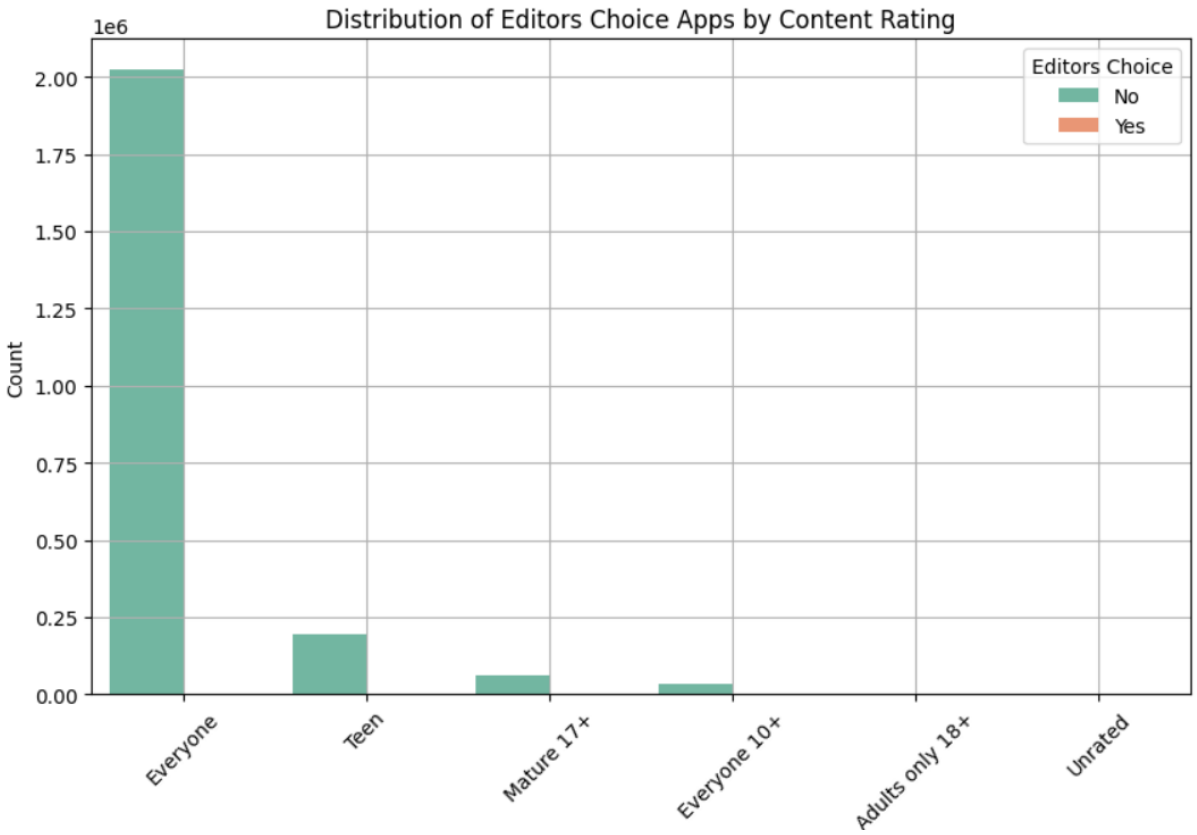



```
data1['Content Rating'].unique()
array(['Everyone', 'Teen', 'Mature 17+', 'Everyone 10+',
 'Adults only 18+', 'Unrated'], dtype=object)
```

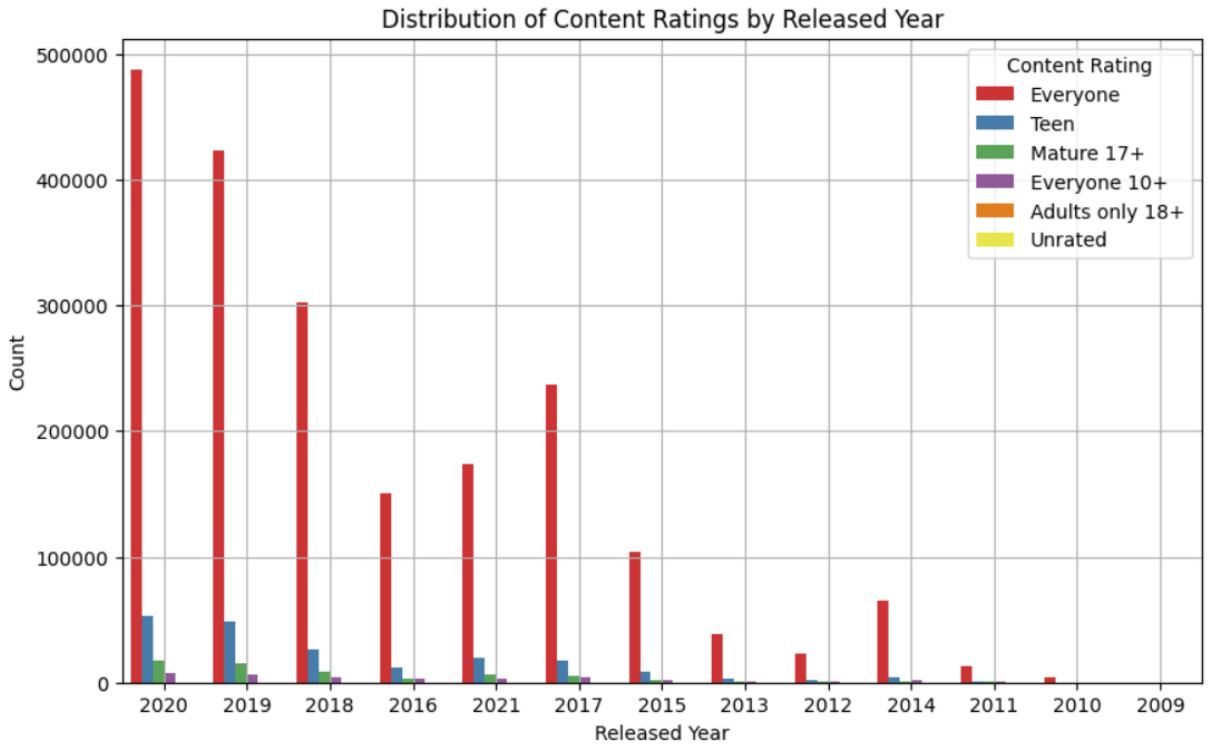
```
plt.figure(figsize=(10, 6))
sns.countplot(data=data1, x='Content Rating', hue='Ad Supported', palette='pastel')
plt.title('Distribution of Ad Supported Apps by Content Rating')
plt.xlabel('Content Rating')
plt.ylabel('Count')
plt.legend(title='Ad Supported', labels=['No', 'Yes'])
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```



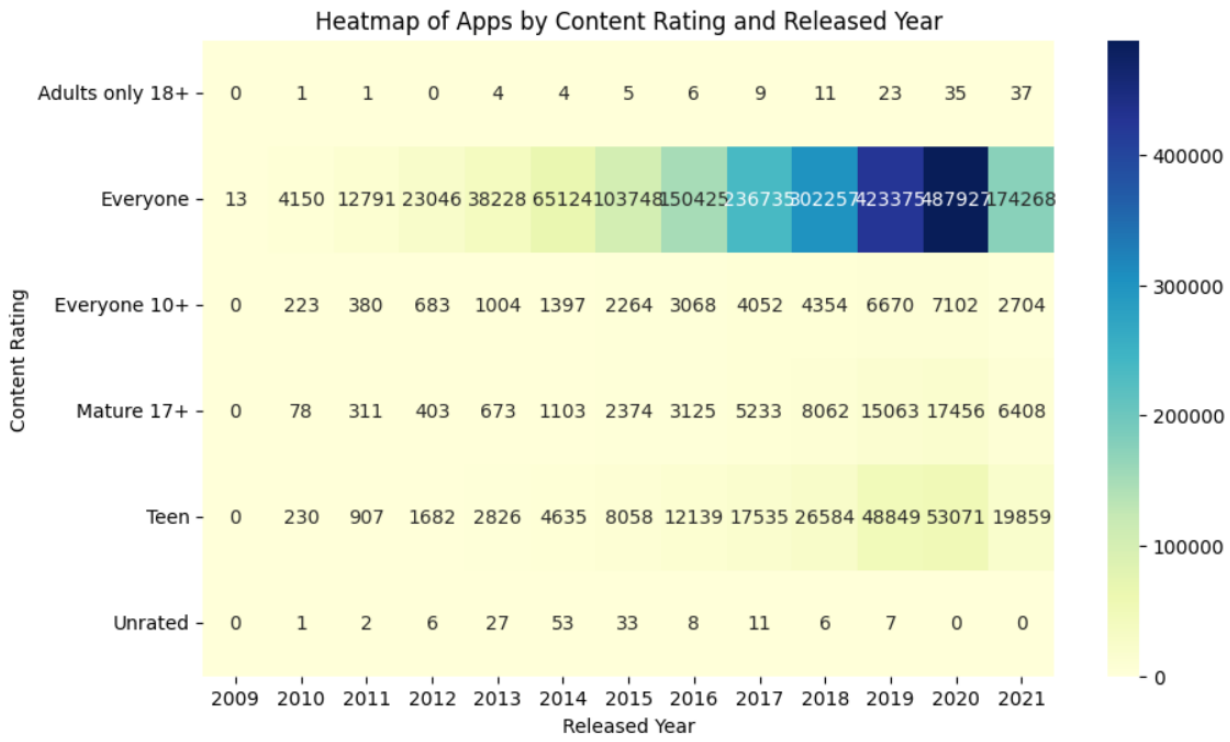

```
plt.figure(figsize=(10, 6))
sns.countplot(data=data1, x='Content Rating', hue='Editors Choice', palette='Set2')
plt.title('Distribution of Editors Choice Apps by Content Rating')
plt.xlabel('Content Rating')
plt.ylabel('Count')
plt.legend(title='Editors Choice', labels=['No', 'Yes'])
plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```



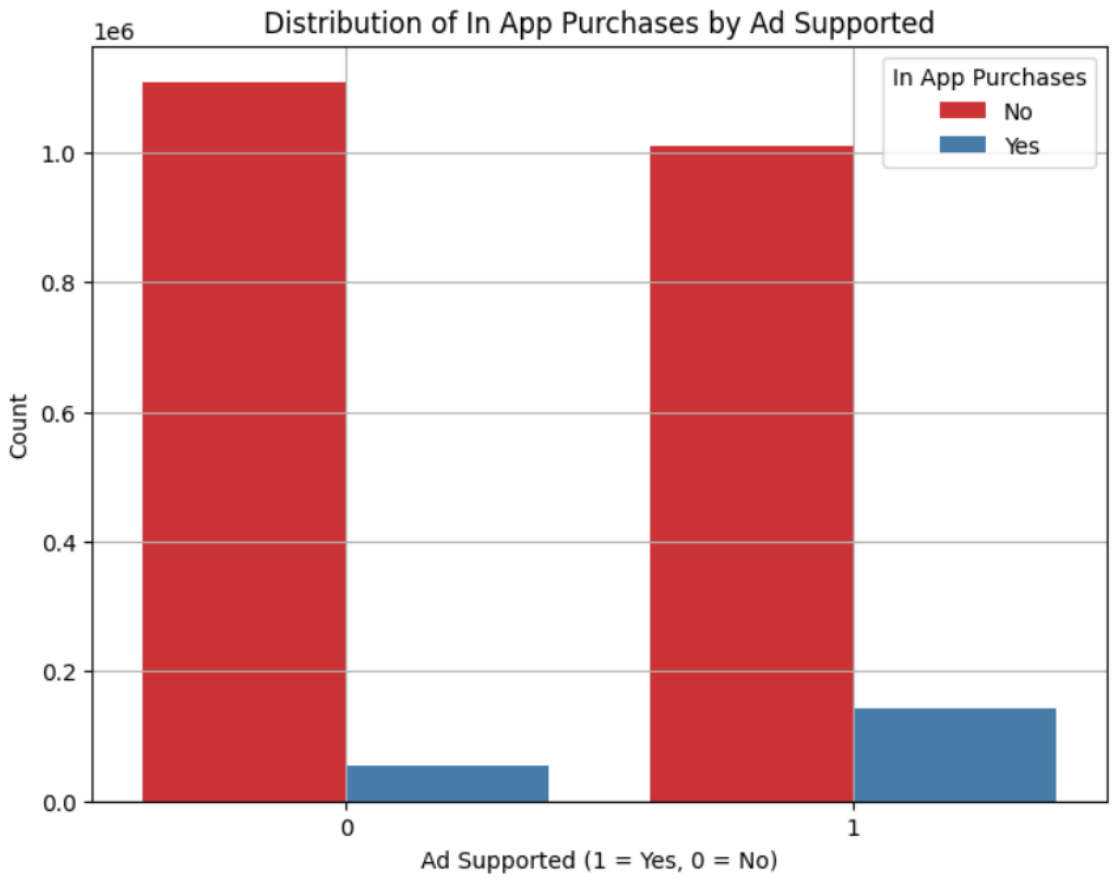

```
plt.figure(figsize=(10, 6))
sns.countplot(data=data1, x='Released_Year', hue='Content Rating', palette='Set1')
plt.title('Distribution of Content Ratings by Released Year')
plt.xlabel('Released Year')
plt.ylabel('Count')
plt.legend(title='Content Rating')
plt.grid(True)
plt.show()
```



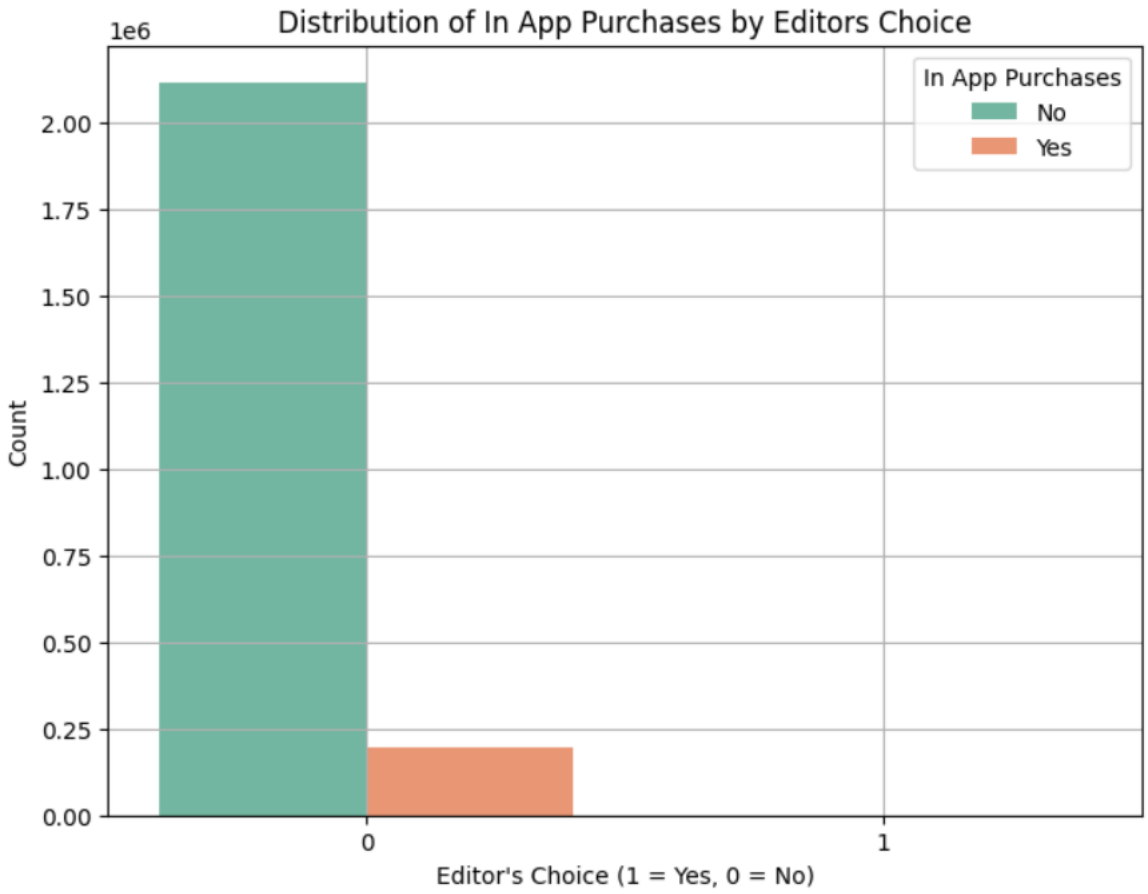

```
pivot_table = data1.pivot_table(index='Content Rating', columns='Released_Year',
aggfunc='size', fill_value=0)
plt.figure(figsize=(10, 6))
sns.heatmap(pivot_table, annot=True, fmt='d', cmap='YlGnBu')
plt.title('Heatmap of Apps by Content Rating and Released Year')
plt.xlabel('Released Year')
plt.ylabel('Content Rating')
plt.show()
```



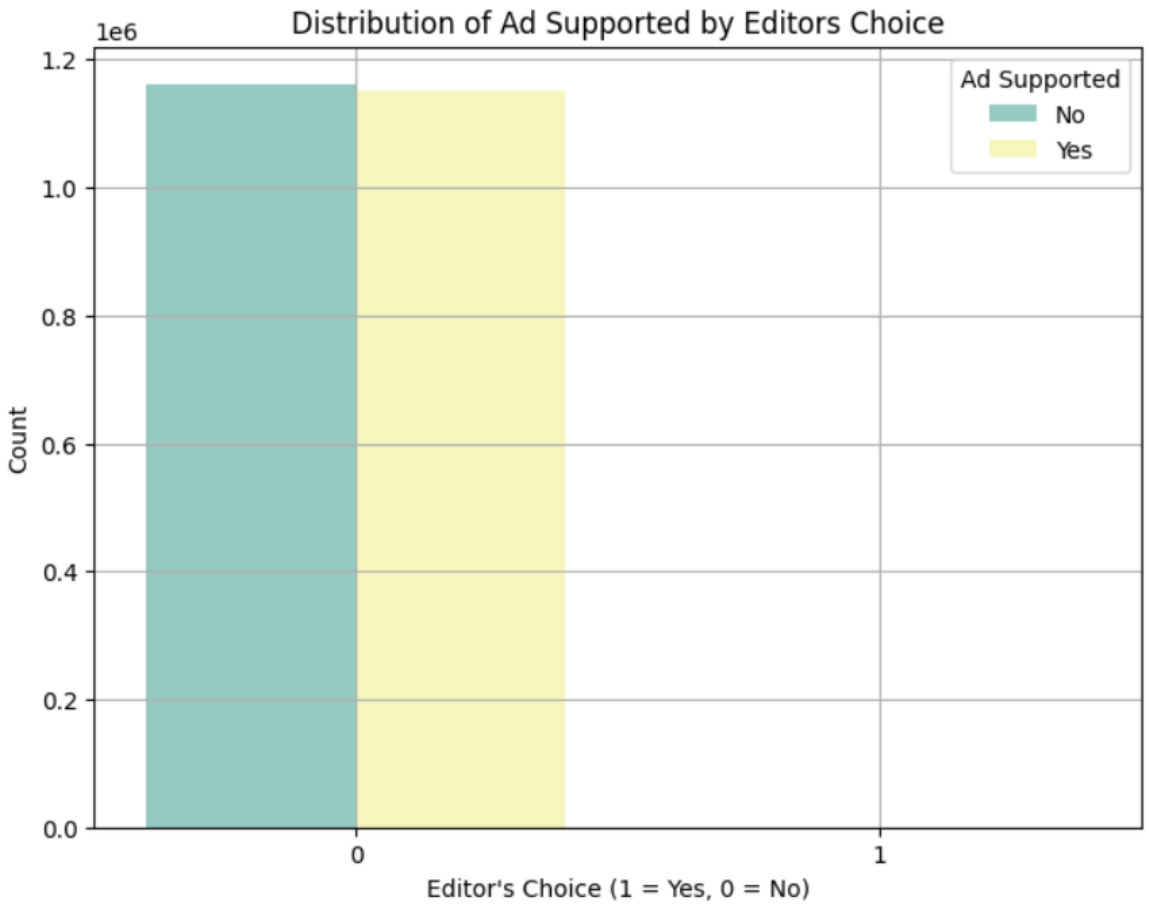

```
plt.figure(figsize=(8, 6))
sns.countplot(data=data1, x='Ad Supported', hue='In App Purchases', palette='Set1')
plt.title('Distribution of In App Purchases by Ad Supported')
plt.xlabel('Ad Supported (1 = Yes, 0 = No)')
plt.ylabel('Count')
plt.legend(title='In App Purchases', labels=['No', 'Yes'])
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(8, 6))
sns.countplot(data=data1, x='Editors Choice', hue='In App Purchases', palette='Set2')
plt.title('Distribution of In App Purchases by Editors Choice')
plt.xlabel("Editor's Choice (1 = Yes, 0 = No)")
plt.ylabel('Count')
plt.legend(title='In App Purchases', labels=['No', 'Yes'])
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(8, 6))
sns.countplot(data=data1, x='Editors Choice', hue='Ad Supported', palette='Set3')
plt.title('Distribution of Ad Supported by Editors Choice')
plt.xlabel("Editor's Choice (1 = Yes, 0 = No)")
plt.ylabel('Count')
plt.legend(title='Ad Supported', labels=['No', 'Yes'])
plt.grid(True)
plt.show()
```


Distribution of Ad Supported by Editors Choice
1e6
1.2
1.0
0.8
0.6
0.4
0.2
0.0
Count
0
1
Editor's Choice (1 = Yes, 0 = No)
Ad Supported
No
Yes